

MICROSOFT BLAZOR • A PRACTICAL GUIDE

Blazor from Zero to Production

A Practical, Project-Based Guide

From your first component to a deployed, tested,
continuously-delivered application

2026

Contents

Preface	1
Who this book is for	1
How this book is organized	1
How to use the code examples	2
 I Foundations	 3
1 Welcome to Blazor	5
1.1 What problem does Blazor solve?	5
1.2 The three ways Blazor can run	5
1.3 Where Blazor sits in the .NET web landscape	7
1.4 What you will build	7
1.5 A note on .NET versions	7
 2 Setting Up Your Development Environment	 9
2.1 Installing the .NET SDK	9
2.2 Choosing an editor	10
2.3 Creating your first project	10
2.4 Anatomy of the generated project	11
2.5 Command-line vs. IDE workflow	13
2.6 Try it yourself	14
 3 Razor Components 101	 15
3.1 What a component actually is	15
3.2 Mixing markup and C#: the @ symbol	15
3.3 Adding a code-behind section with @code	16
3.4 Splitting markup from code entirely: partial classes	17
3.5 Directives you'll use constantly	18
3.6 Global imports with <code>_Imports.razor</code>	19
3.7 Try it yourself	20

4	Data Binding and Events	21
4.1	One-way binding: from field to markup	21
4.2	Two-way binding with @bind	21
4.3	Handling events beyond click	23
4.4	Preventing default behavior and stopping propagation	24
4.5	Building a small interactive list: your first real TaskBoard feature	25
4.6	Try it yourself	26
5	Component Parameters and Composition	29
5.1	Breaking the task list into components	29
5.2	[Parameter] vs. plain properties	31
5.3	Required parameters	32
5.4	EventCallback: letting a child talk to its parent	32
5.5	ChildContent: components that wrap other markup	33
5.6	Templated components: parameterizing <i>how</i> content renders	34
5.7	Cascading values: skipping the middleman	35
5.8	Try it yourself	36
6	The Component Lifecycle	37
6.1	Why lifecycle methods exist	37
6.2	The full sequence	37
6.3	OnInitialized — one-time setup	38
6.4	OnParametersSet — reacting to new parameter values	39
6.5	OnAfterRender — after the DOM has actually updated	40
6.6	StateHasChanged — telling Blazor “something changed, please re-render”	41
6.7	Cleaning up with IDisposable	42
6.8	A complete mental model	43
6.9	Try it yourself	43
II	Building Applications	45
7	Routing and Navigation	47
7.1	How @page becomes a route table	47
7.2	Route parameters	47
7.3	NavigationManager: navigating from code	49
7.4	Reading query strings	50
7.5	Layouts: the shared page frame	50

7.6	Building the navigation menu	51
7.7	A worked example: task list, detail, and create pages wired together	52
7.8	Try it yourself	53
8	Forms and Validation	55
8.1	Why EditForm instead of a plain <form>	55
8.2	The built-in input components	57
8.3	Custom validation logic	58
8.4	Manual validation with EditContext directly	58
8.5	OnValidSubmit VS. OnInvalidSubmit VS. OnSubmit	59
8.6	Building a reusable form field component	60
8.7	Try it yourself	60
9	Dependency Injection and Services	63
9.1	Why services instead of code directly in components	63
9.2	Defining a service	63
9.3	Registering the service	65
9.4	Consuming a service in a component: @inject	66
9.5	Constructor injection in non-component classes	67
9.6	Injecting framework services you'll use constantly	68
9.7	Scoping services per feature area	68
9.8	Try it yourself	68
10	Calling APIs and Working with Data	71
10.1	Setting up HttpClient	71
10.2	Replacing the in-memory service with a real HTTP-backed one	72
10.3	Handling loading, success, and error states	74
10.4	Cancellation for components that navigate away mid-request .	75
10.5	Configuring resilience with HttpClientFactory policies	76
10.6	Try it yourself	77
11	Working with a Real Database: EF Core	79
11.1	Where Entity Framework Core fits	79
11.2	Defining the data model	79
11.3	The DbContext	80
11.4	Migrations: evolving the schema safely	81
11.5	Building the API endpoints	82

11.6	Avoiding the N+1 query trap	84
11.7	Try it yourself	85
12	State Management	87
12.1	Three kinds of state, three different lifetimes	87
12.2	The simplest shared state: a scoped service	87
12.3	Avoiding memory leaks with shared state	89
12.4	When to reach for a dedicated state-management library	89
12.5	Persisting state across a page reload	90
12.6	Try it yourself	91
13	JavaScript Interop and CSS Isolation	93
13.1	Why you sometimes still need JavaScript	93
13.2	Calling JavaScript from C#: IJSRuntime	93
13.3	Preferring JS modules over global scripts	94
13.4	Working with DOM elements directly: ElementReference	96
13.5	Calling C# from JavaScript	96
13.6	CSS isolation: styles that stay with their component	98
13.7	Try it yourself	99
14	Render Modes in Depth	101
14.1	Recap: what a render mode actually controls	101
14.2	Static server rendering (the default, if you specify nothing) . .	101
14.3	Interactive Server	101
14.4	Interactive WebAssembly	102
14.5	Interactive Auto	103
14.6	Where to apply render modes: per-component vs. globally . .	104
14.7	Prerendering: getting a fast first paint even in interactive modes	104
14.8	A decision guide	105
14.9	Try it yourself	105
15	Building a Component Library and Design System	107
15.1	The problem a growing app runs into	107
15.2	Starting with primitives: Button	107
15.3	A small set of primitives goes a long way	108
15.4	Design tokens: one source of truth for colors and spacing . . .	109
15.5	Documenting the library with a living style guide page	110
15.6	Packaging the library as a separate project (when it's worth it)	111

15.7 Try it yourself	112
III Advanced Topics	113
16 Authentication and Authorization	115
16.1 The core abstractions	115
16.2 Guarding a page with [Authorize]	115
16.3 Conditionally showing markup with <AuthorizeView>	115
16.4 Setting up ASP.NET Core Identity (Blazor Server / cookie-based)	116
16.5 Authentication in Blazor WebAssembly: token-based	117
16.6 Custom authorization policies	118
16.7 Securing the API too — never trust the client alone	119
16.8 Try it yourself	119
17 Performance and Optimization	121
17.1 Measure before you optimize	121
17.2 Reducing unnecessary re-renders	121
17.3 @key: helping Blazor's diffing algorithm	122
17.4 Virtualization for long lists	122
17.5 WebAssembly-specific: trimming and AOT compilation	123
17.6 Server-specific: circuit and memory management	125
17.7 Try it yourself	126
18 Testing Blazor Applications	127
18.1 Why testing Blazor doesn't feel like testing plain C	127
18.2 The testing pyramid, applied to Blazor	127
18.3 Unit testing plain services first	128
18.4 Component testing with bUnit	129
18.5 End-to-end testing with Playwright	132
18.6 Where each kind of test earns its cost	133
18.7 Try it yourself	134
19 Real-Time Features with SignalR	135
19.1 Why Blazor and SignalR are already old friends	135
19.2 Setting up a SignalR hub	135
19.3 Connecting from a Blazor component	136
19.4 Broadcasting to specific groups	138
19.5 Pushing updates from outside any hub method	139

19.6 Try it yourself	140
20 Accessibility and Internationalization	141
20.1 Why this belongs in a book about building, not just a checklist at the end	141
20.2 Semantic HTML first	141
20.3 Labeling form inputs correctly	142
20.4 Managing focus after dynamic UI changes	142
20.5 Announcing dynamic content with live regions	143
20.6 Testing accessibility isn't optional-extra work	144
20.7 Internationalization: preparing for more than one language . .	145
20.8 Try it yourself	146
IV Industry Practice	147
21 Architecture and Project Structure	149
21.1 Why a single project stops being enough	149
21.2 A typical solution layout	149
21.3 The dependency rule	150
21.4 Why this is worth the overhead	151
21.5 Where Blazor-specific code fits into this	151
21.6 Feature folders vs. technical folders	152
21.7 A note on TaskBoard.Shared and Blazor Auto mode	152
21.8 Try it yourself	152
22 Git Workflow and Code Review	155
22.1 Branching strategy	155
22.2 Commit message conventions	156
22.3 What a good pull request looks like	156
22.4 What to actually look for reviewing a Blazor PR	157
22.5 Review etiquette that keeps a team functional	158
22.6 Try it yourself	158
23 CI/CD for Blazor	159
23.1 What CI/CD actually buys you	159
23.2 A complete GitHub Actions pipeline	159
23.3 Adding static analysis and formatting checks	161
23.4 Adding a Blazor-specific quality gate: bundle size	162

23.5 Continuous deployment: shipping automatically on merge . . .	163
23.6 Environment-specific configuration	164
23.7 Try it yourself	164
24 Deployment and Hosting	165
24.1 Matching hosting to render mode	165
24.2 Publishing	165
24.3 Deploying to Azure App Service	166
24.4 Deploying with Docker	166
24.5 Deploying Blazor WebAssembly as static files	168
24.6 Deploying to IIS (Windows Server environments)	169
24.7 A checklist before your first production deployment	169
24.8 Try it yourself	170
25 Capstone Project and Next Steps	171
25.1 Putting it all together: TaskBoard, end to end	171
25.2 What this book didn't cover	172
25.3 Where to go next	173
25.4 A closing thought	173
A C# Refresher for Blazor	175
A.1 <code>async/await</code> , in the shape Blazor uses it	175
A.2 Nullable reference types	175
A.3 Records, for immutable data	176
A.4 Pattern matching	176
A.5 LINQ, the querying vocabulary you'll use constantly	177
A.6 Generics, briefly	177
A.7 Extension methods	178
B Razor Syntax Cheat Sheet	179
B.1 Markup and expressions	179
B.2 Directives	179
B.3 Binding and events	180
B.4 Component parameters	181
B.5 Lifecycle methods, in call order	182
B.6 Built-in form input components	182
B.7 Common injectable framework services	182
B.8 Route constraints	183

C	Troubleshooting Common Errors	185
C.1	“Cannot provide a value for property... There is no registered service of type...”	185
C.2	A page shows stale data after clicking a link to “the same” page with a different ID	185
C.3	InvalidOperationException: The current thread is not associated with the Dispatcher	186
C.4	Clicking a button does nothing, with no error at all	186
C.5	A MissingMethodException appears only in a published Release build, never in Debug	186
C.6	Forms: EditForm submits, but validation messages never show up	187
C.7	NavigationException or a blank page after calling <code>NavigationManager.NavigateTo</code>	187
C.8	Blazor Server: the app “freezes” after being idle in a background browser tab	188
C.9	dotnet ef migrations add fails with “Unable to create an object of type ‘ApplicationDbContext’”	188
C.10	A bUnit test throws JSInteropException immediately on render	189
C.11	CI passes locally with dotnet test but fails only in GitHub Actions	189
D	Glossary	191
E	Further Practice Projects	195
E.1	Project 1: Habit Tracker (reinforces Parts I–II)	195
E.2	Project 2: Shared Grocery List (reinforces Parts II–III)	195
E.3	Project 3: Personal Finance Dashboard (reinforces Parts III–IV, full stack)	196
E.4	A note on scope	196
F	Further Reading and Resources	199
F.1	Official documentation	199
F.2	Component libraries worth evaluating before building your own	199
F.3	Testing tools referenced in this book	200
F.4	On architecture and broader software practice	200
F.5	Staying current	200

Preface

Blazor is Microsoft’s framework for building interactive web user interfaces using C# instead of JavaScript. If you already know C#, you already know most of what you need to build modern, component-based web applications — no separate front-end language, no context switching between two ecosystems, no duplicated validation logic between client and server.

This book takes you from a completely empty folder to a production-shaped Blazor application, and then keeps going into the things that separate a tutorial project from a professional one: automated testing, code review discipline, and a CI/CD pipeline that ships your work safely.

Who this book is for

You should be comfortable with C# basics (classes, methods, `async/await`, generics) and have written some HTML and CSS at some point, even briefly. You do **not** need any prior experience with:

- Blazor, Razor, or any other Microsoft web framework
- JavaScript frameworks like React, Vue, or Angular
- Modern DevOps tooling

Every chapter builds on the previous one. Concepts are never used before they are explained, and every non-trivial idea is backed by a runnable example.

How this book is organized

Part I — Foundations gets a Blazor project running on your machine and builds your mental model of components, markup, data binding, parameters, and the component lifecycle. By the end of Part I you can build a static, interactive single-page UI.

Part II — Building Applications adds the pieces every real application needs: routing, forms and validation, dependency injection, calling HTTP APIs, managing state across components, JavaScript interop, and — because it changed significantly in modern .NET — a full explanation of Blazor’s render modes (Server, WebAssembly, and Auto).

Part III — Advanced Topics covers authentication and authorization, performance tuning, automated testing with bUnit and Playwright, and real-time communication with SignalR.

Part IV — Industry Practice is what most tutorials skip. It covers how professional teams structure a Blazor solution, how code review actually works on a Blazor codebase, how to build a CI/CD pipeline with GitHub Actions, and how to deploy to Azure, Docker, and IIS. The book closes with a capstone project that ties everything together.

How to use the code examples

Every example is complete enough to type in and run — nothing is left as “an exercise for the reader” unless explicitly marked **Try it yourself**. Code that changes from a previous example is highlighted in the surrounding text, and file names are always given as a heading above the code block, like this:

Pages/Counter.razor

```
<h3>@( "This is example markup" )</h3>
```

Where a shell command is required, it is shown on its own line prefixed with \$:

```
$ dotnet run
```

You will get the most out of this book by typing the examples yourself rather than copying and pasting. Making the small mistakes — a missing @, a mistyped parameter name — and seeing the compiler or the browser react is how the mental model actually sticks.

Let’s begin.

PART I

Foundations

Welcome to Blazor

1.1 What problem does Blazor solve?

For most of the web's history, building an interactive front end meant writing JavaScript (or a language that compiles to it), while your back end was written in whatever server-side language your team preferred. That split has a real cost: two languages, two type systems, two sets of tooling, and validation or business logic that often has to be written twice — once for the client, once for the server, and kept in sync by hand.

Blazor removes that split for teams already invested in .NET. It lets you write your component logic, your models, and often your validation rules once, in C#, and run that same code either in the browser or on the server, depending on the hosting model you choose.

Blazor is not a rewrite of ASP.NET in JavaScript's clothing. It is a genuine component-based UI framework, architecturally similar to React or Vue: you compose a tree of components, each component owns a piece of markup and state, data flows down through parameters, and events flow up through callbacks. If you have used any component-based UI framework before, the concepts will feel familiar; only the syntax and the language are different.

1.2 The three ways Blazor can run

Blazor supports several **render modes**, which control *where* your component's C# code actually executes and how it communicates with the browser. Understanding this early will save you confusion later, because the same component markup can run under any of these modes with almost no code changes.

1.2.1 Blazor Server

Your component code runs on the server, inside an ASP.NET Core process. The browser holds a lightweight page that opens a persistent connection (using SignalR, which you'll meet properly in Part III) back to the server. When the user clicks a button, the click is sent over that connection to the server, your C# event handler runs there, and the server calculates what changed in the UI and sends back a small diff, which the browser applies to the DOM.

- **Pros:** tiny initial download, full .NET server capabilities available immediately (database access, file system, secrets), works on older browsers, debugging feels like debugging any ASP.NET Core app.
- **Cons:** requires a constant connection to the server (doesn't work offline), every UI interaction has network latency, server memory and connections scale with concurrent users.

1.2.2 Blazor WebAssembly (WASM)

Your compiled C# code and a stripped-down .NET runtime are downloaded to the browser and run entirely on the client, using WebAssembly — a binary instruction format supported natively by all modern browsers. Once loaded, the app runs independently of the server; it can even work offline if built as a Progressive Web App.

- **Pros:** no server round-trip for UI logic, works offline after first load, scales trivially (it's just static files on a CDN).
- **Cons:** larger initial download (a runtime plus your app), client hardware limits performance, no direct access to server-only resources (databases, secrets) — you must call an API for that.

1.2.3 Blazor Auto (hybrid, introduced in .NET 8)

The best of both, chosen automatically: on first visit, the page is served using the Server model for a fast, interactive first paint, while the WebAssembly runtime downloads quietly in the background. On the *next* visit, the browser already has the WASM runtime cached, so the app runs entirely on the client from that point on. You write your component once; the render mode is a hosting decision, not a coding decision.

We will return to this in full technical detail in Chapter 13, once you have enough Blazor vocabulary for the explanation to be concrete rather than abstract. For now,

every example in Parts I and II will work identically regardless of which mode you choose, so don't worry about picking the “wrong” one yet.

1.3 Where Blazor sits in the .NET web landscape

Blazor is not a replacement for ASP.NET Core — it is built *on top of* it. A Blazor Server app is an ASP.NET Core application with a special piece of middleware for the SignalR connection. A Blazor WebAssembly app is typically a static set of files (HTML, CSS, your compiled app, and the .NET runtime) that can be hosted by ASP.NET Core, by Azure Static Web Apps, by a plain nginx server, or by any static file host.

This matters because everything you already know (or will learn) about ASP.NET Core — middleware, configuration, dependency injection, logging, `appsettings.json` — applies directly to Blazor Server and to the *host* of a Blazor WebAssembly app. You are not learning a parallel universe; you are learning a new front-end layer on a platform you can already reason about.

1.4 What you will build

Across this book you will build progressively larger pieces of a single running example: a small **task-tracking application** called *TaskBoard*. It starts as a single component with a hardcoded list in Chapter 3, gains routing and forms in Part II, gains authentication and a real backing API in Part III, and in Part IV you will wire it into a CI/CD pipeline and deploy it. Seeing one project grow, rather than jumping between disconnected snippets, is deliberate: it mirrors how a real codebase evolves.

1.5 A note on .NET versions

This book targets **.NET 8** and its unified Blazor Web App template, which is the current long-term-support (LTS) release at the time of writing and the version Microsoft recommends for new projects. Concepts here also apply directly to newer .NET releases; where an API has changed meaningfully between versions, this is called out in a note box.

Note *If a command or menu you see on your machine looks slightly different from a screenshot-free description in this book, check your installed SDK version with `dotnet --version` before assuming something is wrong. Chapter 2 shows you exactly how to install and verify the right SDK.*

With the landscape in view, let's install the tools and create your first project.

Setting Up Your Development Environment

2.1 Installing the .NET SDK

Blazor is part of ASP.NET Core, which ships inside the .NET SDK. You need the SDK (not just the smaller “runtime”) because it includes the compiler and project templates.

1. Go to <https://dotnet.microsoft.com/download> and download the **.NET 8 SDK** for your operating system (Windows, macOS, or Linux).
2. Run the installer.
3. Verify the install from a terminal:

```
$ dotnet --version  
8.0.100
```

If you see a version starting with 8., you’re ready. If the command isn’t found, close and reopen your terminal (the installer updates your PATH, and open terminals don’t pick that up automatically).

Tip On Linux, you can also install the SDK through your distribution’s package manager, but Microsoft’s own install scripts tend to stay more current. On Debian/Ubuntu-based systems:

```
$ wget https://dot.net/v1/dotnet-install.sh  
$ chmod +x dotnet-install.sh  
$ ./dotnet-install.sh --channel 8.0
```

2.2 Choosing an editor

You have three realistic options. All of them are free.

Editor	Best for	Notes
Visual Studio (Windows)	Beginners, full IDE experience	Richest Blazor tooling: component preview, integrated debugger, designer support
Visual Studio Code + C# Dev Kit	Cross-platform, lightweight	Install the “C# Dev Kit” extension; works well on Linux and macOS
JetBrains Rider	Developers coming from IntelliJ/other JetBrains tools	Excellent refactoring tools, cross-platform, paid (free for non-commercial use)

This book’s instructions use the `dotnet` command-line interface throughout, so every example works identically no matter which editor you use to *view* the code. If you’re on Visual Studio Code, install these two extensions now:

- **C# Dev Kit** (by Microsoft)
- **Razor** support is bundled with C# Dev Kit; no separate install needed

2.3 Creating your first project

Open a terminal, navigate to a folder where you keep projects, and run:

```
$ dotnet new blazor -o TaskBoard
$ cd TaskBoard
```

`dotnet new blazor` is the unified .NET 8 template. It scaffolds a **Blazor Web App**: a single project that can serve components with any of the three render modes discussed in Chapter 1. Earlier .NET versions had separate `blazorserver` and `blazorwasm`

templates; from .NET 8 onward, one template covers all of it, and render mode is chosen per-component or per-page, not per-project.

Run it immediately to confirm everything works:

```
$ dotnet run
```

You'll see output similar to:

```
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
```

Open <https://localhost:5001> (the port shown in your own output) in a browser. You should see the default Blazor starter page with a navigation menu (Home, Counter, Weather). Click **Counter** and click the button — you're already looking at a working, interactive Blazor component, compiled from C#, running with no page reload.

Press Ctrl+C in the terminal to stop the server.

2.4 Anatomy of the generated project

Open the TaskBoard folder in your editor. Here is what matters at this stage:

```
TaskBoard/
├── Components/
│   ├── App.razor           # The root document – <html>, <head>, <body>
│   ├── Routes.razor       # Wires up the router
│   └── Layout/
│       ├── MainLayout.razor # The page shell (nav bar + content area)
│       └── NavMenu.razor   # The left-hand navigation
│   └── Pages/
│       ├── Home.razor
│       ├── Counter.razor
│       └── Weather.razor
```

```
|— wwwroot/           # Static files: CSS, images, favicon
|— Program.cs         # Application startup and service
↪ registration
|— appsettings.json   # Configuration
|— TaskBoard.csproj   # Project file: target framework, package
↪ references
```

Two files deserve a first look before we write any code of our own.

Program.cs is the entry point. Every ASP.NET Core app — Blazor included — starts here:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseAntiforgery();

app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

app.Run();
```

Don't worry about understanding every line yet — `AddInteractiveServerComponents` and `AddInteractiveServerRenderMode` are exactly the render-mode wiring from Chapter 1, and we'll dissect them fully in Chapter 13. For now, notice that this is *ordinary ASP.NET Core*: a builder, a pipeline of middleware, and a `Run()` call, exactly like a plain Web API project.

`TaskBoard.csproj` is the project file:

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

</Project>
```

`Nullable` and `ImplicitUsings` being `enable` is the modern .NET default and applies to Blazor exactly as it does to any other C# project — the compiler will warn you about possible null-reference bugs, which is genuinely useful once your components start taking parameters.

2.5 Command-line vs. IDE workflow

Everything in this book can be done with three commands, which you'll use constantly:

```
$ dotnet new blazor -o <ProjectName> # scaffold a new project
$ dotnet run                         # build and run
$ dotnet watch                       # build, run, and auto-reload on
↪ file save
```

`dotnet watch` is worth adopting immediately. It rebuilds and refreshes the browser automatically whenever you save a `.razor`, `.cs`, or `.css` file, which turns Blazor development into a tight edit-save-see-result loop very similar to modern JavaScript tooling.

```
$ dotnet watch
```

Leave this running in a terminal for the rest of this book's exercises unless told otherwise.

2.6 Try it yourself

1. Create a new project called Playground in a scratch folder.
2. Run it with `dotnet watch`.
3. While it's running, open `Components/Pages/Home.razor` and change the heading text. Save the file and confirm the browser updates without you refreshing it.
4. Delete the Playground folder — you won't need it again; TaskBoard is the project you'll build on from here.

Razor Components 101

3.1 What a component actually is

A Blazor component is a `.razor` file that mixes HTML-like markup with C#. At compile time, the Razor compiler turns that file into an ordinary C# class — specifically, a class that inherits from `ComponentBase` and overrides a method called `BuildRenderTree`. You never write that method by hand; the markup you write is compiled into it for you. This matters conceptually: **a component is just a C# class**, and everything you know about classes (constructors, methods, fields, inheritance) still applies.

Let's prove it. Create a new file:

Components/Pages/Hello.razor

```
@page "/hello"

<h1>Hello, Blazor!</h1>
<p>This paragraph was rendered from a component.</p>
```

The `@page` directive registers this component as a routable page at the URL `/hello`. With `dotnet watch` running, navigate to `https://localhost:5001/hello` and you'll see it rendered.

Notice there was no build step you triggered manually, no import to wire up, and no separate route configuration file to edit. The `@page` directive is enough.

3.2 Mixing markup and C#: the `@` symbol

Anywhere in your markup, `@` switches from HTML mode into C# mode for a single expression. This is the single most important syntax rule in Razor.

Components/Pages/Hello.razor

```
@page "/hello"

@{
    var name = "George";
    var year = DateTime.Now.Year;
}

<h1>Hello, @name!</h1>
<p>The current year is @year.</p>
<p>Two plus two is @(2 + 2).</p>
```

A few rules to internalize now, because they explain almost every Razor syntax error you'll hit later:

- `@{ ... }` is a **code block** — plain C# statements, no output produced automatically.
- `@expression` or `@(expression)` **outputs** the result of an expression into the HTML. Use parentheses when the expression contains something the parser might otherwise misread as markup (spaces, operators next to text).
- Inside an already-open code block (`@{ }` or a method), you don't need another `@` to write C# — you're already in C# mode.

3.3 Adding a code-behind section with `@code`

For anything beyond a trivial expression, put your C# in an `@code` block rather than scattering logic through the markup. This keeps the “what does the page look like” and “how does it behave” concerns visually separate, even though they compile into the same class.

Components/Pages/Hello.razor

```
@page "/hello"

<h1>Hello, @Name!</h1>
<button @onclick="Greet">Change name</button>
<p>@message</p>
```

```
@code {  
    private string Name = "George";  
    private string? message;  
  
    private void Greet()  
    {  
        Name = Name == "George" ? "Ada" : "George";  
        message = $"Name changed to {Name} at {DateTime.Now:T}";  
    }  
}
```

Click the button and watch both the heading and the paragraph update. Two new things happened here that deserve unpacking:

1. `@onclick="Greet"` wires a browser DOM event to a C# method. We'll cover the full range of event binding in Chapter 4.
2. After `Greet()` runs, Blazor **automatically re-renders** the component, because `Greet` is bound to an event Blazor is already tracking. You didn't call anything like `render()` yourself — this automatic re-render on state change, triggered by any Blazor-tracked event, is the core mechanic that makes Blazor feel declarative rather than imperative.

3.4 Splitting markup from code entirely: partial classes

As components grow, some developers prefer to move the `@code` block into a separate C# file entirely, using Razor's partial class support. This is optional — both styles are common in real codebases, and you should be comfortable reading either.

Components/Pages/Hello.razor

```
@page "/hello"  
  
<h1>Hello, @Name!</h1>  
<button @onclick="Greet">Change name</button>  
<p>@message</p>
```

Components/Pages/Hello.razor.cs

```
namespace TaskBoard.Components.Pages;

public partial class Hello
{
    private string Name = "George";
    private string? message;

    private void Greet()
    {
        Name = Name == "George" ? "Ada" : "George";
        message = $"Name changed to {Name} at {DateTime.Now:T}";
    }
}
```

The Razor compiler generates a partial class for `Hello.razor` with the *same* name and namespace convention, so the `.razor.cs` file's partial class `Hello` merges with it at compile time. Use this pattern once a component's logic grows past roughly 15–20 lines, or whenever a teammate who is more comfortable in pure C# needs to work on the logic without wading through markup.

Note The namespace in the code-behind file must match where Razor infers the component's namespace to be, which is based on the project's root namespace plus the folder path. If you get a “does not contain a definition” error after adding a code-behind file, check that the namespace lines up, and that both files have exactly the same class name and casing.

3.5 Directives you'll use constantly

Directive	Purpose
@page "/route"	Makes the component routable at the given URL
@code { }	Embeds C# members (fields, properties, methods)

Directive	Purpose
<code>@using Namespace</code>	Same as a C# <code>using</code> , but scoped to this file (or imported globally — see below)
<code>@inject Type name</code>	Requests a service from dependency injection (Chapter 9)
<code>@inherits BaseClass</code>	Makes the component inherit from a custom base class
<code>@implements IInterface</code>	Declares an interface the component implements (used heavily with <code>IDisposable</code> , Chapter 6)
<code>@attribute [Attr]</code>	Applies a C# attribute to the generated class

3.6 Global imports with `_Imports.razor`

Every folder can contain an `_Imports.razor` file listing `@using` statements that apply to every component in that folder and its subfolders — this avoids repeating the same ten `@using` lines at the top of every page. The default template already ships one at `Components/_Imports.razor`:

```
@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using static Microsoft.AspNetCore.Components.Web.RenderMode
@using TaskBoard.Components
```

If you find yourself typing the same `@using` at the top of several new components, move it here instead.

3.7 Try it yourself

1. Create `Components/Pages/About.razor` routed at `/about`, showing your name and a short bio paragraph, using an `@code` block to hold the bio text as a field.
2. Add a button that toggles between a short bio and a long bio, stored as two separate string fields.
3. Move the `@code` block into a `.razor.cs` code-behind file and confirm the page still works after `dotnet watch` picks up the change.

With the fundamentals of markup and code in one file settled, the next chapter goes deeper into the mechanism that makes the button example above work: data binding and events.

Data Binding and Events

4.1 One-way binding: from field to markup

You've already used one-way binding without naming it — any `@expression` in markup is one-way binding from C# state into the rendered HTML. Every time the component re-renders, the expression is re-evaluated and the DOM is updated if the value changed.

```
<p>Items remaining: @remainingCount</p>

@code {
    private int remainingCount = 5;
}
```

4.2 Two-way binding with `@bind`

For form inputs, you usually want the opposite direction too: when the user types into an `<input>`, you want a C# field updated automatically. `@bind` does both directions at once.

Components/Pages/BindDemo.razor

```
@page "/bind-demo"

<input @bind="taskTitle" />
<p>You typed: @taskTitle</p>

@code {
    private string taskTitle = "";
}
```

By default, `@bind` on a text input updates the field when the input loses focus (the `onchange` event). If you want updates on every keystroke, bind to `oninput` explicitly:

```
<input @bind="taskTitle" @bind:event="oninput" />
```

`@bind` isn't limited to strings and text boxes. It works with checkboxes, select drop-downs, and any type Blazor knows how to convert to and from a string:

```
<input type="checkbox" @bind="isDone" />
<input type="number" @bind="priority" />

<select @bind="status">
  <option value="Todo">To do</option>
  <option value="Doing">In progress</option>
  <option value="Done">Done</option>
</select>

@code {
  private bool isDone;
  private int priority;
  private string status = "Todo";
}
```

For a checkbox, Blazor binds to the `checked` property and the change event automatically — there's no need for `@bind:event` there.

4.2.1 Formatting bound values

Binding a `DateTime` gives you a formatting hook:

```
<input @bind="dueDate" @bind:format="yyyy-MM-dd" />

@code {
  private DateTime dueDate = DateTime.Today;
}
```


4.3 Handling events beyond click

`@onclick` was your first taste of event binding in Chapter 3. The same pattern (`@on<domevent>="Handle"`) works for essentially every DOM event: `@onchange`, `@oninput`, `@onkeydown`, `@onmouseover`, `@onsubmit`, and so on.

Components/Pages/EventsDemo.razor

```
@page "/events-demo"

<input @onkeydown="HandleKeyDown" placeholder="Press Enter" />
<p>@log</p>

@code {
    private string log = "";

    private void HandleKeyDown(KeyboardEventArgs e)
    {
        if (e.Key == "Enter")
        {
            log = $"Enter was pressed at {DateTime.Now:T}";
        }
    }
}
```

Notice the handler's parameter, `KeyboardEventArgs`. Every DOM event type has a matching, strongly-typed C# args class — `MouseEventArgs`, `ChangeEventArgs`, `FocusEventArgs`, and more — so you get IntelliSense and compile-time checking on event data, unlike raw JavaScript's loosely-typed event objects.

4.3.1 Passing arguments to a handler

You often need to know *which* item in a list was clicked. Wrap the call in a lambda:

```
@foreach (var task in tasks)
{
    <li>
        @task.Title
        <button @onclick="() => Remove(task)">Remove</button>
    </li>
}
```

```
</li>
}

@code {
    private List<TaskItem> tasks = new()
    {
        new TaskItem { Title = "Write chapter 4" },
        new TaskItem { Title = "Review chapter 3" },
    };

    private void Remove(TaskItem task) => tasks.Remove(task);

    public class TaskItem
    {
        public string Title { get; set; } = "";
    }
}
```

4.3.2 Async event handlers

Event handlers can be async, and Blazor will await them and re-render automatically once they complete — this is the standard way to call an API in response to user input, which you'll do constantly starting in Chapter 10.

```
<button @onclick="SaveAsync">Save</button>

@code {
    private async Task SaveAsync()
    {
        await Task.Delay(500); // pretend this is a network call
        log = "Saved!";
    }
}
```

4.4 Preventing default behavior and stopping propagation

Two modifiers you'll reach for with forms and nested clickable elements:

```
<form @onsubmit="HandleSubmit" @onsubmit:preventDefault>
  <input @bind="taskTitle" />
  <button type="submit">Add</button>
</form>

<div @onclick="OuterClick">
  Outer
  <button @onclick="InnerClick" @onclick:stopPropagation>Inner</button>
</div>
```

`@onsubmit:preventDefault` stops the browser's native full-page-reload form submission — essential, since Blazor wants to handle the submit itself. `@onclick:stopPropagation` stops a click from also triggering a handler on a parent element, exactly like JavaScript's `event.stopPropagation()`.

4.5 Building a small interactive list: your first real TaskBoard feature

Let's combine everything so far into the first working slice of the TaskBoard app you'll keep extending throughout this book.

Components/Pages/TaskList.razor

```
@page "/tasks"

<h3>My Tasks</h3>

<input @bind="newTaskTitle" @bind:event="oninput"
  placeholder="New task title" />
<button @onclick="AddTask"
  ↪ disabled="@string.IsNullOrEmpty(newTaskTitle)">
  Add
</button>

<ul>
  @foreach (var task in tasks)
  {
```

```

        <li style="@task.IsDone ? "text-decoration: line-through;" :
↵  """)">
            <input type="checkbox" @bind="task.IsDone" />
            @task.Title
            <button @onclick="() => tasks.Remove(task)">x</button>
        </li>
    }
</ul>

<p>@tasks.Count(t => !t.IsDone) task(s) remaining</p>

@code {
    private string newTaskTitle = "";
    private List<TaskItem> tasks = new();

    private void AddTask()
    {
        tasks.Add(new TaskItem { Title = newTaskTitle });
        newTaskTitle = "";
    }

    public class TaskItem
    {
        public string Title { get; set; } = "";
        public bool IsDone { get; set; }
    }
}

```

Everything here is technique you’ve already seen: `@bind` on the input and on each checkbox, `disabled="@expression"` as a plain attribute expression, a `@foreach` loop rendering markup per item, and a lambda-wrapped `@onclick` capturing which task to remove. Notice `task.IsDone` binds directly to a property on an object inside a list — Blazor’s binding works on any settable property, not just local fields.

4.6 Try it yourself

1. Add an “Edit” button next to each task that turns its `<li>` into an `<input>` for renaming, and a “Save” button that turns it back into text.
2. Add a dropdown that filters the list to show **All**, **Active**, or **Completed** tasks only, using `@bind` on a `<select>`.

3. Add a “Clear completed” button that removes every task where `IsDone` is `true`.

The list above works, but it has a structural problem you’ll notice as soon as a second page needs to show tasks too: everything lives in one component. The next chapter fixes that by breaking the list item into its own reusable component, and introduces the mechanism components use to talk to each other — parameters.

Component Parameters and Composition

5.1 Breaking the task list into components

Right now `TaskList.razor` does everything: it owns the data, renders the input, and renders every list item inline. As an application grows, this becomes unmanageable. The fix is the same one you'd apply in any component-based UI framework: extract a `TaskCard` component that knows how to render *one* task, and let the parent own the list.

Components/`TaskCard.razor`

```
<li style="@Task.IsDone ? "text-decoration: line-through;" : """>
  <input type="checkbox" checked="@Task.IsDone"
    @onchange="OnToggle" />
  @Task.Title
  <button @onclick="OnRemove">x</button>
</li>

@code {
    [Parameter]
    public TaskItem Task { get; set; } = default!;

    [Parameter]
    public EventCallback OnToggleRequested { get; set; }

    [Parameter]
    public EventCallback OnRemoveRequested { get; set; }

    private Task OnToggle() => OnToggleRequested.InvokeAsync();
    private Task OnRemove() => OnRemoveRequested.InvokeAsync();
}
```

The [Parameter] attribute on a public property is what makes it settable from a parent component's markup — without it, the property is just an ordinary C# property that Blazor's binding system won't populate.

Components/Pages/TaskList.razor (updated)

```
@page "/tasks"

<h3>My Tasks</h3>

<input @bind="newTaskTitle" @bind:event="oninput" placeholder="New task
↵ title" />
<button @onclick="AddTask">Add</button>

<ul>
    @foreach (var task in tasks)
    {
        <TaskCard Task="task"
            OnToggleRequested="() => task.IsDone = !task.IsDone"
            OnRemoveRequested="() => tasks.Remove(task)" />
    }
</ul>

@code {
    private string newTaskTitle = "";
    private List<TaskItem> tasks = new();

    private void AddTask()
    {
        if (string.IsNullOrEmpty(newTaskTitle)) return;
        tasks.Add(new TaskItem { Title = newTaskTitle });
        newTaskTitle = "";
    }
}
```

Where does TaskItem live now that two files need it? Move it to its own file, outside any component:

Models/TaskItem.cs


```
namespace TaskBoard.Models;

public class TaskItem
{
    public string Title { get; set; } = "";
    public bool IsDone { get; set; }
}
```

Add `@using TaskBoard.Models` to `Components/_Imports.razor` so every component can see it without repeating the using.

5.2 [Parameter] vs. plain properties

Only properties marked `[Parameter]` (or `[CascadingParameter]`, covered shortly) can receive values from markup attributes. A component can absolutely have other public or private properties for internal use — they simply won't be settable from outside.

```
[Parameter]
public TaskItem Task { get; set; } = default!;    // settable from parent
↪ markup

public int InternalRenderCount { get; private set; } // NOT settable from
↪ markup
```

Note *Parameter properties must have a public setter — Blazor sets them via reflection during rendering. Using `default!` for a non-nullable parameter with no sensible default is an accepted convention: it tells the compiler “trust me, this will be set before use,” which is true for any required parameter, and avoids a spurious nullable warning.*

5.3 Required parameters

For parameters that must always be supplied, use `[EditorRequired]` alongside `[Parameter]`. It doesn't enforce anything at runtime, but the compiler and your editor will warn you if a caller forgets it — genuinely useful for catching mistakes before they ship.

```
[Parameter, EditorRequired]
public TaskItem Task { get; set; } = default!;
```

5.4 `EventCallback`: letting a child talk to its parent

Parameters flow data **down**. `EventCallback` (and its generic form `EventCallback<T>`) is how a child notifies its parent that something happened — the up direction. You saw this above with `OnToggleRequested` and `OnRemoveRequested`. `EventCallback<T>` lets you pass a typed payload:

```
@* Child component *@
<button onclick="() => OnRename.InvokeAsync(newTitle)">Rename</button>

@code {
    [Parameter]
    public EventCallback<string> OnRename { get; set; }
    private string newTitle = "Untitled";
}
```

```
@* Parent *@
<RenameableCard OnRename="HandleRename" />

@code {
    private void HandleRename(string newTitle)
    {
        // do something with newTitle
    }
}
```

EventCallback (over a plain C# Action) exists specifically for Blazor's rendering model: invoking it automatically triggers a re-render of the *parent* component afterward, and it correctly handles asynchronous handlers. Prefer it over raw delegates for anything crossing a component boundary.

5.5 ChildContent: components that wrap other markup

Sometimes a component's job is to wrap arbitrary markup — a card, a modal, a collapsible panel — rather than render a fixed shape. The special parameter name `ChildContent`, typed as `RenderFragment`, captures whatever markup is nested inside the component's tags.

Components/Panel.razor

```
<div class="panel">
  <div class="panel-header">@Title</div>
  <div class="panel-body">
    @ChildContent
  </div>
</div>

@code {
  [Parameter, EditorRequired]
  public string Title { get; set; } = "";

  [Parameter]
  public RenderFragment? ChildContent { get; set; }
}
```

Using it:

```
<Panel Title="Task Details">
  <p>Due: @task.DueDate.ToShortDateString()</p>
  <p>Priority: @task.Priority</p>
</Panel>
```

Everything between `<Panel ...>` and `</Panel>` becomes the value of `ChildContent`, rendered exactly where `@ChildContent` appears inside `Panel.razor`. This is how

most “layout” or “wrapper” style components work, including Blazor’s own built-in `<Layout>` mechanism you’ll meet in Chapter 7.

5.6 Templated components: parameterizing *how* content renders

ChildContent is fixed markup. A **templated component** goes further, exposing a `RenderFragment<T>` — a chunk of markup parameterized by a value, effectively a function from data to markup that the *caller* supplies.

Components/TaskListView.razor

```
<ul>
    @foreach (var item in Items)
    {
        <li>@ItemTemplate(item)</li>
    }
</ul>

@code {
    [Parameter, EditorRequired]
    public List<TaskItem> Items { get; set; } = new();

    [Parameter, EditorRequired]
    public RenderFragment<TaskItem> ItemTemplate { get; set; } =
    ↪ default!;
}
```

Using it, with the special context keyword representing the per-item value (or a custom name via `Context="task"`):

```
<TaskListView Items="tasks" Context="task">
    <ItemTemplate>
        <strong>@task.Title</strong> – @(task.IsDone ? "done" :
    ↪ "pending")
    </ItemTemplate>
</TaskListView>
```

Wait — the markup above uses `<ItemTemplate>` as a nested element even though the parameter is named `ItemTemplate` on the C# side. That's not a coincidence: Blazor matches a nested element whose tag name equals a `RenderFragment/RenderFragment<T>` parameter name and treats its contents as that parameter's template. This pattern is exactly how ASP.NET Core's own data grid and list components (and most third-party Blazor component libraries) let you customize per-row rendering without subclassing anything.

5.7 Cascading values: skipping the middleman

Parameters are explicit and easy to trace, but passing the same value through four levels of nesting just so a deeply-nested component can read it becomes tedious (“prop drilling,” in front-end parlance). `CascadingValue` broadcasts a value to *every* descendant, no matter how deep, without each intermediate component needing to know or forward it.

Components/Layout/MainLayout.razor (simplified)

```
<CascadingValue Value="currentTheme">
  <div class="app">
    @Body
  </div>
</CascadingValue>

@code {
    private string currentTheme = "light";
}
```

Any descendant, at any depth, can opt in:

```
@code {
    [CascadingParameter]
    public string Theme { get; set; } = "light";
}
```

Use cascading values sparingly — for genuinely cross-cutting concerns like the current theme, the current authenticated user (Chapter 14 uses exactly this mechanism),

or a validation context (Chapter 8). For anything specific to one parent-child relationship, an explicit [Parameter] is easier to trace when you're debugging six months from now and have forgotten how the cascade was wired.

5.8 Try it yourself

1. Extract the “add task” input and button from `TaskList.razor` into its own `AddTaskBox` component that takes an `EventCallback<string>` fired when the user submits a new title.
2. Give `TaskCard` an `EditorRequired` parameter for a CSS class name, so the parent can control styling per-card.
3. Build a `Modal` component using `ChildContent` that shows a semi-transparent overlay and a centered box, then use it to show task details when a card is clicked.

You now know how components share data and communicate. The next chapter covers the events that fire automatically as a component is created, updated, and destroyed — its lifecycle — which is where `OnInitialized`, `OnParametersSet`, and cleanup logic live.

The Component Lifecycle

6.1 Why lifecycle methods exist

A component doesn't just appear fully formed — it goes through a predictable sequence of stages: it's created, it receives its initial parameters, it renders for the first time, it may receive new parameters later and re-render, and eventually it's removed from the page. Blazor exposes hooks at each of these stages so you can run setup code (loading data when a component first appears), reaction code (responding when a parameter changes), and teardown code (cancelling a timer before the component disappears).

6.2 The full sequence

For a component's **first** render, methods run in this order:

1. Constructor (rarely used directly in Blazor — prefer the methods below)
2. `SetParametersAsync` (rarely overridden directly)
3. `OnInitialized` / `OnInitializedAsync`
4. `OnParametersSet` / `OnParametersSetAsync`
5. Markup is rendered
6. `OnAfterRender` / `OnAfterRenderAsync` (with `firstRender: true`)

On **every subsequent** update (a parameter changes, or `StateHasChanged` is called):

1. `OnParametersSet` / `OnParametersSetAsync`
2. Markup is re-rendered (if anything actually changed)
3. `OnAfterRender` / `OnAfterRenderAsync` (with `firstRender: false`)

6.3 OnInitialized — one-time setup

Runs exactly once, the first time the component is instantiated. This is the right place for anything that should happen once and only once: loading initial data, subscribing to an event, setting up initial state derived from parameters.

Components/Pages/TaskList.razor (excerpt)

```
@code {
    private List<TaskItem> tasks = new();
    private bool isLoading = true;

    protected override async Task OnInitializedAsync()
    {
        isLoading = true;
        tasks = await TaskService.GetAllAsync(); // a service, see
↪ Chapter 9
        isLoading = false;
    }
}
```

```
@if (isLoading)
{
    <p>Loading tasks...</p>
}
else
{
    <ul>
        @foreach (var task in tasks)
        {
            <TaskCard Task="task" ... />
        }
    </ul>
}
```

Always prefer the Async variant (`OnInitializedAsync`) when your setup logic does any I/O — reading from a database, calling an API, reading a file. Blazor awaits it properly and re-renders once it completes, which is exactly the loading-state pattern shown above.

6.4 OnParametersSet — reacting to new parameter values

Called every time the component's parameters are set — including the very first time, right after `OnInitialized`. Use this when a component needs to *react* to a parameter changing, not just read it once.

```
@code {
    [Parameter]
    public int TaskId { get; set; }

    private TaskItem? task;
    private int lastLoadedId = -1;

    protected override async Task OnParametersSetAsync()
    {
        if (TaskId != lastLoadedId)
        {
            task = await TaskService.GetByIdAsync(TaskId);
            lastLoadedId = TaskId;
        }
    }
}
```

The `lastLoadedId` guard matters: `OnParametersSet` fires on *every* parameter update, including ones where `TaskId` didn't actually change but some other parameter did. Without the guard, you'd re-fetch the same task unnecessarily on every unrelated update.

Note A very common beginner mistake is putting data-loading logic that depends on a route parameter (like `TaskId` above) inside `OnInitialized` instead of `OnParametersSet`. `OnInitialized` runs once ever; if the user navigates from `/tasks/1` to `/tasks/2` and Blazor reuses the same component instance (which it does, for efficiency, when navigating between two routes that render the same component type), `OnInitialized` will **not** run again, and your component will keep

showing task 1’s data. `OnParametersSet` runs on every navigation, so it’s the correct hook for this.

6.5 `OnAfterRender` — after the DOM has actually updated

Runs after the component has rendered to the actual browser DOM. This is the *only* safe place to call JavaScript that needs to inspect or manipulate rendered DOM elements (covered fully in Chapter 12), because it’s the only lifecycle method guaranteed to run after the markup exists on the page.

```
@code {
    private ElementReference searchInputRef;

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            await searchInputRef.FocusAsync();
        }
    }
}
```

```
<input @ref="searchInputRef" placeholder="Search tasks..." />
```

The `firstRender` flag lets you distinguish “the component just appeared for the first time” (good moment to auto-focus an input, initialize a third-party JS widget) from “the component just re-rendered because something changed” (usually you don’t want to repeat the setup).

Important: *never call `StateHasChanged()` unconditionally inside `OnAfterRender` — doing so triggers another render, which triggers `OnAfterRender` again, which is an infinite loop. If you must update*

state here, guard it carefully and only call `StateHasChanged()` when something genuinely new needs to be shown.

6.6 `StateHasChanged` — telling Blazor “something changed, please re-render”

Blazor automatically re-renders a component after any event handler it invoked directly (`@onclick`, `@onchange`, and so on) finishes running. But if state changes from somewhere Blazor *isn't* watching — a background timer, a callback from a long-running task, a message arriving over SignalR — you must call `StateHasChanged()` yourself to tell the component to re-render.

```
@page "/live-clock"
@implements IDisposable

<p>The time is: @currentTime.ToString("T")</p>

@code {
    private DateTime currentTime = DateTime.Now;
    private Timer? timer;

    protected override void OnInitialized()
    {
        timer = new Timer(_ =>
        {
            currentTime = DateTime.Now;
            InvokeAsync(StateHasChanged);
        }, null, dueTime: 0, period: 1000);
    }

    public void Dispose() => timer?.Dispose();
}
```

Two details matter here. First, `System.Threading.Timer` callbacks run on a thread-pool thread, not the thread Blazor’s rendering is synchronized to — calling `StateHasChanged()` directly from that thread would throw in Blazor Server. `InvokeAsync(StateHasChanged)` correctly marshals the call back onto the component’s synchronization context. Sec-

ond — and this leads directly into the next section — the timer is disposed in a `Dispose` method.

6.7 Cleaning up with `IDisposable`

Anything a component sets up that outlives a single render — timers, event subscriptions, open connections — must be torn down when the component is removed from the page, or you'll leak resources. This especially matters in Blazor Server, where a leaked subscription keeps consuming server memory for as long as the server process runs, potentially affecting every user, not just the one whose page left.

```
@implements IDisposable

@code {
    protected override void OnInitialized()
    {
        SomeAppWideEventBus.TaskAdded += HandleTaskAdded;
    }

    private void HandleTaskAdded(object? sender, TaskItem task)
    {
        // react to the event
        InvokeAsync(StateHasChanged);
    }

    public void Dispose()
    {
        SomeAppWideEventBus.TaskAdded -= HandleTaskAdded;
    }
}
```

If your cleanup needs to be asynchronous (closing a network connection gracefully, for instance), implement `IAsyncDisposable` instead — Blazor supports and correctly awaits both.

```
@implements IAsyncDisposable

@code {
```

```

private HubConnection? hubConnection; // see Chapter 17

public async ValueTask DisposeAsync()
{
    if (hubConnection is not null)
    {
        await hubConnection.DisposeAsync();
    }
}
}

```

6.8 A complete mental model

Method	Runs when	Typical use
OnInitialized(Async)	Once, on first creation	Load initial data, subscribe to events
OnParametersSet(Async)	First render, and every time parameters update	React to a changed route parameter or parent-supplied value
OnAfterRender(Async)	After every render, once DOM exists	JS interop that touches rendered elements
StateHasChanged()	Called manually	Force a re-render after external state changes
Dispose / DisposeAsync	Component removed from the tree	Unsubscribe, close connections, stop timers

6.9 Try it yourself

1. Add a LastUpdated label to TaskCard that shows how long ago the task was created, updating every second using a timer — remember to dispose it.
2. Convert the task-loading logic in TaskList to depend on a `[Parameter] public string? FilterStatus { get; set; }`, and use `OnParametersSet` to re-fetch only when the filter actually changes.

3. Intentionally forget to call `Dispose` on a timer-based component, navigate away and back to it ten times, and watch your CPU usage climb — then fix it and confirm it stops climbing. This is worth doing once; it makes the “why disposal matters” lesson visceral rather than theoretical.

Part I has given you the vocabulary to build a fully interactive, single-page component tree. Part II adds what’s needed for a multi-page application: routing between pages, forms with validation, calling real APIs, and managing state that spans more than one component.

PART II

Building Applications

Routing and Navigation

7.1 How @page becomes a route table

Every component with an @page directive is discovered automatically at startup and added to a route table — you never maintain a central list of routes by hand, unlike many other web frameworks. The router component, `Routes.razor`, scans the assembly for these directives and matches the current URL against them.

Components/Routes.razor (as generated by the template)

```
<Router AppAssembly="typeof(Program).Assembly">
  <Found Context="routeData">
    <RouteView RouteData="routeData"
    ↪ DefaultLayout="typeof(Layout.MainLayout)" />
    <FocusOnNavigate RouteData="routeData" Selector="h1" />
  </Found>
  <NotFound>
    <LayoutView Layout="typeof(Layout.MainLayout)">
      <p>Sorry, there's nothing at this address.</p>
    </LayoutView>
  </NotFound>
</Router>
```

`<Found>` renders whenever a matching route is found, wrapping the matched page in the default layout. `<NotFound>` renders for any URL that doesn't match anything — your 404 page.

7.2 Route parameters

A route segment can capture part of the URL as a strongly-typed parameter.

Components/Pages/TaskDetail.razor

```
@page "/tasks/{TaskId:int}"

<h3>Task #{@TaskId}</h3>

@code {
    [Parameter]
    public int TaskId { get; set; }
}
```

The `:int` **route constraint** tells the router to only match this route when the segment parses as an integer, and to hand you an already-parsed `int` — visiting `/tasks/7` sets `TaskId` to 7; visiting `/tasks/abc` simply doesn't match this route at all (it falls through to `NotFound` unless another route matches). Common constraints:

Constraint	Matches
<code>{Id:int}</code>	Integers
<code>{Id:guid}</code>	GUIDs
<code>{Slug}</code> (no constraint)	Any non-empty string segment
<code>{Rest:nonfile}</code>	Anything without a file extension — useful combined with catch-all

A component can have **multiple** `@page` directives if it should be reachable from more than one URL:

```
@page "/tasks/{TaskId:int}"
@page "/tasks/{TaskId:int}/details"
```

7.2.1 Optional parameters

Add `?` to make a segment optional:

```
@page "/tasks/{TaskId:int?}"

@code {
```

```

[Parameter]
public int? TaskId { get; set; }

protected override void OnParametersSet()
{
    if (TaskId is null)
    {
        // show a "no task selected" state
    }
}

```

7.3 NavigationManager: navigating from code

Clicking an `<a href="...">` navigates the normal, browser-native way. When you need to navigate **from C#** — after saving a form, for instance — inject `NavigationManager`.

```

@page "/tasks/new"
@Inject NavigationManager Nav

<button @onclick="SaveAndReturn">Save</button>

@code {
    private async Task SaveAndReturn()
    {
        var newId = await TaskService.CreateAsync(new TaskItem { Title =
↪ "New task" });
        Nav.NavigateTo($"/tasks/{newId}");
    }
}

```

`NavigationManager` also exposes the current URL and an event for reacting to navigation happening elsewhere in the app:

```

@Inject NavigationManager Nav
implements IDisposable

@code {

```

```
protected override void OnInitialized()
{
    Nav.LocationChanged += OnLocationChanged;
}

private void OnLocationChanged(object? sender,
    ↪ LocationChangedEventArgs e)
{
    // e.Location is the new URL
}

public void Dispose() => Nav.LocationChanged -= OnLocationChanged;
}
```

7.4 Reading query strings

Query strings aren't part of @page's route pattern; read them with the [SupplyParameterFromQuery] attribute.

```
@page "/tasks"

@code {
    [SupplyParameterFromQuery]
    public string? Status { get; set; }

    // visiting /tasks?status=Done populates Status = "Done"
}
```

7.5 Layouts: the shared page frame

A **layout** is a component that wraps every page rendered through it, providing shared chrome like a navigation bar or footer. `MainLayout.razor` is the default one, referenced from `Routes.razor` above.

Components/Layout/MainLayout.razor

```
@inherits LayoutComponentBase

<div class="page">
  <div class="sidebar">
    <NavMenu />
  </div>

  <main>
    <div class="top-row px-4">
      <a href="https://learn.microsoft.com/aspnet/core/blazor"
↵ target="_blank">About</a>
    </div>

    <article class="content px-4">
      @Body
    </article>
  </main>
</div>
```

Two things make a component usable as a layout: inheriting `LayoutComponentBase`, and rendering `@Body` somewhere — `@Body` is where the routed page's own markup gets injected.

7.5.1 Assigning a different layout per page

Override the default with `@layout` on any page:

```
@page "/tasks/new"
@layout MinimalLayout

<h3>New Task</h3>
```

This is the standard way to give, for example, a login page or a print-friendly view a different (or absent) chrome from the rest of the app.

7.6 Building the navigation menu

Components/Layout/NavMenu.razor (trimmed to the relevant part)

```
<nav class="flex-column">
  <div class="nav-item px-3">
    <NavLink class="nav-link" href="" Match="NavLinkMatch.All">
      Home
    </NavLink>
  </div>
  <div class="nav-item px-3">
    <NavLink class="nav-link" href="tasks">
      Tasks
    </NavLink>
  </div>
</nav>
```

`<NavLink>` renders an `<a>` tag but additionally applies an active CSS class automatically when its `href` matches the current URL — which is how “highlight the current page in the nav bar” works with zero manual code. `Match="NavLinkMatch.All"` requires an exact match (used for the home link, so it isn’t marked active on every page); the default, `NavLinkMatch.Prefix`, matches if the current URL merely *starts with* the link’s `href`, which is usually what you want for section links like “Tasks” staying highlighted across `/tasks`, `/tasks/5`, and `/tasks/new`.

7.7 A worked example: task list, detail, and create pages wired together

Components/Pages/TaskList.razor (excerpt)

```
@page "/tasks"
@inject NavigationManager Nav

<h3>My Tasks</h3>
<button @onclick='() => Nav.NavigateTo("/tasks/new")'>+ New task</button>

<ul>
  @foreach (var task in tasks)
  {
    <li>
      <NavLink href="@($" /tasks/{task.Id}")">@task.Title</NavLink>
    </li>
  }
</ul>
```

```
        </li>
    }
</ul>
```

Components/Pages/TaskDetail.razor

```
@page "/tasks/{TaskId:int}"
@inject NavigationManager Nav

<h3>@task?.Title</h3>
<button @onclick='() => Nav.NavigateTo("/tasks")'>Back to list</button>

@code {
    [Parameter]
    public int TaskId { get; set; }

    private TaskItem? task;

    protected override async Task OnParametersSetAsync()
    {
        task = await TaskService.GetByIdAsync(TaskId);
    }
}
```

Notice TaskDetail uses `OnParametersSetAsync`, not `OnInitializedAsync`, exactly for the reason discussed in Chapter 6 — clicking from task 3 to task 5 reuses the same component instance, so only `OnParametersSet` is guaranteed to run again.

7.8 Try it yourself

1. Add a route `/tasks/{TaskId:int}/edit` with a form pre-populated from the existing task (you'll build the actual form logic properly in Chapter 8).
2. Add a catch-all route (`@page "/tasks/{*segments}"`) that logs the unmatched segments — useful for understanding how catch-all routes behave before you need them for something like a documentation-browser feature.
3. Style `NavMenu` so the active link has a distinct background color, using the automatically applied active class.

Routing gets users to the right page. The next chapter tackles what happens once they're there and need to submit data back: forms and validation.

Forms and Validation

8.1 Why `EditForm` instead of a plain `<form>`

You could bind a plain `<form>` with `@onsubmit`, and for very simple cases that's fine. But real forms need validation, need to track which fields the user has touched, and need a way for input components to report errors back to a shared context. `EditForm` provides all of this out of the box, backed by an `EditContext` that tracks the model, its validation state, and every field's "modified" status.

Components/Pages/NewTask.razor

```
@page "/tasks/new"
@inject NavigationManager Nav

<h3>New Task</h3>

<EditForm Model="model" OnValidSubmit="HandleValidSubmit">
  <DataAnnotationsValidator />
  <ValidationSummary />

  <div>
    <label>Title</label>
    <InputText @bind-Value="model.Title" />
    <ValidationMessage For="() => model.Title" />
  </div>

  <div>
    <label>Due date</label>
    <InputDate @bind-Value="model.DueDate" />
    <ValidationMessage For="() => model.DueDate" />
  </div>

  <div>
    <label>Priority</label>
```

```
<InputSelect @bind-Value="model.Priority">
    <option value="Low">Low</option>
    <option value="Medium">Medium</option>
    <option value="High">High</option>
</InputSelect>
</div>

<button type="submit">Create</button>
</EditForm>

@code {
    private NewTaskModel model = new();

    private async Task HandleValidSubmit()
    {
        var id = await TaskService.CreateAsync(model);
        Nav.NavigateTo($"/tasks/{id}");
    }
}
```

Models/NewTaskModel.cs

```
using System.ComponentModel.DataAnnotations;

namespace TaskBoard.Models;

public class NewTaskModel
{
    [Required(ErrorMessage = "A title is required.")]
    [StringLength(100, ErrorMessage = "Title cannot exceed 100
↪ characters.")]
    public string Title { get; set; } = "";

    [Required(ErrorMessage = "Pick a due date.")]
    [DataType(DataType.Date)]
    public DateTime DueDate { get; set; } = DateTime.Today.AddDays(1);

    public string Priority { get; set; } = "Medium";
}
```

[Required], [StringLength], and the rest of System.ComponentModel.DataAnnotations are the same attributes used by ASP.NET Core MVC and Web API model valida-

tion — if you've validated a request DTO in a Web API controller before, this is the identical attribute set, now driving client-visible validation too.

8.2 The built-in input components

Blazor ships a matching input component for each common HTML input type, each supporting `@bind-Value` and automatically wiring into the surrounding `EditForm`'s validation:

Component	Renders	Binds to
<code>InputText</code>	<code><input type="text"></code>	string
<code>InputTextArea</code>	<code><textarea></code>	string
<code>InputNumber</code>	<code><input type="number"></code>	any numeric type
<code>InputDate</code>	<code><input type="date"></code>	<code>DateTime</code> / <code>DateOnly</code>
<code>InputCheckbox</code>	<code><input type="checkbox"></code>	bool
<code>InputSelect</code>	<code><select></code>	any type (often an enum)
<code>InputRadioGroup</code> + <code>InputRadio</code>	radio buttons	any type

Using `InputSelect` with an enum, rather than magic strings, is worth doing from the start:

```
public enum TaskPriority { Low, Medium, High }
```

```
<InputSelect @bind-Value="model.Priority">
  @foreach (var priority in Enum.GetValues<TaskPriority>())
  {
    <option value="@priority">@priority</option>
  }
</InputSelect>
```

8.3 Custom validation logic

`DataAnnotationsValidator` covers attribute-based rules. For anything that depends on comparing two fields, or checking against data outside the model (does this title already exist?), implement `IValidatableObject` on the model:

```
using System.ComponentModel.DataAnnotations;

public class NewTaskModel : IValidatableObject
{
    public string Title { get; set; } = "";
    public DateTime DueDate { get; set; }
    public DateTime? StartDate { get; set; }

    public IEnumerable<ValidationResult> Validate(ValidationContext
        ↪ context)
    {
        if (StartDate.HasValue && DueDate < StartDate)
        {
            yield return new ValidationResult(
                "Due date cannot be before the start date.",
                new[] { nameof(DueDate) });
        }
    }
}
```

`DataAnnotationsValidator` automatically picks up `IValidatableObject` implementations alongside the attribute rules — no extra wiring required.

8.4 Manual validation with `EditContext` directly

For validation logic that can't be expressed as attributes at all — perhaps it requires an async call to check for a duplicate title — take direct control of the `EditContext`.

```
<EditForm EditContext="editContext" OnValidSubmit="HandleValidSubmit">
    <DataAnnotationsValidator />
    <ValidationSummary />
    <InputText @bind-Value="model.Title" />
```

```

        <button type="submit">Create</button>
    </EditForm>

@code {
    private NewTaskModel model = new();
    private EditContext editContext = default!;

    protected override void OnInitialized()
    {
        editContext = new EditContext(model);
        editContext.OnValidationRequested += async (sender, args) =>
        {
            if (await TaskService.TitleExistsAsync(model.Title))
            {
                var messages = new ValidationMessageStore(editContext);
                messages.Add(() => model.Title, "A task with this title
↪ already exists.");
            }
        };
    }

    private async Task HandleValidSubmit()
    {
        var id = await TaskService.CreateAsync(model);
        Nav.NavigateTo($"/tasks/{id}");
    }
}

```

8.5 OnValidSubmit VS. OnInvalidSubmit VS. OnSubmit

EditForm exposes three submit callbacks, and you use exactly one per form:

- **OnValidSubmit** — fires only if validation passes. The common case.
- **OnInvalidSubmit** — fires only if validation fails. Useful for logging or focusing the first invalid field.
- **OnSubmit** — fires regardless of validation state; you check `editContext.Validate()` yourself. Use this when you need full manual control (for instance, showing a confirmation dialog even on a technically-valid form before actually saving).

8.6 Building a reusable form field component

As forms multiply across an app, the `<label> / input / <ValidationMessage>` triplet repeats constantly. Wrap it once:

Components/FormField.razor

```
<div class="form-field">
  <label>@Label</label>
  @ChildContent
  <ValidationMessage For="For" />
</div>

@code {
    [Parameter, EditorRequired]
    public string Label { get; set; } = "";

    [Parameter, EditorRequired]
    public Expression<Func<object>> For { get; set; } = default!;

    [Parameter]
    public RenderFragment? ChildContent { get; set; }
}
```

```
<FormField Label="Title" For="() => model.Title">
  <InputText @bind-Value="model.Title" />
</FormField>
```

8.7 Try it yourself

1. Add a `Description` field to `NewTaskModel` using `InputTextArea`, with a `[StringLength(500)]` limit and a live character counter below the box.
2. Add cross-field validation ensuring `DueDate` is never in the past, using `IValidatableObject`.
3. Convert the label/input/validation-message repetition in your form to use the `FormField` component above.

Forms get user data into your model; the next chapter covers where that model actually goes once submitted — dependency injection and the service layer that talks to your backend.

Dependency Injection and Services

9.1 Why services instead of code directly in components

Every example so far has quietly referenced a `TaskService` without explaining where it comes from. If component code called a database or an HTTP API directly, every component that needs task data would duplicate that logic, and testing a component would require a real database or network connection. **Dependency injection (DI)** solves this: components declare *what* they need (a `ITaskService`), and the framework supplies *which implementation* satisfies that need, configured once, in one place.

This is not a Blazor-specific idea — it's the same DI container used throughout ASP.NET Core, extended to also work inside components. If you've registered a service in `Program.cs` for an MVC or Web API project before, you already know most of this chapter.

9.2 Defining a service

Start with an interface — this is what makes the service swappable and testable, which matters enormously once you reach Chapter 16.

Services/ITaskService.cs

```
namespace TaskBoard.Services;

public interface ITaskService
{
    Task<List<TaskItem>> GetAllAsync();
    Task<TaskItem?> GetByIdAsync(int id);
    Task<int> CreateAsync(NewTaskModel model);
}
```

```
Task UpdateAsync(TaskItem task);  
Task DeleteAsync(int id);  
}
```

Services/InMemoryTaskService.cs (a simple implementation to get moving; Chapter 10 replaces this with a real HTTP-backed one)

```
namespace TaskBoard.Services;  
  
public class InMemoryTaskService : ITaskService  
{  
    private readonly List<TaskItem> tasks = new();  
    private int nextId = 1;  
  
    public Task<List<TaskItem>> GetAllAsync() =>  
        Task.FromResult(tasks.ToList());  
  
    public Task<TaskItem?> GetByIdAsync(int id) =>  
        Task.FromResult(tasks.FirstOrDefault(t => t.Id == id));  
  
    public Task<int> CreateAsync(NewTaskModel model)  
    {  
        var task = new TaskItem  
        {  
            Id = nextId++,  
            Title = model.Title,  
            DueDate = model.DueDate,  
        };  
        tasks.Add(task);  
        return Task.FromResult(task.Id);  
    }  
  
    public Task UpdateAsync(TaskItem task)  
    {  
        var index = tasks.FindIndex(t => t.Id == task.Id);  
        if (index >= 0) tasks[index] = task;  
        return Task.CompletedTask;  
    }  
  
    public Task DeleteAsync(int id)  
    {  
        tasks.RemoveAll(t => t.Id == id);  
    }  
}
```

```

        return Task.CompletedTask;
    }
}

```

9.3 Registering the service

Program.cs

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

builder.Services.AddScoped<ITaskService, InMemoryTaskService>(); // <--
↪ new line

var app = builder.Build();
// ...

```

AddScoped is one of three **lifetimes** the DI container supports, and choosing the right one matters more in Blazor than in a typical Web API, because of how Blazor Server manages connections.

Lifetime	New instance created...	In Blazor Server	In Blazor WebAssembly
Transient	Every time it's requested	New instance per request	New instance per request
Scoped	Once per "scope"	Once per user circuit (effectively, once per browser tab's session)	Once per app instance (WASM has no per-request concept — the whole app is one long-lived scope)

	New instance created...	In Blazor Server	In Blazor WebAssembly
Lifetime			
Singleton	Once, for the entire application	Shared across every user on the server	Shared across the whole app instance (same as Scoped, practically, in WASM)

Important: *Scoped in Blazor Server means “one instance per connected user,” not “one instance per HTTP request” as in a typical Web API. This is a common source of subtle bugs: a Singleton service holding per-user mutable state (like “the currently selected task”) would leak that state across every user connected to the server, because they all share the same instance. Default to Scoped for anything holding user-specific state, and reserve Singleton for genuinely shared, stateless, or thread-safe infrastructure (a caching layer, a configuration object).*

9.4 Consuming a service in a component: `@inject`

```
@page "/tasks"
@inject ITaskService TaskService

@code {
    private List<TaskItem> tasks = new();

    protected override async Task OnInitializedAsync()
    {
        tasks = await TaskService.GetAllAsync();
    }
}
```

`@inject Type PropertyName` generates a public property of that type, decorated with `[Inject]`, and populates it automatically when the component is created. It’s equivalent to writing:

```
[Inject]
public ITaskService TaskService { get; set; } = default!;
```

Use whichever form you prefer; `@inject` is simply more concise and is the more common convention in `.razor` markup, while `[Inject]` is typical in code-behind files where there's no markup to put a directive in.

9.5 Constructor injection in non-component classes

DI isn't limited to components. Any service you register can itself depend on other registered services via its constructor — this is how you build a layered service graph.

```
public class TaskNotificationService
{
    private readonly ITaskService taskService;
    private readonly ILogger<TaskNotificationService> logger;

    public TaskNotificationService(ITaskService taskService,
        ↪ ILogger<TaskNotificationService> logger)
    {
        this.taskService = taskService;
        this.logger = logger;
    }

    public async Task NotifyOverdueTasksAsync()
    {
        var tasks = await taskService.GetAllAsync();
        var overdue = tasks.Where(t => t.DueDate < DateTime.Today &&
            ↪ !t.IsDone);
        foreach (var task in overdue)
        {
            logger.LogInformation("Task {Title} is overdue", task.Title);
        }
    }
}
```

`ILogger<T>` above is itself a framework-provided service, injected the same way — logging is DI-based throughout ASP.NET Core, which is why you never manually construct a logger.

9.6 Injecting framework services you’ll use constantly

A handful of built-in services are available for injection in nearly every Blazor app:

```
@inject NavigationManager Navigation      @* URL and navigation,  
    ↳ Chapter 7 *@  
@inject HttpClient Http                  @* calling APIs, Chapter 10  
    ↳ *@  
@inject IJSRuntime JSRuntime              @* JavaScript interop,  
    ↳ Chapter 12 *@  
@inject AuthenticationStateProvider AuthState @* current user, Chapter  
    ↳ 14 *@  
@inject ILogger<MyComponent> Logger      @* structured logging *@
```

9.7 Scoping services per feature area

As an application grows, resist the temptation to make every service a singleton “for convenience.” A well-scoped service graph — small, focused interfaces, registered with deliberately chosen lifetimes — is one of the biggest differences between a code-base that stays maintainable at 50 components and one that turns into an untraceable web of shared mutable state. This distinction becomes concrete again in Chapter 18, when we look at how professional teams lay out a multi-project Blazor solution.

9.8 Try it yourself

1. Extract a second service, `INotificationService`, responsible only for showing transient “Task created” style messages, and inject it alongside `ITaskService` in `NewTask.razor`.

2. Register `InMemoryTaskService` as `Singleton` instead of `Scoped`, open the app in two different browsers (not two tabs of the same browser — use a normal window and a private/incognito window so they get separate circuits), and observe that tasks created in one appear in the other. Then change it back to `Scoped` and confirm the isolation returns. This exercise is the fastest way to make the `Scoped`-vs-`Singleton` distinction stick.
3. Add a constructor-injected `ILogger<InMemoryTaskService>` to your service and log a line every time a task is created.

Your service layer currently stores everything in memory, which disappears every time the app restarts. The next chapter connects it to a real HTTP API instead.

Calling APIs and Working with Data

10.1 Setting up `HttpClient`

Blazor Server and Blazor WebAssembly configure `HttpClient` slightly differently, because of where the code actually runs — but once configured, the code you write against it is identical either way.

10.1.1 Blazor WebAssembly

In WASM, requests genuinely leave the user’s browser, so `HttpClient` needs a base address up front, typically the same origin the app was served from:

Program.cs (WebAssembly client project)

```
builder.Services.AddScoped(sp => new HttpClient
{
    BaseAddress = new Uri(builder.HostEnvironment.BaseAddress)
});
```

10.1.2 Blazor Server

In Server, your component code runs on the server already, so calling “your own” API means making a genuine outbound HTTP call from server to server (or, more efficiently, injecting the API’s services directly if they live in the same process — see the note below). Register a **typed client** pointing at wherever the API actually lives:

Program.cs (Server project)

```
builder.Services.AddHttpClient<ITaskService, HttpTaskService>(client =>
{
```

```
client.BaseAddress = new Uri("https://localhost:7100/"); // the API
↪ project's address
});
```

Note If your Blazor Server app and your API are the same ASP.NET Core project (a common setup for small applications), you don't need `HttpClient` at all for server-side code — just inject your data-access or business-logic services directly, the way `InMemoryTaskService` was injected in Chapter 9. Reach for `HttpClient` when the API is a genuinely separate service, which is the norm for anything beyond a small application, and is required for Blazor WebAssembly regardless, since WASM code cannot access a server-side database directly.

10.2 Replacing the in-memory service with a real HTTP-backed one

Services/HttpTaskService.cs

```
using System.Net.Http.Json;

namespace TaskBoard.Services;

public class HttpTaskService : ITaskService
{
    private readonly HttpClient http;

    public HttpTaskService(HttpClient http) => this.http = http;

    public async Task<List<TaskItem>> GetAllAsync()
    {
        var result = await
            ↪ http.GetFromJsonAsync<List<TaskItem>>("api/tasks");
        return result ?? new List<TaskItem>();
    }
}
```

```

public async Task<TaskItem?> GetByIdAsync(int id) =>
    await http.GetFromJsonAsync<TaskItem>($"api/tasks/{id}");

public async Task<int> CreateAsync(NewTaskModel model)
{
    var response = await http.PostAsJsonAsync("api/tasks", model);
    response.EnsureSuccessStatusCode();
    var created = await
        ↳ response.Content.ReadFromJsonAsync<TaskItem>();
    return created!.Id;
}

public async Task UpdateAsync(TaskItem task)
{
    var response = await http.PutAsJsonAsync($"api/tasks/{task.Id}",
        ↳ task);
    response.EnsureSuccessStatusCode();
}

public async Task DeleteAsync(int id)
{
    var response = await http.DeleteAsync($"api/tasks/{id}");
    response.EnsureSuccessStatusCode();
}
}

```

System.Net.Http.Json (already imported via the default `_Imports.razor` shown in Chapter 3) provides `GetFromJsonAsync`, `PostAsJsonAsync`, and friends — thin wrappers over `HttpClient` that handle JSON serialization and deserialization with `System.Text.Json` for you, so you're never manually calling `JsonSerializer.Serialize` in application code.

Update the DI registration to swap implementations — notice that **no component needs to change**, because every component depends on the `ITaskService` interface, not on `HttpTaskService` or `InMemoryTaskService` directly. This is the entire payoff of coding to an interface, made concrete:

```

// builder.Services.AddScoped<ITaskService, InMemoryTaskService>(); //
↳ old
builder.Services.AddHttpClient<ITaskService, HttpTaskService>(client =>
{

```

```
client.BaseAddress = new Uri("https://localhost:7100/");  
});
```

10.3 Handling loading, success, and error states

Every real API call has three outcomes your UI needs to represent: it's in progress, it succeeded, or it failed. Skipping the error case is one of the most common gaps between tutorial code and production code.

Components/Pages/TaskList.razor

```
@page "/tasks"  
@inject ITaskService TaskService  
  
@if (isLoading)  
{  
    <p>Loading tasks...</p>  
}  
else if (loadError is not null)  
{  
    <div class="alert alert-danger">  
        Could not load tasks: @loadError  
        <button @onclick="LoadTasksAsync">Retry</button>  
    </div>  
}  
else if (tasks.Count == 0)  
{  
    <p>No tasks yet. Create your first one!</p>  
}  
else  
{  
    <ul>  
        @foreach (var task in tasks)  
        {  
            <TaskCard Task="task" />  
        }  
    </ul>  
}  
  
@code {
```

```
private List<TaskItem> tasks = new();
private bool isLoading = true;
private string? loadError;

protected override async Task OnInitializedAsync() => await
↪ LoadTasksAsync();

private async Task LoadTasksAsync()
{
    isLoading = true;
    loadError = null;
    try
    {
        tasks = await TaskService.GetAllAsync();
    }
    catch (HttpRequestException ex)
    {
        loadError = "The task service is unreachable. Check your
↪ connection.";
        Logger.LogError(ex, "Failed to load tasks");
    }
    finally
    {
        isLoading = false;
    }
}
```

Catching `HttpRequestException` specifically (rather than a bare `catch`) is deliberate: it distinguishes “the network call itself failed” from a programming bug elsewhere in the method, which you generally do *not* want to silently swallow and present to the user as if it were a normal network hiccup.

10.4 Cancellation for components that navigate away mid-request

If a user navigates away from `TaskList` while a slow request is still in flight, the response comes back and tries to update a component that no longer exists — harmless in most cases, but worth guarding against for anything expensive or long-running, using a `CancellationTokenSource` tied to the component’s lifetime.

```
@implements IDisposable

@code {
    private readonly CancellationTokenSource cts = new();

    protected override async Task OnInitializedAsync()
    {
        tasks = await TaskService.GetAllAsync(cts.Token);
    }

    public void Dispose()
    {
        cts.Cancel();
        cts.Dispose();
    }
}
```

This requires threading a `CancellationToken` parameter through your service interface (`Task<List<TaskItem>> GetAllAsync(CancellationToken ct = default)`), which is good practice for any I/O-bound method regardless of whether Blazor is involved.

10.5 Configuring resilience with `HttpClientFactory` policies

For anything talking to a network that might be flaky — which is all networks, eventually — configure retry behavior once, centrally, rather than wrapping every call site in a manual retry loop. .NET 8's `Microsoft.Extensions.Http.Resilience` package (built on Polly) integrates directly with the typed-client registration you already have:

```
$ dotnet add package Microsoft.Extensions.Http.Resilience
```

```
builder.Services.AddHttpClient<ITaskService, HttpTaskService>(client =>
{
    client.BaseAddress = new Uri("https://localhost:7100/");
```

```
})  
.AddStandardResilienceHandler(); // retries, circuit breaker, timeout –  
↪ sensible defaults
```

This single line adds automatic retries with exponential backoff, a circuit breaker that stops hammering a genuinely-down dependency, and an overall timeout — the kind of resilience a production service needs, applied uniformly to every call made through `HttpTaskService`.

10.6 Try it yourself

1. Add a `SearchTasksAsync(string query)` method to `ITaskService` and `HttpTaskService`, calling `api/tasks/search?q=...`, and wire it to a search box on the task list page with a proper loading state.
2. Simulate a slow network by adding `await Task.Delay(2000)` inside a test API endpoint, and confirm your loading UI actually appears rather than flashing by unnoticed.
3. Add `AddStandardResilienceHandler()` to your `HttpTaskService` registration, then temporarily point `BaseAddress` at a URL that doesn't exist, and observe the retry attempts in your console log output.

Your components now load and save real data over HTTP. The next chapter addresses a question that becomes pressing as soon as more than one page needs to share the same data: where should application-wide state actually live?

Working with a Real Database: EF Core

11.1 Where Entity Framework Core fits

Chapter 10 replaced the in-memory task service with one backed by `HttpClient`, calling an API. That API has to store data somewhere durable, and for the overwhelming majority of .NET backends, that means **Entity Framework Core (EF Core)** — Microsoft's object-relational mapper, letting you work with a database through ordinary C# classes and LINQ instead of hand-written SQL. This chapter builds the API side that `HttpTaskService` from Chapter 10 talks to.

Nothing here is Blazor-specific — EF Core is exactly as useful, and used exactly the same way, in any ASP.NET Core backend. That's a feature, not a limitation: what you learn here transfers directly to any other backend work you do.

11.2 Defining the data model

`TaskBoard.Domain/Entities/TaskEntity.cs`

```
namespace TaskBoard.Domain.Entities;

public class TaskEntity
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public string? Description { get; set; }
    public DateTime DueDate { get; set; }
    public bool IsDone { get; set; }
    public TaskPriority Priority { get; set; }
    public string OwnerId { get; set; } = "";
```

```
}  
  
public enum TaskPriority { Low, Medium, High }
```

11.3 The DbContext

A DbContext represents a session with the database — it tracks which entities exist, what's changed, and translates your LINQ queries into SQL.

TaskBoard.Infrastructure/Persistence/ApplicationDbContext.cs

```
using Microsoft.EntityFrameworkCore;  
using TaskBoard.Domain.Entities;  
  
namespace TaskBoard.Infrastructure.Persistence;  
  
public class ApplicationDbContext : DbContext  
{  
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext>  
        ↪ options)  
        : base(options) { }  
  
    public DbSet<TaskEntity> Tasks => Set<TaskEntity>();  
  
    protected override void OnModelCreating(ModelBuilder modelBuilder)  
    {  
        modelBuilder.Entity<TaskEntity>(entity =>  
        {  
            entity.Property(t => t.Title).IsRequired().HasMaxLength(100);  
            entity.Property(t => t.Priority).HasConversion<string>();  
            entity.HasIndex(t => t.OwnerId);  
        });  
    }  
}
```

HasConversion<string>() on Priority stores the enum as a readable string ("High") rather than an opaque integer — a small choice that makes the raw table data far easier to inspect and debug, at a negligible storage cost.

Register it in the API project:

```
$ dotnet add package Microsoft.EntityFrameworkCore.SqlServer
$ dotnet add package Microsoft.EntityFrameworkCore.Design
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString(
        ↪ "DefaultConnection"));
```

appsettings.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\|
    ↪ mssqllocaldb;Database=TaskBoard;Trusted_Connection=True;"
  }
}
```

11.4 Migrations: evolving the schema safely

Instead of hand-writing CREATE TABLE statements, EF Core generates them from your model, and tracks every schema change as a **migration** — a versioned, reviewable, reversible script.

```
$ dotnet ef migrations add InitialCreate
$ dotnet ef database update
```

The first command inspects your DbContext and entity classes, diffs them against the last known schema (or nothing, the first time), and generates a migration file:

Migrations/20260115120000_InitialCreate.cs (generated, abbreviated)

```
public partial class InitialCreate : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
```

```

migrationBuilder.CreateTable(
    name: "Tasks",
    columns: table => new
    {
        Id = table.Column<int>(nullable: false)
            .Annotation("SqlServer:Identity", "1, 1"),
        Title = table.Column<string>(maxLength: 100, nullable:
↪ false),
        DueDate = table.Column<DateTime>(nullable: false),
        IsDone = table.Column<bool>(nullable: false),
        Priority = table.Column<string>(nullable: false),
        OwnerId = table.Column<string>(nullable: false),
    },
    constraints: table => table.PrimaryKey("PK_Tasks", x =>
↪ x.Id));

migrationBuilder.CreateIndex(
    name: "IX_Tasks_OwnerId", table: "Tasks", column: "OwnerId");
}

protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropTable(name: "Tasks");
}
}

```

Always commit migration files to source control alongside the model change that produced them — a migration is part of your codebase’s history, not a generated artifact to .gitignore. Applying migrations automatically as part of your CI/CD pipeline (Chapter 20), against a staging database before production, is standard practice; running `dotnet ef database update` by hand against a production database, without it going through the same reviewed, tested pipeline as everything else, is a common source of production incidents.

11.5 Building the API endpoints

A minimal API (no separate controller class needed) exposing the CRUD operations `HttpTaskService` from Chapter 10 expects:

Program.cs (API project)

```
app.MapGet("/api/tasks", async (ApplicationDbContext db, ClaimsPrincipal
↳ user) =>
{
    var ownerId = user.FindFirstValue(ClaimTypes.NameIdentifier)!;
    return await db.Tasks.Where(t => t.OwnerId == ownerId).ToListAsync();
}).RequireAuthorization();

app.MapGet("/api/tasks/{id:int}", async (int id, ApplicationDbContext db)
↳ =>
{
    var task = await db.Tasks.FindAsync(id);
    return task is not null ? Results.Ok(task) : Results.NotFound();
}).RequireAuthorization();

app.MapPost("/api/tasks", async (NewTaskModel model, ApplicationDbContext
↳ db, ClaimsPrincipal user) =>
{
    var task = new TaskEntity
    {
        Title = model.Title,
        DueDate = model.DueDate,
        OwnerId = user.FindFirstValue(ClaimTypes.NameIdentifier)!,
    };
    db.Tasks.Add(task);
    await db.SaveChangesAsync();
    return Results.Created($"/api/tasks/{task.Id}", task);
}).RequireAuthorization();

app.MapDelete("/api/tasks/{id:int}", async (int id, ApplicationDbContext
↳ db, ClaimsPrincipal user) =>
{
    var task = await db.Tasks.FindAsync(id);
    if (task is null) return Results.NotFound();
    if (task.OwnerId != user.FindFirstValue(ClaimTypes.NameIdentifier))
        return Results.Forbid();

    db.Tasks.Remove(task);
    await db.SaveChangesAsync();
    return Results.NoContent();
}).RequireAuthorization();
```

Notice every endpoint checks `OwnerId` against the authenticated user, and carries `.RequireAuthorization()` — this is exactly the server-side enforcement Chapter 14 insisted on: the Blazor client’s `<AuthorizeView>` improves the user experience, but these checks, running on the server, are the actual security boundary.

11.6 Avoiding the N+1 query trap

A classic EF Core performance mistake, easy to introduce accidentally once relationships are involved: loading a list of entities, then lazily triggering a separate query *per item* for related data.

```
// BAD: one query for tasks, then one MORE query per task for its  
↳ comments  
var tasks = await db.Tasks.ToListAsync();  
foreach (var task in tasks)  
{  
    var comments = task.Comments; // triggers a lazy-load query, per task  
}
```

Fix it with eager loading, using `Include`, collapsing everything into a single, efficient query:

```
var tasks = await db.Tasks  
    .Include(t => t.Comments)  
    .ToListAsync();
```

For read-only queries — which is most of what a UI needs — add `.AsNoTracking()` too. It tells EF Core not to bother setting up change-tracking machinery for entities you’re only going to display, never update, which is a measurable performance win on any list endpoint of meaningful size.

```
var tasks = await db.Tasks.AsNoTracking()  
    .Where(t => t.OwnerId == ownerId)  
    .ToListAsync();
```

11.7 Try it yourself

1. Add a `TaskComment` entity with a foreign key back to `TaskEntity`, generate a migration for it, and apply it.
2. Write an endpoint that returns a task together with its comments using `Include`, and confirm (using EF Core's logging, enabled via `options.LogTo(Console.WriteLine)`) that it issues one query, not one-plus-N.
3. Add `.AsNoTracking()` to the `GET /api/tasks` endpoint above and confirm, via the same query logging, that EF Core's generated SQL and tracking behavior changes.

With a real, migration-managed database behind your API, the next chapter turns from data to presentation discipline: building a small, reusable component library so your growing set of TaskCard-style components has consistent styling and behavior across the whole application.

State Management

12.1 Three kinds of state, three different lifetimes

Not all “state” in an application behaves the same way, and treating it uniformly is where a lot of accidental complexity comes from. It’s worth separating:

1. **Component-local state** — a field inside one component’s `@code` block. Lives and dies with that component instance. Everything in Parts I and II so far is this kind.
2. **Shared application state** — data multiple, otherwise-unrelated components need to read or modify: the current user’s task filter preference, a shopping cart, a notification count in a nav bar badge. This is this chapter’s main subject.
3. **Server-authoritative state** — the actual source of truth, typically a database behind your API. Component and shared state are usually just a client-side cache or working copy of this.

12.2 The simplest shared state: a scoped service

You already have every tool needed for shared state — it’s simply a service (Chapter 9), registered with the right lifetime, that raises an event when it changes so subscribed components can react.

Services/TaskFilterState.cs

```
namespace TaskBoard.Services;

public class TaskFilterState
{
    public event Action? OnChange;

    private string status = "All";
}
```

```
public string Status
{
    get => status;
    set
    {
        if (status == value) return;
        status = value;
        OnChange?.Invoke();
    }
}
```

Register it as **Scoped** — remember from Chapter 9 that in Blazor Server, Scoped means “one instance per connected user,” which is exactly the isolation you want for per-user UI state:

```
builder.Services.AddScoped<TaskFilterState>();
```

Any component can now read and write the shared filter, and any component that cares about changes subscribes to `OnChange`:

Components/Layout/NavMenu.razor (excerpt)

```
@inject TaskFilterState FilterState

<select value="@FilterState.Status" @onchange="e => FilterState.Status =
↪ e.Value?.ToString() ?? \"All\"">
    <option value="All">All</option>
    <option value="Active">Active</option>
    <option value="Done">Done</option>
</select>
```

Components/Pages/TaskList.razor (excerpt)

```
@inject TaskFilterState FilterState
implements IDisposable

@code {
```

```
protected override void OnInitialized()
{
    FilterState.OnChange += StateHasChanged;
}

public void Dispose()
{
    FilterState.OnChange -= StateHasChanged;
}
}
```

`FilterState.OnChange += StateHasChanged` subscribes the component's own re-render method directly as the event handler — a concise and very common Blazor idiom. When `NavMenu` changes `FilterState.Status`, the `OnChange` event fires, `TaskList.StateHasChanged()` runs, and the list re-renders with the new filter applied — even though `NavMenu` and `TaskList` have no parameter relationship between them at all. This is the shared-service pattern doing exactly the job `CascadingValue` (Chapter 5) does for read-mostly data, but for state that changes from multiple, unrelated places.

12.3 Avoiding memory leaks with shared state

Every subscription in the pattern above **must** be paired with an unsubscription in `Dispose`, exactly as with the timer example in Chapter 6 — a `Scoped` state service lives as long as the user's session, so any component that subscribes and never unsubscribes leaks for the entire session, not just briefly.

12.4 When to reach for a dedicated state-management library

For small to medium applications, the `scoped-service-plus-event` pattern above is genuinely sufficient, and is what most production Blazor codebases use. As an application grows, teams sometimes adopt a more structured pattern, most commonly **Fluxor**, a Redux-style library for Blazor built around three ideas:

- **State** is a single, immutable record describing everything at once.

- **Actions** are plain objects describing “what happened” (`AddTaskAction`, `MarkTaskDoneAction`).
- **Reducers** are pure functions that take the current state and an action, and return a *new* state — never mutating the old one.

```
public record TaskState(ImmutableList<TaskItem> Tasks, bool IsLoading);

public record AddTaskAction(TaskItem Task);

public static class TaskReducers
{
    [ReducerMethod]
    public static TaskState ReduceAddTask(TaskState state, AddTaskAction
    ↪ action) =>
        state with { Tasks = state.Tasks.Add(action.Task) };
}
```

Components dispatch actions instead of calling methods directly, and subscribe to slices of state via injected `IState<TaskState>`. The benefit is traceability at scale: every state change in the entire app happened through exactly one mechanism (a dispatched action through a reducer), which makes large applications with many contributors easier to reason about and debug — every change has a name and a single place it’s handled. The cost is more ceremony for simple cases. As a rule of thumb: reach for the scoped-service pattern first, and only introduce Fluxor (or a similar library) once you’ve felt real pain from state changes being hard to trace across a large component tree — don’t adopt it preemptively for a small app.

12.5 Persisting state across a page reload

Neither component-local nor scoped-service state survives a full browser reload — a reload creates an entirely new circuit (Server) or reinitializes the WASM app from scratch. For state that should survive a reload (a draft the user was typing, a “remember my filter” preference), persist it to the browser using `ProtectedLocalStorage` (Blazor Server, encrypted using ASP.NET Core’s data protection) or plain `localStorage` via JS interop (WebAssembly, covered next chapter).

```
builder.Services.AddScoped<ProtectedLocalStorage>();
```

```
@inject ProtectedLocalStorage LocalStorage

@code {
    protected override async Task OnInitializedAsync()
    {
        var result = await LocalStorage.GetAsync<string>("taskFilter");
        if (result.Success)
        {
            FilterState.Status = result.Value!;
        }
    }

    private async Task SaveFilterAsync(string status)
    {
        FilterState.Status = status;
        await LocalStorage.SetAsync("taskFilter", status);
    }
}
```

12.6 Try it yourself

1. Add a NotificationCount shared state service that a task-creation flow increments and a nav-bar badge component displays, following the OnChange event pattern above.
2. Deliberately remove one Dispose unsubscription from a component using shared state, then open and close that page fifty times in a row (a simple for loop calling NavigationManager.NavigateTo in browser dev tools console can drive this), and confirm via logging that the event handler list keeps growing. Fix it and confirm the growth stops.
3. Persist the task filter using ProtectedLocalStorage so it survives a full page reload.

Shared state services solve “how do two unrelated components talk to each other” from the C# side. The next chapter covers the other direction entirely: talking to the

browser's own JavaScript APIs, and scoping CSS so components don't leak styles onto each other.

JavaScript Interop and CSS Isolation

13.1 Why you sometimes still need JavaScript

Blazor lets you avoid writing JavaScript for the vast majority of an application's logic, but a handful of things are only reachable through the browser's JavaScript APIs: reading `localStorage` in WebAssembly, calling a third-party JS charting library, triggering a browser file download, or reading precise element measurements for a custom layout. **JS interop** is the bridge that lets your C# call into JavaScript, and lets JavaScript call back into your C#.

13.2 Calling JavaScript from C#: `IJSRuntime`

Inject `IJSRuntime` and call `InvokeAsync<T>`, passing the name of a JavaScript function and its arguments.

`wwwroot/js/taskboard.js`

```
window.taskboard = {
  showAlert: function (message) {
    alert(message);
  },
  getLocalStorage: function (key) {
    return localStorage.getItem(key);
  },
  setLocalStorage: function (key, value) {
    localStorage.setItem(key, value);
  }
};
```

Reference the script in your root document:

Components/App.razor (add near the closing `</body>`)

```
<script src="js/taskboard.js"></script>
```

Components/Pages/TaskList.razor

```
@inject IJSRuntime JS

<button @onclick="ShowSavedMessage">Save</button>

@code {
    private async Task ShowSavedMessage()
    {
        await JS.InvokeVoidAsync("taskboard.showAlert", "Tasks saved!");
    }

    private async Task<string?> LoadFilterAsync() =>
        await JS.InvokeAsync<string?>("taskboard.getLocalStorage",
↵  "taskFilter");
}
```

`InvokeVoidAsync` is for JS functions that don't return a value you care about; `InvokeAsync<T>` deserializes the JS return value into a C# type `T` (JSON-compatible types only — strings, numbers, booleans, and JSON-serializable objects).

13.3 Preferring JS modules over global scripts

Attaching functions to `window` (as above) works but pollutes the global namespace and can collide with other scripts. The modern, preferred approach uses ES modules, loaded on demand:

wwwroot/js/taskboard.js (as an ES module — no `window.` needed)

```
export function showAlert(message) {
    alert(message);
}
```



```

}

export function focusElement(element) {
    element.focus();
}

```

```

@Inject IJSRuntime JS
@implements IAsyncDisposable

@code {
    private IJSObjectReference? module;

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            module = await JS.InvokeAsync<IJSObjectReference>(
                "import", "./js/taskboard.js");
        }
    }

    private async Task ShowAlertAsync(string message)
    {
        if (module is not null)
            await module.InvokeVoidAsync("showAlert", message);
    }

    public async ValueTask DisposeAsync()
    {
        if (module is not null)
            await module.DisposeAsync();
    }
}

```

Loading the module in `OnAfterRenderAsync` (rather than `OnInitializedAsync`) matters: `import` is a browser API, and the JS interop channel to the browser isn't fully ready until after the first render — calling JS-returning interop before that point throws in some hosting models. Disposing the module reference in `DisposeAsync` releases the corresponding JS-side reference, avoiding a memory leak on the JavaScript side that mirrors the C#-side leaks discussed in Chapter 6.

13.4 Working with DOM elements directly: `ElementReference`

To pass an actual rendered DOM element to a JS function (to focus it, measure it, or attach a third-party widget to it), capture it with `@ref`.

```
<input @ref="titleInput" />
<button @onclick="FocusTitle">Focus title field</button>

@code {
    private ElementReference titleInput;

    private async Task FocusTitle()
    {
        await titleInput.FocusAsync(); // built-in helper, no custom JS
        ↪ needed
    }
}
```

`FocusAsync()` is a convenience method Blazor ships for the extremely common “focus this element” case. For anything more custom, pass the `ElementReference` into your own JS function, where it arrives as the actual DOM element:

```
export function highlightElement(element) {
    element.style.outline = "2px solid orange";
}
```

```
await module.InvokeVoidAsync("highlightElement", titleInput);
```

13.5 Calling C# from JavaScript

The reverse direction — JavaScript initiating a call into your C# — uses `[JSInvokable]`.

```
[JSInvokable]
public static Task<int> GetTaskCount()
{
```

```
return Task.FromResult(currentTaskCount);
}
```

```
DotNet.invokeMethodAsync('TaskBoard', 'GetTaskCount')
    .then(count => console.log(`There are ${count} tasks`));
```

For **instance** methods (rather than static ones), pass a `DotNetObjectReference` to JavaScript first:

```
@code {
    private DotNetObjectReference<TaskList>? dotNetRef;

    protected override void OnAfterRender(bool firstRender)
    {
        if (firstRender)
        {
            dotNetRef = DotNetObjectReference.Create(this);
            // pass dotNetRef into a JS function that will call back
↪ later
        }
    }

    [JSInvokable]
    public void HandleJavaScriptCallback(string message)
    {
        log = message;
        StateHasChanged(); // required here – this call didn't originate
↪ from a Blazor-tracked event
    }

    public void Dispose() => dotNetRef?.Dispose();
}
```

Note the explicit `StateHasChanged()` call and the `Dispose()` on the reference — a JS-initiated callback is exactly the kind of “Blazor isn’t watching this” scenario from Chapter 6, and the `DotNetObjectReference` itself is an unmanaged-feeling resource that must be disposed to avoid leaking.

13.6 CSS isolation: styles that stay with their component

Without any special effort, a `<style>` block or a shared stylesheet applies globally — a `.card` class defined for `TaskCard` might accidentally also style an unrelated `.card` element elsewhere in the app. Blazor’s **CSS isolation** solves this automatically: create a file named `<ComponentName>.razor.css` next to your component, and its rules are scoped to *only* that component’s markup, no matter what selectors you write.

Components/TaskCard.razor.css

```
.card {  
    border: 1px solid #ddd;  
    border-radius: 6px;  
    padding: 0.75rem 1rem;  
    margin-bottom: 0.5rem;  
}  
  
.card.done {  
    opacity: 0.6;  
    text-decoration: line-through;  
}
```

Components/TaskCard.razor

```
<li class="card @(Task.IsDone ? "done" : "")">  
    @Task.Title  
</li>
```

Behind the scenes, Blazor rewrites this at build time: it adds a unique, auto-generated attribute (something like `b-3xy8f2k1z9`) to every element `TaskCard` renders, and rewrites `.card { ... }` in the CSS file to `.card[b-3xy8f2k1z9] { ... }`. You never see or write that attribute yourself — it’s entirely a build-time mechanism — but it’s worth knowing it exists, because it explains why a CSS-isolated selector won’t match an element rendered by a *child* component (that child has its own, different scope attribute) even if you nest the child’s markup visually inside the parent.

13.6.1 Sharing styles across components deliberately

If you genuinely want a style to apply to a child component’s rendered output from the parent’s isolated stylesheet, use the `::deep` combinator:

```
/* In TaskList.razor.css */  
::deep .card {  
    box-shadow: 0 1px 3px rgba(0,0,0,0.1);  
}
```

This tells the isolation system “apply this rule to any `.card` element found anywhere inside what `TaskList` renders, including inside child components” — an intentional, explicit escape hatch rather than accidental global leakage.

13.7 Try it yourself

1. Write a JS module function that copies a task’s title to the clipboard using `navigator.clipboard.writeText`, and wire it to a “Copy” button on `TaskCard`.
2. Add a `TaskCard.razor.css` file (if you haven’t already) giving each priority level a distinct left-border color, and confirm in browser dev tools that the generated scope attribute is present and that the rule doesn’t leak onto unrelated elements.
3. Use `@ref` and a small JS module to measure the height of the task list container and log it to the console after the component first renders.

You can now reach into the browser when Blazor’s own model isn’t enough, and keep your CSS from becoming a global free-for-all as the app grows. The next chapter finally gives the full technical explanation of Server, WebAssembly, and Auto render modes promised back in Chapter 1 — now that you have the vocabulary (components, DI, JS interop) to understand exactly what differs underneath each one.

Render Modes in Depth

14.1 Recap: what a render mode actually controls

A render mode answers exactly one question for a given component: **where does this component’s C# execute, and by what channel does the browser learn about UI changes?** Everything else — your markup, your parameters, your event handlers, your lifecycle methods — is identical regardless of the answer. This chapter explains the mechanics precisely enough that you can make an informed choice per page, rather than treating it as a coin flip.

14.2 Static server rendering (the default, if you specify nothing)

Before .NET 8’s interactive modes even enter the picture, every Blazor component can be rendered **statically**: the server runs your component’s markup-generation logic exactly once per HTTP request, produces plain HTML, and sends it to the browser — no persistent connection, no interactivity after the page loads, functionally identical to a classic server-rendered page (like old ASP.NET Razor Pages). Buttons won’t respond to clicks; there’s no C# left running to respond to them. This is the fastest possible mode for content that’s genuinely just... content: a terms-of-service page, a blog post, a marketing landing page.

14.3 Interactive Server

Add `@rendermode InteractiveServer` to a component (or apply it globally, as the default template does in `Program.cs` via `AddInteractiveServerRenderMode()`), and the component now runs its C# on the server for the entire duration the user has the

page open, communicating over a persistent SignalR connection (the same underlying technology covered fully in Chapter 17).

```
@page "/tasks"  
@rendermode InteractiveServer  
  
<h3>My Tasks</h3>
```

What actually happens on a click: the browser’s small Blazor Server JavaScript runtime captures the click, serializes “button X was clicked” into a message, sends it over the open SignalR connection to the server. The server locates the correct component instance (kept alive in server memory for this specific user’s circuit), invokes your `@onclick` handler, computes the new render tree, diffs it against the previous one, and sends back only the *differences* as a compact binary patch. The browser’s JS runtime applies that patch to the real DOM.

This means: your C# breakpoints work exactly as they would debugging any server app (because it *is* a server app); no .NET runtime is downloaded to the browser at all (tiny initial payload); but every interaction has a network round-trip, and the server holds live, per-user memory (the “circuit”) for as long as that browser tab is open — 20,000 concurrently open tabs means 20,000 live circuits your server is managing.

14.4 Interactive WebAssembly

```
@page "/tasks"  
@rendermode InteractiveWebAssembly
```

Here, the browser downloads: a stripped-down .NET runtime compiled to WebAssembly, your compiled application DLLs, and the .NET base class library DLLs your app actually uses (trimmed to remove what it doesn’t). All of this runs **inside the browser’s own WebAssembly sandbox** — no server connection is needed for the app to function once loaded, and clicking a button runs your C# `@onclick` handler on the user’s own CPU, updates an in-browser representation of the render tree, and Blazor’s WASM-side rendering logic patches the real DOM directly, with zero network round-trip.

The cost is the initial download (the runtime plus your app, though this is cached by the browser after the first visit) and the fact that anything requiring server-only resources — a database connection string, a secret API key — simply cannot live in this code, because it's running on hardware you don't control, inside a browser the user could inspect. Server access *must* go through an HTTP API call from this point on, exactly as shown in Chapter 10.

14.5 Interactive Auto

```
@page "/tasks"  
@rendermode InteractiveAuto
```

This is the .NET 8 hybrid: on a user's **first ever** visit, Blazor uses the Server model (fast first paint, small download) while quietly downloading the WebAssembly runtime and your app's assemblies in the background, using the browser's caching so this only happens once. On a **subsequent** visit (the cache is warm), Blazor skips Server entirely and boots straight into WebAssembly.

The appeal is obvious: fast first load *and* offline-capable, zero-latency interaction on every visit after the first — but it comes with a real constraint worth internalizing now: **your component's code must be able to run correctly under both models**, because it genuinely will, for different users at different times. Concretely, this means:

- Any service you inject must be registered in *both* the server project and the client (WASM) project.
- Code that only makes sense server-side (raw database access, file system access) must be moved behind an HTTP API call, exactly as if you'd committed to pure WebAssembly — you cannot “cheat” and rely on server-only APIs just because the first visit happens to run on the server.

14.6 Where to apply render modes: per-component vs. globally

You saw `@rendermode` on an individual `@page` component above. You can also set it as the *default* for the whole app, from `Program.cs`:

```
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode()
    .AddInteractiveWebAssemblyRenderMode();
```

And apply it once for many components using `@rendermode` on a shared layout or root component, rather than repeating it on every single page — but be deliberate here: not every page in a real application needs the same mode. A public marketing page benefits from static rendering (fastest, most SEO-friendly); an admin dashboard used by a handful of internal staff is a fine candidate for `InteractiveServer` (simplifies deployment, no need to worry about API surface exposure); a customer-facing, highly-interactive page used by thousands of concurrent users benefits from `InteractiveWebAssembly` or `InteractiveAuto` to keep server load and connection count down.

14.7 Prerendering: getting a fast first paint even in interactive modes

By default, interactive components are **prerendered**: the server renders the component to static HTML *first* (exactly like static rendering) and sends that immediately, so the user sees content right away, and *then* the interactive runtime (Server or WASM) attaches to that already-rendered markup and “wakes it up.” This is why `OnInitializedAsync` can appear to run twice in Blazor Server if you’re not careful — once during the prerender pass, and once again when the interactive circuit truly starts. If a component has side effects that must run exactly once (sending an email, incrementing an analytics counter), guard against this or disable prerendering for that component:

```
@page "/"tasks"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
```

14.8 A decision guide

Scenario	Recommended mode
Static marketing/content page	Static (no @rendermode at all)
Small internal tool, few concurrent users	InteractiveServer
Public app, must work offline, many concurrent users	InteractiveWebAssembly
Public app, want fast first load <i>and</i> offline capability	InteractiveAuto
Anything touching secrets, databases, or server-only APIs directly	InteractiveServer (or a static/API-backed page)

14.9 Try it yourself

1. Apply `InteractiveServer` to one page and `InteractiveWebAssembly` to another in the same project, and open your browser's Network tab to compare what's actually downloaded for each.
2. Add a `Console.WriteLine` (or `Logger.LogInformation`) inside `OnInitializedAsync` on an `InteractiveServer` page with prerendering enabled, run the app, and count how many times the log line appears for a single page load. Then set `prerender: false` and confirm it changes.
3. Take the `TaskList` page and deliberately try `InteractiveAuto` — confirm every service it depends on (`ITaskService` and anything it, in turn, needs) is registered in both the server-side and client-side DI containers, and fix any that aren't.

Part II is complete: you can build a multi-page, stateful, API-backed Blazor application and choose deliberately how it runs. Part III moves into concerns that separate a

demo from a production system: securing it, making it fast, testing it automatically, and adding real-time features.

Building a Component Library and Design System

15.1 The problem a growing app runs into

By this point, TaskBoard has buttons, form fields, cards, and alerts scattered across several pages, each styled a little differently because each was written at a different point in the book without a shared design vocabulary. In a real team, this is how inconsistent UIs happen: not from anyone deciding to make things inconsistent, but from nobody owning a shared set of building blocks. A **component library** is that shared set — a small collection of generic, reusable, well-tested components that every page builds from, instead of each page inventing its own button styling.

15.2 Starting with primitives: `Button`

`Components/UI/Button.razor`

```
<button class="btn btn-@Variant.ToString().ToLower() @(IsBlock ?
↪  "btn-block" : "")"
    disabled="@Disabled"
    @onclick="OnClick">
    @ChildContent
</button>

@code {
    [Parameter] public RenderFragment? ChildContent { get; set; }
    [Parameter] public ButtonVariant Variant { get; set; } =
↪  ButtonVariant.Primary;
    [Parameter] public bool IsBlock { get; set; }
    [Parameter] public bool Disabled { get; set; }
    [Parameter] public EventCallback OnClick { get; set; }
```

```
}  
  
public enum ButtonVariant { Primary, Secondary, Danger, Ghost }
```

Components/UI/Button.razor.css

```
.btn {  
    padding: 0.5rem 1rem;  
    border-radius: 6px;  
    border: none;  
    font-weight: 600;  
    cursor: pointer;  
    transition: background-color 0.15s ease;  
}  
  
.btn-primary { background: #5b2a86; color: white; }  
.btn-secondary { background: #e9e4f0; color: #3a1b57; }  
.btn-danger { background: #c0392b; color: white; }  
.btn-ghost { background: transparent; color: #5b2a86; }  
.btn:disabled { opacity: 0.5; cursor: not-allowed; }  
.btn-block { width: 100%; }
```

Every page that previously wrote its own `<button>` now writes:

```
<Button Variant="ButtonVariant.Danger" OnClick="() =>  
    ↪ tasks.Remove(task)">  
    Delete  
</Button>
```

The payoff is not merely fewer keystrokes — it’s that changing what a “danger” button looks like, application-wide, is now a one-file change instead of a find-and-replace across dozens of components.

15.3 A small set of primitives goes a long way

Realistically, most applications need only a handful of these before diminishing returns set in: Button, Card, Badge, Alert, Modal (built in Chapter 5’s exercises), and the FormField wrapper from Chapter 8. Resist the urge to build an enormous, speculative library up front — extract a component into the shared library the *second* time

a pattern repeats, not the first time you write it, and not preemptively for patterns that might repeat someday.

Components/UI/Badge.razor

```
<span class="badge
↳ badge-@Color.ToString().ToLower()">@ChildContent</span>

@code {
    [Parameter] public RenderFragment? ChildContent { get; set; }
    [Parameter] public BadgeColor Color { get; set; } =
↳ BadgeColor.Neutral;
}

public enum BadgeColor { Neutral, Success, Warning, Danger }
```

```
<Badge Color="@(task.Priority == TaskPriority.High ? BadgeColor.Danger :
↳ BadgeColor.Neutral)">
    @task.Priority
</Badge>
```

15.4 Design tokens: one source of truth for colors and spacing

Rather than repeating hex codes (#5b2a86) across a dozen .razor.css files, define them once as CSS custom properties, and have every component reference the token, not the raw value.

wwwroot/css/tokens.css

```
:root {
    --color-primary: #5b2a86;
    --color-primary-dark: #3a1b57;
    --color-danger: #c0392b;
    --color-surface: #f5f3f8;
    --color-border: #dcd5e6;
```

```
--space-xs: 0.25rem;
--space-sm: 0.5rem;
--space-md: 1rem;
--space-lg: 1.5rem;

--radius-sm: 4px;
--radius-md: 6px;

}
```

```
/* Button.razor.css, updated to use tokens */
.btn-primary { background: var(--color-primary); color: white; }
.btn-danger { background: var(--color-danger); color: white; }
```

Reference `tokens.css` once, globally, in `App.razor` — never per-component — so there’s exactly one place a rebrand or a dark-mode variant needs to touch:

```
<link rel="stylesheet" href="css/tokens.css" />
```

15.4.1 A basic dark-mode variant, using the same tokens

```
@media (prefers-color-scheme: dark) {
  :root {
    --color-surface: #241a30;
    --color-border: #3d2f4f;
  }
}
```

Because every component reads `var(--color-surface)` rather than a hardcoded value, this single media query change ripples through the entire component library automatically — no component itself needed to change.

15.5 Documenting the library with a living style guide page

A component library nobody can discover is barely better than no library at all — new team members, and you in six months, need a single place to see what exists before

reaching for a one-off <button>. A dedicated, unlisted page rendering every variant of every shared component (sometimes called a “living style guide” or a lightweight in-house Storybook) costs little to build and pays for itself the first time it prevents a duplicate component from being created.

Components/Pages/StyleGuide.razor

```
@page "/dev/style-guide"

<h1>Component Library</h1>

<h2>Buttons</h2>
<Button Variant="ButtonVariant.Primary">Primary</Button>
<Button Variant="ButtonVariant.Secondary">Secondary</Button>
<Button Variant="ButtonVariant.Danger">Danger</Button>
<Button Variant="ButtonVariant.Ghost">Ghost</Button>

<h2>Badges</h2>
<Badge Color="BadgeColor.Neutral">Neutral</Badge>
<Badge Color="BadgeColor.Success">Success</Badge>
<Badge Color="BadgeColor.Warning">Warning</Badge>
<Badge Color="BadgeColor.Danger">Danger</Badge>
```

Gate this page behind a development-only check (or an admin-only [Authorize] policy, from Chapter 14) so it never becomes a stray, confusing page a real end user can stumble onto in production.

15.6 Packaging the library as a separate project (when it's worth it)

For a single application, the shared components living in `TaskBoard.Web/Components/UI/` (Chapter 18's layout) is entirely sufficient. Once **more than one application** needs the same component library — a second internal tool, a public marketing site sharing the same look and feel — extract it into its own class library project (`TaskBoard.UI.csproj`, a plain Razor Class Library created with `dotnet new razorclasslib`), publish it as a NuGet package to your organization's internal feed, and version it independently. Do this when a second real consumer exists, not speculatively for a single application

that might, someday, need it — premature extraction adds packaging and versioning overhead for zero actual benefit until there's a second consumer to justify it.

15.7 Try it yourself

1. Extract the alert boxes used for loading/error states throughout earlier chapters (Chapter 10) into a shared `Alert` component with `Variant` (Info, Success, Danger) and `ChildContent` parameters.
2. Move every hardcoded color in your `.razor.css` files so far to reference the design tokens above instead.
3. Build the `/dev/style-guide` page and add every shared component you've built across this book to it, gated behind an `[Authorize(Roles = "Admin")]` check.

A consistent, well-documented set of building blocks makes every future page faster to build and harder to get visually wrong. The next chapter turns to a different, often-neglected kind of consistency: making sure the application actually works for every user, including those using assistive technology or a language other than the one you wrote it in.

PART III

Advanced Topics

Authentication and Authorization

16.1 The core abstractions

Regardless of render mode, Blazor’s authentication model is built on two ASP.NET Core abstractions you may already recognize: `ClaimsPrincipal` (representing “who is the current user, and what claims — roles, email, id — do they carry”) and `AuthenticationStateProvider` (an injectable service that answers “what’s the current authentication state right now,” asynchronously, since checking often means validating a token or cookie).

Blazor wraps these in a component-friendly layer: `<AuthorizeView>` for conditionally showing markup, and the `[Authorize]` attribute for guarding entire pages.

16.2 Guarding a page with `[Authorize]`

```
@page "/tasks"
[Authorize]

<h3>My Tasks</h3>
```

An unauthenticated user hitting `/tasks` is redirected (in a typical setup) to a login page, configured once in `Program.cs`. You can also restrict by role or policy:

```
[Authorize(Roles = "Admin")]
[Authorize(Policy = "CanManageTasks")]
```

16.3 Conditionally showing markup with `<AuthorizeView>`

Within a page (or shared layout) that anyone can reach, show different content depending on whether — and who — is logged in:

```
<AuthorizeView>
  <Authorized>
    <p>Welcome back, @context.User.Identity!.Name!</p>
    <button @onclick="Logout">Log out</button>
  </Authorized>
  <NotAuthorized>
    <a href="/login">Log in</a>
  </NotAuthorized>
</AuthorizeView>

<AuthorizeView Roles="Admin">
  <Authorized>
    <a href="/admin">Admin panel</a>
  </Authorized>
</AuthorizeView>
```

context inside `<Authorized>` is an `AuthenticationState`, giving you access to `context.User`, a `ClaimsPrincipal`.

16.4 Setting up ASP.NET Core Identity (Blazor Server / cookie-based)

For a Blazor Server app (or the server-hosted part of an Auto app) talking to its own database, ASP.NET Core Identity is the standard, well-tested starting point for account management: registration, login, password hashing, email confirmation, two-factor auth — all implemented and security-reviewed for you.

```
$ dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString(
        ↪ "DefaultConnection")));

builder.Services.AddIdentity<ApplicationUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();
```

```
builder.Services.AddCascadingAuthenticationState();
```

`AddCascadingAuthenticationState()` is what makes `<AuthorizeView>` and `[Authorize]` work throughout the component tree — it registers the plumbing that cascades the current `AuthenticationState` down automatically, using the exact `CascadingValue` mechanism from Chapter 5 under the hood.

The default .NET project templates (`dotnet new blazor -au Individual`) scaffold all of the login/register/logout Razor Pages and identity configuration automatically — for a real project, start from that scaffold rather than wiring Identity up by hand, and adapt from there.

16.5 Authentication in Blazor WebAssembly: token-based

A pure WASM app has no server-side session or cookie context of its own — it's a static app running in the browser, talking to APIs. The standard approach is token-based: the user authenticates against an identity provider (your own API, or an external one like Microsoft Entra ID, Auth0, or Okta), receives a JWT (JSON Web Token), and that token is attached to every subsequent API call.

```
builder.Services.AddOidcAuthentication(options =>
{
    builder.Configuration.Bind("Auth0", options.ProviderOptions);
});
```

```
<AuthorizeView>
  <Authorized>
    <p>Hi, @context.User.Identity!.Name</p>
  </Authorized>
  <NotAuthorized>
    <button @onclick="BeginLogin">Log in</button>
  </NotAuthorized>
</AuthorizeView>
```

```
@code {  
    [Inject] private NavigationManager Nav { get; set; } = default!;  
    private void BeginLogin() =>  
    ↪ Nav.NavigateToLogin("authentication/login");  
}
```

Attaching the token to outgoing API calls uses an `AuthorizationMessageHandler`, wired into your typed `HttpClient` registration:

```
builder.Services.AddHttpClient("TaskBoard.API", client =>  
    client.BaseAddress = new Uri("https://localhost:7100"))  
    .AddHttpMessageHandler<CustomAuthorizationMessageHandler>();
```

16.6 Custom authorization policies

Roles (“Admin”, “User”) are the simplest form of authorization, but real applications often need finer-grained rules — “can edit this specific task if you created it or you’re an admin.” Policies express this as reusable, named, testable logic.

```
builder.Services.AddAuthorization(options =>  
{  
    options.AddPolicy("CanManageTasks", policy =>  
        policy.RequireAssertion(context =>  
            context.User.IsInRole("Admin") ||  
            context.User.HasClaim("Permission", "ManageTasks")));  
});
```

```
@attribute [Authorize(Policy = "CanManageTasks")]
```

For rules that depend on the specific *resource* being accessed (not just the user’s general permissions), use `IAuthorizationService` directly inside a component or service:


```
[Inject] private IAuthorizationService AuthService { get; set; } =  
    ↪ default!;  
[CascadingParameter] private Task<AuthenticationState>? AuthStateTask {  
    ↪ get; set; }  
  
private async Task<bool> CanEditAsync(TaskItem task)  
{  
    var authState = await AuthStateTask!;  
    var result = await AuthService.AuthorizeAsync(authState.User, task,  
        ↪ "CanEditTask");  
    return result.Succeeded;  
}
```

16.7 Securing the API too — never trust the client alone

A critical point that trips up teams new to Blazor WebAssembly specifically: **[Authorize] and <AuthorizeView> only control what renders in the browser.** They are a *user-experience* mechanism, hiding buttons and redirecting navigation — they provide zero actual security on their own, because the compiled WebAssembly code sits entirely on a machine the user controls, and a sufficiently motivated user can bypass client-side checks trivially (browser dev tools, a modified request, a decompiled assembly). **Every API endpoint your app calls must independently enforce authorization on the server**, using [Authorize] on your API controllers or minimal API endpoints exactly as you would for any other backend, regardless of what the Blazor client already checked. Treat client-side authorization purely as a UI convenience; treat server-side authorization as the actual security boundary.

16.8 Try it yourself

1. Scaffold a fresh project with `dotnet new blazor -au Individual` and walk through registering and logging in as a new user, to see the full Identity flow Microsoft provides out of the box.
2. Add a `CanDeleteTask` policy that only succeeds for the task's original creator or an Admin, and apply it to the delete button in `TaskCard`.

3. Deliberately call your API's delete endpoint directly (using a tool like `curl` or Postman, bypassing the Blazor UI entirely) as a user without permission, and confirm the server rejects it — this proves your security boundary is actually on the server, not just hidden in the UI.

With access control in place, the next chapter turns to making the application fast — for the user's browser, for your server's memory, and for everyone waiting on your bundle to download.

Performance and Optimization

17.1 Measure before you optimize

Every performance chapter should open with this, and this one does: profile first. Browser DevTools' Performance and Network tabs, and .NET's own `dotnet-trace` and `dotnet-counters` tools for server-side work, tell you *where* time and bytes are actually going, which is very often not where intuition suggests. The techniques below are genuinely useful, but applying all of them blindly to a small internal tool used by ten people is wasted effort; applying none of them to a public, high-traffic app is a real problem. Match the effort to the actual stakes.

17.2 Reducing unnecessary re-renders

Blazor re-renders a component whenever `StateHasChanged` runs on it (directly, or as a consequence of a parameter changing on a parent that re-rendered). By default, when a parent re-renders, **every child re-renders too**, even if none of that child's actual parameter values changed. For a small component tree this is invisible; for a list of a thousand `TaskCards`, re-rendering all thousand because one checkbox changed is wasteful and, past a certain size, visibly slow.

17.2.1 ShouldRender

Override `ShouldRender()` to skip a re-render when you can prove nothing relevant changed:

```
private TaskItem? lastRenderedTask;

protected override bool ShouldRender()
{
    if (ReferenceEquals(Task, lastRenderedTask)) return false;
}
```

```
lastRenderedTask = Task;  
return true;  
}
```

17.2.2 Splitting components so re-renders stay localized

Often the cleanest fix isn't a manual `ShouldRender` check at all — it's making sure state that changes frequently lives in a small, isolated component rather than a large one. If a live “seconds ago” timestamp forces a re-render every second, keep that timestamp in its own tiny child component rather than in the parent that also renders the entire task list; only the tiny component re-renders every second, not the whole tree.

17.3 @key: helping Blazor's diffing algorithm

When rendering a list with `@foreach`, Blazor's diffing algorithm by default matches old and new elements positionally — element 3 in the old render is compared against element 3 in the new render. If items are reordered, inserted in the middle, or removed, this positional matching can cause Blazor to needlessly re-render or even lose UI state (like an open `<details>` element or focus) that logically belonged to a specific item, because it was structurally attached to a *position*, not an *identity*.

```
@foreach (var task in tasks)  
{  
    <TaskCard @key="task.Id" Task="task" />  
}
```

`@key` tells Blazor “match by this identity, not by position,” letting it correctly recognize that an item merely moved rather than being destroyed and recreated. For any list backed by data with a stable identifier (which is almost always — a database Id), add `@key` as a matter of habit, not just when you notice a bug.

17.4 Virtualization for long lists

Rendering ten thousand `TaskCards` produces ten thousand real DOM nodes, almost all of which are scrolled off-screen and invisible at any given moment — pure waste.

Blazor's built-in `<Virtualize>` component renders only the items currently visible (plus a small buffer), dramatically cutting DOM size and render time for long lists.

```
<Virtualize Items="tasks" Context="task">
    <TaskCard @key="task.Id" Task="task" />
</Virtualize>
```

For data too large to hold in memory at once, `<Virtualize>` also supports an `ItemsProvider` delegate that fetches only the requested page of data on demand, straight from your API:

```
<Virtualize ItemsProvider="LoadTasksAsync" Context="task">
    <TaskCard @key="task.Id" Task="task" />
</Virtualize>

@code {
    private async ValueTask<ItemsProviderResult<TaskItem>>
    ↪ LoadTasksAsync(ItemsProviderRequest request)
    {
        var page = await TaskService.GetPageAsync(request.StartIndex,
    ↪ request.Count, request.CancellationToken);
        return new ItemsProviderResult<TaskItem>(page.Items,
    ↪ page.TotalCount);
    }
}
```

17.5 WebAssembly-specific: trimming and AOT compilation

17.5.1 Trimming

The .NET runtime and base class library are large; a WASM app ships only what it actually uses, thanks to the .NET **trimmer**, which analyzes your compiled app and removes unreferenced code. Trimming is on by default for WASM apps in Release builds; you rarely need to configure it, but reflection-heavy code (including some JSON serialization patterns and dependency injection edge cases) can confuse the trimmer into removing something actually needed at runtime — if you see a

`MissingMethodException` only in a trimmed Release build but not in Debug, this is almost always the cause, and the fix is a `[DynamicDependency]` attribute or a small trimmer configuration file telling it to preserve the specific type.

17.5.2 Ahead-of-time (AOT) compilation

By default, WASM apps ship your app's IL (intermediate language) and interpret it in the browser. Enabling AOT compiles your C# all the way down to WebAssembly's native instruction format at build time, trading a noticeably larger download for substantially faster runtime execution — often 2-3x for CPU-heavy code.

```
<PropertyGroup>
  <RunAOTCompilation>true</RunAOTCompilation>
</PropertyGroup>
```

Because AOT increases download size, it's a genuine trade-off, not a free win: for an app whose bottleneck is initial load time (a mostly-idle, UI-light app), AOT can actively hurt the user's experience; for a CPU-bound app (heavy client-side data processing, a game, complex calculations), it's usually a clear win. Measure both ways before committing.

17.5.3 Lazy loading assemblies

If only a small, rarely-visited part of your app needs a large dependency (a charting library, a PDF generator), you don't have to pay for its download on every visit — load the assembly only when the user navigates to the page that needs it.

```
<ItemGroup>
  <BlazorWebAssemblyLazyLoad Include="ReportGenerator.dll" />
</ItemGroup>
```

```
protected override async Task OnNavigateAsync(NavigationContext args)
{
    if (args.Path.EndsWith("/reports"))
    {
```

```

        await LazyAssemblyLoader.LoadAssembliesAsync(new[] {
↪   "ReportGenerator.dll" });
    }
}

```

17.6 Server-specific: circuit and memory management

For Blazor Server, the main resource under your control isn't download size — it's server memory and connection count, since every open browser tab is a live circuit consuming server-side resources for its entire duration.

- **Dispose aggressively.** Chapters 6 and 11 already emphasized this, and it matters most here: every subscription a component holds is memory kept alive per user, for as long as their circuit lives.
- **Tune circuit retention.** By default, a disconnected circuit (user's Wi-Fi blipped, tab was backgrounded) is kept alive for a short grace period so a reconnect can resume seamlessly. Tune this based on your actual usage pattern:

```

builder.Services.AddServerSideBlazor(options =>
{
    options.DisconnectedCircuitMaxRetained = 100;
    options.DisconnectedCircuitRetentionPeriod = TimeSpan.FromMinutes(3);
});

```

- **Watch payload size per interaction.** Since every update crosses the network, avoid triggering large re-renders (a huge unvirtualized list, Chapter 15's earlier point) in direct response to frequent events like `oninput` on a search box — debounce instead:

```

<input @oninput="OnSearchInput" />

@code {
    private CancellationTokensource? debounceCts;

    private async Task OnSearchInput(ChangeEventArgs e)

```

```
{
    debounceCts?.Cancel();
    debounceCts = new CancellationTokenSource();
    var token = debounceCts.Token;
    try
    {
        await Task.Delay(300, token);
        await RunSearchAsync(e.Value?.ToString() ?? "");
    }
    catch (TaskCanceledException) { /* superseded by a newer
↪ keystroke */ }
}
```

17.7 Try it yourself

1. Add `@key="task.Id"` to every list you've built so far in the TaskBoard app, and confirm in the browser's Elements panel that reordering the underlying list no longer recreates DOM nodes unnecessarily.
2. Convert the task list to use `<Virtualize>` with an `ItemsProvider`, backed by a paged endpoint on `ITaskService`, and generate a thousand fake tasks to see the difference in initial render time with and without it.
3. If you have a Blazor WebAssembly project handy, toggle `RunAOTCompilation` on and off, publish both ways (`dotnet publish -c Release`), and compare the resulting `wwwroot` folder size and a rough load-time measurement in your browser's Network tab.

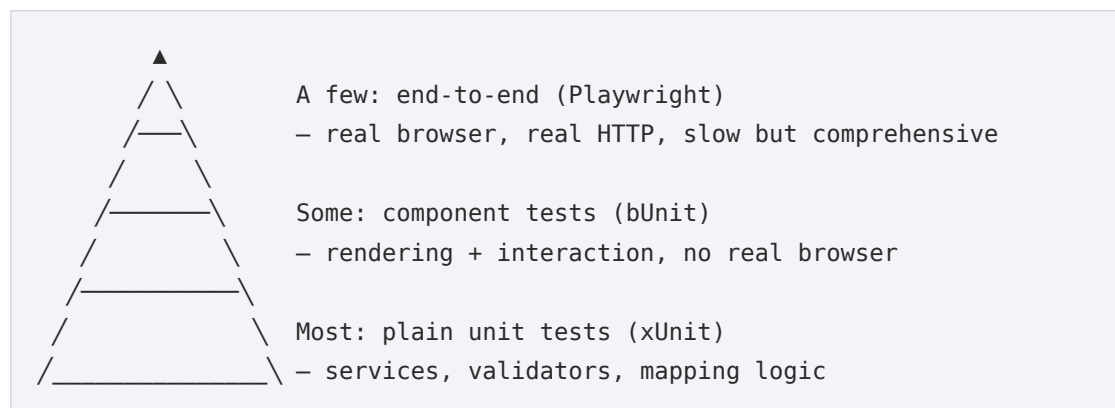
Fast and secure only matters if it's also *correct* — and correct, for a codebase more than one person will ever touch, means automatically verified. The next chapter covers testing Blazor components and full user flows.

Testing Blazor Applications

18.1 Why testing Blazor doesn't feel like testing plain C

A Blazor component isn't a plain class you can new up and call a method on in a unit test — it needs a rendering context, a way to simulate clicks, and a way to inspect the resulting markup. Two libraries cover the two levels that matter: **bUnit**, for testing components in isolation without a real browser, and **Playwright**, for testing complete user flows through a real, automated browser. Both complement, rather than replace, ordinary xUnit tests on your plain C# classes (services, validators, business logic) — those still need no special tooling at all and should still make up the bulk of your test suite.

18.2 The testing pyramid, applied to Blazor



Keep this shape. A large number of fast, plain unit tests catches most logic bugs cheaply; a moderate number of component tests catches rendering and interaction bugs; a small number of end-to-end tests catches “does the whole thing actually work together,” which is valuable but expensive to run and maintain at scale.

18.3 Unit testing plain services first

Before touching any Blazor-specific tooling, make sure the logic that doesn't need a component at all is tested the ordinary way. Anything implementing `ITaskService`, any validator, any mapping code — test these with plain xUnit, exactly as you would in a non-Blazor project.

```
$ dotnet new xunit -o TaskBoard.Tests
$ cd TaskBoard.Tests
$ dotnet add reference ../TaskBoard/TaskBoard.csproj
```

TaskBoard.Tests/InMemoryTaskServiceTests.cs

```
using TaskBoard.Models;
using TaskBoard.Services;
using Xunit;

public class InMemoryTaskServiceTests
{
    [Fact]
    public async Task CreateAsync_AssignsIncrementingIds()
    {
        var service = new InMemoryTaskService();

        var firstId = await service.CreateAsync(new NewTaskModel { Title
            ↪ = "First" });
        var secondId = await service.CreateAsync(new NewTaskModel { Title
            ↪ = "Second" });

        Assert.NotEqual(firstId, secondId);
    }

    [Fact]
    public async Task DeleteAsync_RemovesTheTask()
    {
        var service = new InMemoryTaskService();
        var id = await service.CreateAsync(new NewTaskModel { Title =
            ↪ "Temporary" });

        await service.DeleteAsync(id);
    }
}
```

```

        var result = await service.GetByIdAsync(id);

        Assert.Null(result);
    }
}

```

Nothing here is Blazor-specific — this is why designing services behind interfaces (Chapter 9) pays off immediately: they’re trivially testable in complete isolation from any rendering concern.

18.4 Component testing with bUnit

bUnit renders a real Blazor component into an in-memory test host — no browser, no server, just the actual Razor-compiled component logic running against a fake renderer, which makes these tests fast (typically single-digit milliseconds each) while still exercising real component behavior: real lifecycle methods, real event dispatch, real markup generation.

```
$ dotnet add package bunit
```

TaskBoard.Tests/TaskCardTests.cs

```

using Bunit;
using TaskBoard.Components;
using TaskBoard.Models;
using Xunit;

public class TaskCardTests : TestContext
{
    [Fact]
    public void RendersTaskTitle()
    {
        var task = new TaskItem { Id = 1, Title = "Write chapter 16" };

        var component = RenderComponent<TaskCard>(parameters =>
            parameters
                .Add(p => p.Task, task));
    }
}

```

```
        Assert.Contains("Write chapter 16", component.Markup);
    }

    [Fact]
    public void ClickingCheckbox_RaisesToggleCallback()
    {
        var task = new TaskItem { Id = 1, Title = "Test task", IsDone =
            ↪ false };
        var toggled = false;

        var component = RenderComponent<TaskCard>(parameters =>
            ↪ parameters
                .Add(p => p.Task, task)
                .Add(p => p.OnToggleRequested, () => toggled = true));

        component.Find("input[type=checkbox]").Change(true);

        Assert.True(toggled);
    }

    [Fact]
    public void DoneTask_HasStrikethroughStyle()
    {
        var task = new TaskItem { Id = 1, Title = "Done task", IsDone =
            ↪ true };

        var component = RenderComponent<TaskCard>(parameters =>
            ↪ parameters
                .Add(p => p.Task, task));

        Assert.Contains("line-through", component.Markup);
    }
}
```

TestContext (from bUnit) sets up the fake renderer; RenderComponent<T> renders your component with the given parameters and returns a wrapper you can query with CSS-selector-style Find() calls, or assert against directly using component.Markup for the raw generated HTML.

18.4.1 Testing components that use dependency injection

Register fakes or real lightweight implementations directly into bUnit's own service collection, exactly as you would in `Program.cs`:

```
public class TaskListTests : TestContext
{
    [Fact]
    public void ShowsEmptyState_WhenNoTasksExist()
    {
        Services.AddSingleton<ITaskService>(new InMemoryTaskService());

        var component = RenderComponent<TaskList>();

        Assert.Contains("No tasks yet", component.Markup);
    }

    [Fact]
    public void ShowsErrorState_WhenServiceThrows()
    {
        var failingService = new Mock<ITaskService>();
        failingService.Setup(s => s.GetAllAsync())
            .ThrowsAsync(new HttpRequestException("network down"));
        Services.AddSingleton(failingService.Object);

        var component = RenderComponent<TaskList>();

        Assert.Contains("Could not load tasks", component.Markup);
    }
}
```

The second test uses Moq (`dotnet add package Moq`) to fake `ITaskService` entirely, forcing the exact failure path that's genuinely hard to trigger against a real dependency — this is precisely the payoff promised back in Chapter 9 for coding against an interface rather than a concrete class.

18.4.2 Simulating navigation and route parameters

```
[Fact]
public void LoadsCorrectTask_ForGivenRouteParameter()
```

```
{
    Services.AddSingleton<ITaskService>(new InMemoryTaskService());

    var component = RenderComponent<TaskDetail>(parameters => parameters
        .Add(p => p.TaskId, 42));

    Assert.Contains("Task #42", component.Markup);
}
```

18.5 End-to-end testing with Playwright

bUnit proves a component behaves correctly in isolation; it does **not** prove the whole app works — real routing, a real running server, real network calls, real browser rendering quirks. Playwright drives an actual, automated browser (Chromium, Firefox, or WebKit) against your app’s real address, which is the closest thing to “does this work the way an actual user would experience it.”

```
$ dotnet new nunit -o TaskBoard.E2ETests
$ cd TaskBoard.E2ETests
$ dotnet add package Microsoft.Playwright.NUnit
$ dotnet build
$ pwsh bin/Debug/net8.0/playwright.ps1 install
```

TaskBoard.E2ETests/TaskFlowTests.cs

```
using Microsoft.Playwright;
using Microsoft.Playwright.NUnit;
using NUnit.Framework;

[TestFixture]
public class TaskFlowTests : PageTest
{
    [Test]
    public async Task UserCanCreateAndCompleteATask()
    {
        await Page.GotoAsync("https://localhost:5001/tasks");
    }
}
```

```

    await Page.FillAsync("input[placeholder='New task title']", "Buy
↪ milk");
    await Page.ClickAsync("button:has-text('Add')");

    await Expect(Page.Locator("li", new() { HasText = "Buy milk"
↪ })).ToBeVisibleAsync();

    await Page.CheckAsync("li:has-text('Buy milk')
↪ input[type=checkbox]");

    await Expect(Page.Locator("li:has-text('Buy milk')"))
        .ToHaveCSSAsync("text-decoration-line", "line-through");
  }
}

```

This test opens a real browser, types into a real input, clicks a real button, and asserts on the resulting real DOM state — genuinely end-to-end, at the cost of being slower than a bUnit test (typically hundreds of milliseconds to a few seconds, rather than single-digit milliseconds) and requiring a running instance of the application (commonly started automatically as part of the test run in CI, which Chapter 20 shows).

18.6 Where each kind of test earns its cost

Test type	Speed	What it actually proves	Run how often
Plain xUnit (services, logic)	Milliseconds	Business logic is correct	Every save, every commit
bUnit (components)	Milliseconds	A component renders and reacts correctly in isolation	Every commit
Playwright (E2E)	Seconds	The whole system genuinely works for a real user	Every PR, before merge; not necessarily on every keystroke

18.7 Try it yourself

1. Write a bUnit test proving the “Save” button in `NewTask.razor` is disabled (or the form fails validation) when the title field is empty.
2. Write a bUnit test using a mocked `ITaskService` to prove that a “Retry” button, shown on load failure, actually calls `GetAllAsync` again when clicked.
3. Write a Playwright test that creates a task, navigates to its detail page via the rendered link, and asserts the correct title appears — exercising routing (Chapter 7) end-to-end rather than in isolation.

You can now prove your application behaves correctly, automatically, without a human clicking through it by hand every time. The final chapter of Part III adds one more capability real applications often need: pushing updates to the browser the moment something changes elsewhere, without the user refreshing.

Real-Time Features with SignalR

19.1 Why Blazor and SignalR are already old friends

If you've been paying close attention since Chapter 13, this chapter's subject shouldn't be a total surprise: **Blazor Server's entire communication model already runs over a SignalR connection.** Every click, every re-render patch, every JS interop call in Server mode travels over the same persistent connection SignalR provides for any other real-time feature. This chapter is about using SignalR **explicitly**, for your own application messages — live notifications, a shared presence indicator, a collaborative “someone else is editing this task” warning — on top of (or independent from) whatever render mode you've chosen.

19.2 Setting up a SignalR hub

A **hub** is a server-side class exposing methods clients can call, and providing a way to push messages back out to connected clients.

Hubs/TaskHub.cs

```
using Microsoft.AspNetCore.SignalR;

namespace TaskBoard.Hubs;

public class TaskHub : Hub
{
    public async Task NotifyTaskCreated(string title)
    {
        // Broadcast to every other connected client
        await Clients.Others.SendAsync("TaskCreated", title);
    }
}
```

Register it in Program.cs:

```
builder.Services.AddSignalR();

// ... after app.Build():
app.MapHub<TaskHub>("/hubs/tasks");
```

19.3 Connecting from a Blazor component

Both Blazor Server and Blazor WebAssembly connect to a hub the same way, using the `Microsoft.AspNetCore.SignalR.Client` package — this is a genuinely render-mode-agnostic technique, unlike some of the topics in earlier chapters.

```
$ dotnet add package Microsoft.AspNetCore.SignalR.Client
```

Components/Pages/TaskList.razor

```
@page "/tasks"
@inject NavigationManager Nav
@implements IAsyncDisposable

<h3>My Tasks</h3>
@if (liveNotification is not null)
{
    <div class="alert alert-info">@liveNotification</div>
}

@code {
    private HubConnection? hubConnection;
    private string? liveNotification;

    protected override async Task OnInitializedAsync()
    {
        hubConnection = new HubConnectionBuilder()
            .WithUrl(Nav.ToAbsoluteUri("/hubs/tasks"))
            .WithAutomaticReconnect()
            .Build();
    }
}
```

```

        hubConnection.On<string>("TaskCreated", (title) =>
        {
            liveNotification = $"Someone just added: {title}";
            InvokeAsync(StateHasChanged);
        });

        await hubConnection.StartAsync();
    }

    private async Task AddTask()
    {
        var id = await TaskService.CreateAsync(new NewTaskModel { Title =
↪ newTaskTitle });
        if (hubConnection is not null)
        {
            await hubConnection.SendAsync("NotifyTaskCreated",
↪ newTaskTitle);
        }
        newTaskTitle = "";
    }

    public async ValueTask DisposeAsync()
    {
        if (hubConnection is not null)
        {
            await hubConnection.DisposeAsync();
        }
    }
}

```

Three details here connect directly back to earlier chapters, worth calling out explicitly:

- **hubConnection.On<string>("TaskCreated", callback)** registers a handler for messages arriving from outside any Blazor-tracked event — exactly the “Blazor isn’t watching this” scenario from Chapter 6, which is why the callback explicitly calls `InvokeAsync(StateHasChanged)` rather than relying on automatic re-rendering.
- **WithAutomaticReconnect()** handles transient network blips gracefully — without it, a dropped Wi-Fi connection would permanently disconnect the hub connection until a full page reload.

- **DisposeAsync** tears down the connection when the component is removed, exactly the cleanup discipline from Chapter 6’s `IAsyncDisposable` coverage — a leaked hub connection here is a leaked server-side resource for as long as the server process runs.

19.4 Broadcasting to specific groups

Sending every message to every connected client doesn’t scale past a trivial demo. SignalR groups let you scope broadcasts — for a TaskBoard app, a natural grouping is “everyone currently viewing this specific task”:

```
public class TaskHub : Hub
{
    public async Task JoinTaskGroup(int taskId)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId,
↪ $"task-{taskId}");
    }

    public async Task LeaveTaskGroup(int taskId)
    {
        await Groups.RemoveFromGroupAsync(Context.ConnectionId,
↪ $"task-{taskId}");
    }

    public async Task NotifyEditing(int taskId, string userName)
    {
        await Clients.OthersInGroup($"task-{taskId}")
            .SendAsync("SomeoneIsEditing", userName);
    }
}
```

A component viewing `TaskDetail` for task 42 joins “task-42” in `OnInitializedAsync`, leaves it in `DisposeAsync`, and now only clients actually looking at that specific task receive “someone is editing” notifications — a small change that matters enormously once you have more than a handful of concurrent users.

19.5 Pushing updates from outside any hub method

Sometimes the event that should trigger a broadcast doesn't originate from a client calling a hub method at all — a scheduled background job detecting an overdue task, for instance. Inject `IHubContext<TaskHub>` into any other service to push messages from anywhere in your application:

```
public class OverdueTaskChecker : BackgroundService
{
    private readonly IHubContext<TaskHub> hubContext;
    private readonly IServiceScopeFactory scopeFactory;

    public OverdueTaskChecker(IHubContext<TaskHub> hubContext,
        ↪ IServiceScopeFactory scopeFactory)
    {
        this.hubContext = hubContext;
        this.scopeFactory = scopeFactory;
    }

    protected override async Task ExecuteAsync(CancellationToken
        ↪ stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            using var scope = scopeFactory.CreateScope();
            var taskService =
                ↪ scope.ServiceProvider.GetRequiredService<ITaskService>();
            var overdue = (await taskService.GetAllAsync())
                .Where(t => t.DueDate < DateTime.Today && !t.IsDone);

            foreach (var task in overdue)
            {
                await hubContext.Clients.All.SendAsync("TaskOverdue",
                ↪ task.Title, stoppingToken);
            }

            await Task.Delay(TimeSpan.FromMinutes(15), stoppingToken);
        }
    }
}
```

```
builder.Services.AddHostedService<OverdueTaskChecker>();
```

Notice the `IServiceScopeFactory` dance: `OverdueTaskChecker` is registered as a `Singleton` (all `BackgroundServices` are), but `ITaskService` is `Scoped` — exactly the lifetime mismatch flagged as a hazard in Chapter 9. Creating an explicit scope per iteration, rather than injecting `ITaskService` directly into the background service’s constructor, is the correct way to bridge a singleton-lifetime component to a scoped dependency.

19.6 Try it yourself

1. Extend `TaskHub` with a `JoinTaskGroup/LeaveTaskGroup` pair and wire `TaskDetail.razor` to join on `OnInitializedAsync` and leave on `DisposeAsync`.
2. Add a small “3 people viewing this task” indicator, incrementing and decrementing a per-group counter as clients join and leave (a `ConcurrentDictionary<int, int>` inside the hub, keyed by task id, is sufficient for a first pass).
3. Build the `OverdueTaskChecker` background service above and confirm, using two open browser tabs, that both receive the “task overdue” notification without either tab reloading.

Part III is complete — your application is secured, reasonably fast, automatically tested, and can push live updates. Part IV shifts focus entirely: not new Blazor APIs, but the practices — architecture, code review, CI/CD, deployment — that separate a project one person can run locally from one a team can ship and maintain.

Accessibility and Internationalization

20.1 Why this belongs in a book about building, not just a checklist at the end

Accessibility (often abbreviated a11y — “a”, eleven letters, “y”) and internationalization (i18n) are usually treated as afterthoughts, bolted on once an application is “done.” Both are dramatically cheaper to build in from the start than to retrofit, and both are, ultimately, about the same underlying idea as everything else in this book: your markup and your C# should express *what something means*, not just *what it looks like* — a button is a `<button>`, not a styled `<div>` with a click handler; user-facing text is a resource that can be swapped, not a string literal welded into your markup.

20.2 Semantic HTML first

The single highest-leverage accessibility practice costs nothing extra to write: use the HTML element whose *meaning* matches your intent, and let assistive technology (screen readers, in particular) do the rest automatically.

```
@* Avoid: a div pretending to be a button *@  
<div class="btn" @onclick="Save">Save</div>  
  
@* Prefer: an actual button *@  
<button class="btn" @onclick="Save">Save</button>
```

A real `<button>` is automatically keyboard-focusable, activates on both Enter and Space, and is announced as “button” by a screen reader — all free, all lost the moment

you reach for a styled `<div>` with a click handler instead. The `Button` component built in the previous chapter already does this correctly, which is itself an argument for building a shared component library: get the accessible foundation right once, in one place, rather than re-deciding it (and potentially re-getting it wrong) on every page.

20.3 Labeling form inputs correctly

Every `<input>` needs an associated, programmatically-connected label — not just visually adjacent text, which a screen reader has no way to associate with the field.

```
<label for="task-title">Title</label>
<InputText id="task-title" @bind-Value="model.Title" />
```

For inputs where a visible label doesn't fit the design (an icon-only search box, for instance), use `aria-label` rather than omitting labeling entirely:

```
<input type="search" aria-label="Search tasks" @bind="searchQuery" />
```

20.4 Managing focus after dynamic UI changes

Blazor's re-rendering model makes it easy to swap large chunks of the page — showing a modal, replacing a list with a “no results” message — without a full page navigation. Sighted mouse users notice the visual change immediately; a screen reader user, or anyone navigating by keyboard, needs focus **explicitly moved** to the new content, or they're left exactly where they were, unaware anything changed.

```
@if (showModal)
{
    <div class="modal" role="dialog" aria-modal="true"
    ↪ aria-labelledby="modal-title" @ref="modalRef">
        <h2 id="modal-title">Confirm delete</h2>
        <p>Delete "@taskToDelete?.Title"? This cannot be undone.</p>
```



```

        <Button Variant="ButtonVariant.Danger"
↪   OnClick="ConfirmDelete">Delete</Button>
        <Button Variant="ButtonVariant.Ghost"
↪   OnClick="CloseModal">Cancel</Button>
    </div>
}

@code {
    private ElementReference modalRef;
    private bool wasModalOpen;

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (showModal && !wasModalOpen)
        {
            await modalRef.FocusAsync();
        }
        wasModalOpen = showModal;
    }
}

```

`role="dialog"` and `aria-modal="true"` tell assistive technology this element is a modal dialog, taking over interaction until dismissed; `aria-labelledby` points a screen reader at the element that names the dialog. Moving actual keyboard focus into it, in `OnAfterRenderAsync` — the correct lifecycle hook for exactly this kind of post-render DOM interaction, as established in Chapter 6 — is what makes the dialog usable without a mouse at all, not just visually present.

20.5 Announcing dynamic content with live regions

Some updates — a “task saved” confirmation, a validation error appearing after a failed submission — happen without any focus change at all, and a screen reader has no way to know to announce them unless told to watch that region of the page.

```

<div aria-live="polite" role="status">
    @statusMessage
</div>

```

`aria-live="polite"` tells assistive technology “when this element’s content changes, announce it, but wait for the user to finish whatever they’re currently doing first” — the right choice for most status messages. Reserve `aria-live="assertive"` (interrupts immediately) for genuinely urgent information, like a session-expiry warning; overusing assertive for routine updates is a common way to make an application actively unpleasant to use with a screen reader.

20.6 Testing accessibility isn’t optional-extra work

Two cheap habits catch the majority of real issues before they ship:

1. **Unplug your mouse for ten minutes** and try to complete your application’s core flows using only Tab, Shift+Tab, Enter, and arrow keys. Anything you can’t reach or activate this way is inaccessible to anyone who can’t use a mouse, for any reason.
2. **Automated scanning as a CI gate.** Tools like axe-core (via a Playwright integration) catch a meaningful subset of issues — missing labels, insufficient color contrast, missing alt text — automatically, on every pull request, the same way the tests in Chapter 16 catch functional regressions.

```
[Test]
public async Task TaskListPage_HasNoAxeViolations()
{
    await Page.GotoAsync("https://localhost:5001/tasks");
    var results = await Page.RunAxeAsync();
    Assert.That(results.Violations, Is.Empty);
}
```

Automated scanning finds perhaps a third of real accessibility issues — it cannot tell you whether your focus management “feels” right, or whether your `aria-label` text is actually clear. Treat it as a floor, not a substitute for occasionally testing with a real screen reader (NVDA on Windows and VoiceOver on macOS are both free) and genuine keyboard-only navigation.

20.7 Internationalization: preparing for more than one language

Even if TaskBoard only ships in English today, structuring text as **resources** rather than inline literals costs little now and saves a painful, error-prone retrofit later.

```
$ dotnet add package Microsoft.Extensions.Localization
```

Resources/TaskList.resx (default, English)

Key: TaskList_Heading	Value: My Tasks
Key: TaskList_AddButton	Value: Add
Key: TaskList_EmptyState	Value: No tasks yet. Create your first one!

Resources/TaskList.ro.resx (Romanian)

Key: TaskList_Heading	Value: Sarcinile mele
Key: TaskList_AddButton	Value: Adaugă
Key: TaskList_EmptyState	Value: Nicio sarcină încă. Creează prima!

```
@inject IStringLocalizer<TaskList> Localizer

<h3>@Localizer["TaskList_Heading"]</h3>
<button onclick="AddTask">@Localizer["TaskList_AddButton"]</button>

@if (tasks.Count == 0)
{
    <p>@Localizer["TaskList_EmptyState"]</p>
}
```

Registering supported cultures and letting ASP.NET Core pick the right .resx file based on the request's Accept-Language header (or a user preference you store) is configured once, centrally:

```
var supportedCultures = new[] { "en", "ro" };
var localizationOptions = new RequestLocalizationOptions()
    .SetDefaultCulture(supportedCultures[0])
    .AddSupportedCultures(supportedCultures)
    .AddSupportedUICultures(supportedCultures);

app.UseRequestLocalization(localizationOptions);
```

Beyond swapping strings, watch for two things that trip up applications adding a second language late: text expansion (German and Finnish translations routinely run 30–40% longer than English — a button sized exactly for “Add” will visually break for “Hinzufügen”), and date/number formatting, which `DateTime` and numeric formatting already handle correctly *if* you consistently use culture-aware formatting (`ToString("d", CultureInfo.CurrentCulture)`) rather than hardcoded format strings assuming a US date order.

20.8 Try it yourself

1. Unplug your mouse and attempt to create, edit, and delete a task in your TaskBoard app using only the keyboard — note every place you get stuck, and fix the highest-impact one.
2. Add `aria-live="polite"` status announcements to the “Saved!” and error-state messages built back in Chapters 8 and 10.
3. Extract the visible strings from `TaskList.razor` into a `.resx` resource file, add a second language of your choice, and confirm switching your browser’s language preference actually changes the rendered text.

An application that’s fast, tested, and secure but unusable without a mouse, or unreadable to anyone who doesn’t speak its one hardcoded language, has only solved part of the problem. With this chapter, the technical core of the book is complete — Part IV’s remaining chapters return to the practices that let a team ship and maintain all of this together, starting with how the codebase itself should be structured.

PART IV

Industry Practice

Architecture and Project Structure

21.1 Why a single project stops being enough

Every example so far lived in one project, TaskBoard, and that was the right call for learning — fewer moving parts means fewer places to get lost. A real team codebase, though, almost always splits into multiple projects, for reasons that have nothing to do with Blazor specifically and everything to do with how teams share code, enforce boundaries, and deploy independently.

21.2 A typical solution layout

```
TaskBoard.sln
├─ src/
│   ├─ TaskBoard.Shared/      # DTOs, enums, validation models –
│   │   ↪ referenced by everything
│   ├─ TaskBoard.Domain/      # Entities, business rules, interfaces
│   │   ↪ (no framework dependencies)
│   ├─ TaskBoard.Infrastructure/ # EF Core, external API clients –
│   │   ↪ implements Domain interfaces
│   ├─ TaskBoard.Api/          # ASP.NET Core Web API – the actual
│   │   ↪ backend
│   ├─ TaskBoard.Web/          # The Blazor app itself (Server or
│   │   ↪ WASM host)
│   │   └─ TaskBoard.Web.Client/ # If using Auto mode: the
│   │       ↪ WASM-specific client project
│   └─ tests/
│       ├─ TaskBoard.Domain.Tests/
│       ├─ TaskBoard.Web.Tests/    # bUnit component tests
│       └─ TaskBoard.E2ETests/     # Playwright tests
```

This is a variant of **Clean Architecture** (also called Onion Architecture or Hexagonal Architecture in different communities) — the specific names for each project

matter less than the *dependency direction* they enforce, which is the actual point of the pattern.

21.3 The dependency rule

The single rule that makes this layout worth the extra ceremony: **dependencies point inward, toward Domain, never outward.**

```
TaskBoard.Web —depends on—> TaskBoard.Infrastructure —depends on—>
↳ TaskBoard.Domain
TaskBoard.Api —depends on—> TaskBoard.Infrastructure —depends on—>
↳ TaskBoard.Domain
```

TaskBoard.Domain contains your entities (TaskItem, TaskPriority) and the *interfaces* your business logic needs (ITaskRepository), but has **no reference to Entity Framework, no reference to ASP.NET Core, no reference to Blazor at all**. It's plain C#, meaning it's trivially unit-testable and, more importantly, it can't accidentally develop a dependency on how data happens to be stored or how the UI happens to be built.

TaskBoard.Infrastructure implements those interfaces using real technology:

```
namespace TaskBoard.Infrastructure.Persistence;

public class EfTaskRepository : ITaskRepository
{
    private readonly ApplicationDbContext db;

    public EfTaskRepository(ApplicationDbContext db) => this.db = db;

    public async Task<List<TaskItem>> GetAllAsync() =>
        await db.Tasks.ToListAsync();

    // ...
}
```

TaskBoard.Web (your Blazor project) depends on the *interface* (ITaskRepository, or the ITaskService abstraction wrapping it, from Chapter 9) and never on EfTaskRepository

directly — the actual implementation is wired up once, in `Program.cs`, exactly like every DI registration you’ve already written throughout this book.

```
// Program.cs, in TaskBoard.Web or TaskBoard.Api
builder.Services.AddScoped<ITaskRepository, EfTaskRepository>();
```

21.4 Why this is worth the overhead

For a solo hobby project, splitting into six projects is genuine overkill — one project, as used throughout this book, is the right call. The payoff shows up specifically as a team and codebase grow:

- **Independent testability.** `TaskBoard.Domain` tests need no database, no web server, no Blazor test host — they run in milliseconds and can’t be flaky because of infrastructure.
- **Swappable infrastructure.** Moving from SQL Server to PostgreSQL, or from a REST API to gRPC, touches only `Infrastructure` — `Domain` and the UI are unaffected, because they only ever depended on an interface.
- **Enforced boundaries.** A junior developer physically *cannot* accidentally call `DbContext` directly from a Razor component, because `TaskBoard.Web` doesn’t have a project reference to Entity Framework at all — the compiler enforces the architecture, not just a wiki page nobody reads.
- **Shared DTOs without shared implementation.** `TaskBoard.Shared` (referenced by both `Web` and `Api`) holds request/response models both the Blazor client and the API agree on, without either depending on the other’s internals.

21.5 Where Blazor-specific code fits into this

A subtlety worth calling out: your Razor **components** — the actual `.razor` files, with their markup and UI logic — are legitimately part of the “outer” layer, alongside API controllers, because they’re both, fundamentally, presentation concerns talking to the same application core. It’s entirely normal and good practice for `TaskBoard.Web` to contain both your Blazor components *and* a thin layer of “application services” that orchestrate domain logic for the UI’s benefit — the important

boundary is that this application-services layer still depends on Domain's interfaces, not on Infrastructure's concrete classes.

21.6 Feature folders vs. technical folders

Within `TaskBoard.Web`, you have a further choice: organize by **technical type** (`Components/Pages/`, `Components/Shared/`, `Services/` — what this book has done throughout, for clarity while learning) or by **feature** (`Features/Tasks/TaskList.razor`, `Features/Tasks/TaskCard.razor`, `Features/Tasks/ITaskService.cs`, all in one folder). As a codebase grows past roughly 30–40 components, feature folders tend to age better: finding everything related to “tasks” means opening one folder, rather than hunting across `Pages`, `Shared`, `Services`, and `Models` for the pieces of a single feature. Neither is objectively correct — pick one convention as a team and apply it consistently; the cost of an inconsistent mix of both styles is worse than either style alone.

21.7 A note on `TaskBoard.Shared` and Blazor Auto mode

If you're using `InteractiveAuto` (Chapter 13), you'll typically have **two** Blazor-facing projects: the main `TaskBoard.Web` (which runs on the server, including for prerendering and the Server-mode fallback) and a separate `TaskBoard.Web.Client` project (compiled to `WebAssembly`, containing anything that must run client-side). Components and services shared between the two — which, for Auto mode, is usually most of your UI, per Chapter 13's constraint that Auto-mode code must run correctly under both models — live in a project both reference, commonly folded into `TaskBoard.Web.Client` itself with `TaskBoard.Web` referencing it, rather than a fourth separate project. `dotnet new blazor -au Individual --interactivity` Auto scaffolds exactly this shape for you; inspect it once as a working reference before building your own.

21.8 Try it yourself

1. Split your `TaskBoard` project from earlier chapters into `TaskBoard.Domain` (containing `TaskItem`, `TaskPriority`, `ITaskService`) and `TaskBoard.Web` (contain-

- ing everything else), and fix the resulting compile errors by adding a project reference.
2. Add `TaskBoard.Infrastructure` containing `HttpTaskService` (moved out of `Web`), and confirm `Web` only ever references it through DI registration in `Program.cs`, never directly by type name in a component.
 3. Write one test in a new `TaskBoard.Domain.Tests` project that references *only* `TaskBoard.Domain` — no Blazor, no EF Core, no ASP.NET Core packages at all — to prove the boundary is real, not just conventional.

With a codebase shaped for more than one contributor, the next chapter covers the practice that actually makes multiple contributors work well together: a sane Git workflow and a code review discipline suited to a Blazor-shaped codebase.

Git Workflow and Code Review

22.1 Branching strategy

Two strategies cover the overwhelming majority of professional teams, and the right choice depends far more on your deployment cadence than on Blazor specifics.

Trunk-based development — everyone commits small, frequent changes directly to `main` (behind feature flags for anything incomplete), with short-lived branches (hours to a couple of days) merged quickly. This is the dominant approach at companies deploying multiple times a day, and pairs naturally with the CI/CD pipeline built in the next chapter, since `main` is always releasable.

```
$ git checkout -b feature/task-priority-filter
# ... small, focused commits ...
$ git push origin feature/task-priority-filter
# open a PR, get it reviewed and merged same-day
```

GitFlow (or a simplified variant) — longer-lived `develop` and `release` branches, feature branches merged into `develop`, periodic release branches cut from it. This suits teams shipping on a fixed schedule (a mobile app tied to app-store review cycles, an enterprise product with quarterly releases) more than a continuously-deployed web app, but you'll encounter it in plenty of real Blazor codebases, especially older or more regulated ones.

For a new project with no strong constraint pulling you toward GitFlow's ceremony, default to trunk-based development with feature branches and short PR lifetimes — it's simpler, and it matches the CI/CD pipeline you'll build in Chapter 20.

22.2 Commit message conventions

Conventional Commits is a lightweight, widely-adopted format worth adopting even on a small team, because it makes the commit history genuinely useful (searchable, and mechanically parseable for generating changelogs and deciding version bumps):

```
feat(tasks): add priority filter to task list
fix(auth): correct redirect loop on expired token
refactor(services): extract ITaskRepository from ITaskService
test(taskcard): add bUnit coverage for toggle callback
docs(readme): document local HTTPS certificate setup
chore(deps): bump bunit to 1.28.9
```

The `type(scope): summary` shape lets tooling (and human reviewers, skimming a long history) instantly separate “this changed behavior” (`feat`, `fix`) from “this didn’t” (`refactor`, `test`, `docs`, `chore`) — a distinction that matters enormously when deciding what belongs in release notes and what doesn’t.

22.3 What a good pull request looks like

A PR description for a Blazor change should answer, briefly, four questions a reviewer otherwise has to reconstruct from the diff alone:

1. **What** changed, in plain language (not just “see title”).
2. **Why** — what problem does this solve, or what request/ticket does it close.
3. **How to verify it** — which page to visit, which flow to click through, or which new test proves it.
4. **Anything the reviewer should pay special attention to** — a tricky render-mode interaction, a new dependency, a migration that needs to run.

```
## What
Adds a priority filter dropdown to the task list page.

## Why
Closes #142 – users with 50+ tasks had no way to narrow the view.
```

```
## How to verify
1. `dotnet run`, navigate to /tasks
2. Create tasks with different priorities
3. Confirm the dropdown filters correctly, and that the filter
   persists across a page reload (uses ProtectedLocalStorage)

## Notes for reviewers
`TaskFilterState` is now Scoped rather than Singleton – please double-
check I haven't broken the multi-user isolation from Chapter 9's
discussion (there's a new bUnit test for this in TaskFilterStateTests).
```

22.4 What to actually look for reviewing a Blazor PR

Beyond the review discipline that applies to any language, a few things are specifically worth a reviewer's attention in a Blazor codebase, precisely because they're easy to get subtly wrong and hard to catch just by reading the diff quickly:

- **Disposal.** Does every `OnInitialized` that subscribes to an event, starts a timer, or opens a hub connection have a matching `Dispose/DisposeAsync`? (Chapters 6, 11, 17.)
- **Service lifetimes.** Does a newly-registered service hold per-user state but get registered as `Singleton`? Does it hold genuinely shared, stateless infrastructure but get registered as `Scoped` for no reason, wasting instances? (Chapter 9.)
- **`OnInitialized` vs. `OnParametersSet`.** Does data-loading logic that depends on a route or parent-supplied parameter correctly live in `OnParametersSet`, with a guard against redundant reloads, rather than `OnInitialized`? (Chapter 6.)
- **Client-side “security.”** Does a change rely on `<AuthorizeView>` or `[Authorize]` alone to protect something, without a matching server-side check on the actual API endpoint? (Chapter 14 — this one is worth a reviewer treating as a hard blocker, not a style nit.)
- **@key on dynamic lists.** Was a new `@foreach` over a mutable, reorderable, or filterable collection given a stable `@key`? (Chapter 15.)

- **Missing tests for the failure path**, not just the happy path — does a new API call have a corresponding test (bUnit, with a mocked service) proving the loading and error states actually render, not just the success case? (Chapter 16.)

22.5 Review etiquette that keeps a team functional

Two habits do more for review culture than any checklist:

- **Distinguish blocking feedback from suggestions.** Prefix optional nitpicks clearly (Nit:, or your team’s equivalent convention) so the author can tell “this must change before merge” from “consider this, take it or leave it” without a back-and-forth conversation to clarify.
- **Review the tests, not just the code.** A PR that adds a feature with no new test coverage, or with a test that would pass even if the feature were broken (a common failure mode: a bUnit test that renders a component but asserts nothing meaningful about the specific behavior being added), should be sent back before it’s merged — the test suite is a shared team asset, and a false sense of coverage is worse than an honest gap you know about.

22.6 Try it yourself

1. Write a PR description, using the template above, for the @key-adding change from Chapter 15’s exercises.
2. Review your own TaskFilterState implementation from Chapter 11 against the reviewer checklist above — specifically the “service lifetimes” and “disposal” points — and fix anything it reveals.
3. Set up a CONTRIBUTING.md in your repository documenting your team’s chosen branching strategy and commit message convention, so it’s discoverable rather than tribal knowledge.

A good review process catches problems before they merge. The next chapter builds the automation that catches problems the moment code is pushed — before a human reviewer even opens the PR.

CI/CD for Blazor

23.1 What CI/CD actually buys you

Continuous Integration means every change is automatically built and tested the moment it's pushed, so problems are caught in minutes, not discovered days later by a teammate — or, worse, by a user in production. **Continuous Deployment** (or the more common **Continuous Delivery**, where a human still clicks “approve”) means a change that passes every check can reach production with no manual, error-prone deployment ritual. Neither concept is Blazor-specific, but a Blazor pipeline has a few genuine specifics worth getting right: it needs to build both a server component and, for WASM, a distinct client artifact; it benefits from running bUnit and Playwright as clearly separate stages; and its “publish” step produces a different shape of output depending on your hosting model, which the next chapter covers in detail.

23.2 A complete GitHub Actions pipeline

`.github/workflows/ci.yml`

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
```

```
- name: Checkout
  uses: actions/checkout@v4

- name: Setup .NET
  uses: actions/setup-dotnet@v4
  with:
    dotnet-version: '8.0.x'

- name: Restore dependencies
  run: dotnet restore

- name: Build
  run: dotnet build --no-restore --configuration Release

- name: Run unit and component tests
  run: dotnet test --no-build --configuration Release
      --filter "FullyQualifiedName!~E2ETests"
      --logger "trx;LogFileName=test-results.trx"
      --collect:"XPlat Code Coverage"

- name: Publish test results
  uses: dorny/test-reporter@v1
  if: always()
  with:
    name: Test Results
    path: '**/test-results.trx'
    reporter: dotnet-trx

e2e-tests:
  runs-on: ubuntu-latest
  needs: build-and-test

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Setup .NET
      uses: actions/setup-dotnet@v4
      with:
        dotnet-version: '8.0.x'

    - name: Build the app
      run: dotnet build --configuration Release
```

```

- name: Install Playwright browsers
  run: |
    dotnet build tests/TaskBoard.E2ETests
    pwsh tests/TaskBoard.E2ETests/bin/Debug/net8.0/playwright.ps1
↪ install --with-deps

- name: Start the app in the background
  run: |
    dotnet run --project src/TaskBoard.Web --configuration Release
↪ &
    sleep 10

- name: Run Playwright tests
  run: dotnet test tests/TaskBoard.E2ETests --configuration Release

```

A few decisions in this file are worth explaining rather than copying blindly:

- **Two separate jobs**, not one. Unit and component tests are fast and should fail the build within a minute or two; Playwright tests are slow (spinning up a real app, real browsers) and gate merging without slowing down every single push. `needs: build-and-test` makes the E2E job wait for the first to succeed, avoiding wasted CI minutes running slow browser tests against code that doesn't even compile.
- **--filter "FullyQualifiedName!~E2ETests"** on the fast job explicitly excludes anything in an E2E test project, keeping the two test tiers cleanly separated even if `dotnet test` is pointed at the whole solution.
- **Code coverage collection** (`--collect:"XPlat Code Coverage"`) produces a `.cobertura.xml` report per test project; wiring this into a coverage-reporting service (Codecov, Coveralls) is a natural next step once the basic pipeline is solid, and is a good early warning that a PR reduced coverage on a critical path.

23.3 Adding static analysis and formatting checks

Catching style and obvious-bug issues before a human reviewer spends time on them is cheap to add and saves real review time:

```
lint:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with:
        dotnet-version: '8.0.x'
    - name: Check formatting
      run: dotnet format --verify-no-changes --verbosity diagnostic
```

`dotnet format --verify-no-changes` fails the build if any file doesn't match your `.editorconfig` rules, rather than silently reformatting — exactly what you want in CI, where you want a clear failure, not a surprise auto-commit. Pair this with a pre-commit hook (or your editor's format-on-save) locally so developers see the same failures before pushing, not just after.

23.4 Adding a Blazor-specific quality gate: bundle size

For a Blazor WebAssembly app, download size directly affects real users, in a way that's easy to regress silently (one new NuGet package, one un-trimmed dependency) without anyone noticing until a user complains. A simple guard:

```
- name: Publish WASM app
  run: dotnet publish src/TaskBoard.Web -c Release -o
↔ ./publish-output

- name: Check WASM bundle size
  run: |
    SIZE=$(du -sb ./publish-output/wwwroot/_framework | cut -f1)
    echo "Framework payload size: $SIZE bytes"
    if [ "$SIZE" -gt 15000000 ]; then
      echo "::error::WASM framework payload exceeds 15MB budget"
      exit 1
    fi
```

The specific threshold is a placeholder — set your own budget based on a real measurement of your app’s current size plus reasonable headroom, and treat any PR that blows past it as worth a deliberate conversation, not a silent merge.

23.5 Continuous deployment: shipping automatically on merge

Extending the pipeline to deploy after tests pass, gated to only run on main (never on a PR from a fork, and never before tests succeed):

```
deploy:
  runs-on: ubuntu-latest
  needs: [build-and-test, e2e-tests]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'

  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with:
        dotnet-version: '8.0.x'

    - name: Publish
      run: dotnet publish src/TaskBoard.Web -c Release -o
↪ ./publish-output

    - name: Deploy to Azure App Service
      uses: azure/webapps-deploy@v3
      with:
        app-name: taskboard-prod
        publish-profile: ${ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }
        package: ./publish-output
```

`secrets.AZURE_WEBAPP_PUBLISH_PROFILE` is stored in the repository’s GitHub Actions secrets, never committed to source control — the general rule (not Blazor-specific, but worth stating plainly) is that **no credential, connection string, or API key should ever appear in a workflow file or a source file**, only referenced through the platform’s secret storage.

23.6 Environment-specific configuration

A pipeline that deploys to more than one environment (staging, then production) needs configuration that changes per environment without changing code. ASP.NET Core’s layered configuration handles this cleanly, and Blazor Server/hosted apps use it identically to any other ASP.NET Core app:

```
appsettings.json           # defaults, safe to commit
appsettings.Development.json # local overrides, safe to commit
appsettings.Staging.json   # staging-specific, safe to commit (no
↪ secrets)
appsettings.Production.json # production-specific, safe to commit (no
↪ secrets)
```

Actual secrets (connection strings, API keys) are supplied at deploy time via environment variables or the hosting platform’s secret store (Azure App Service’s “Application settings,” for instance), which override whatever’s in the JSON files — never hardcode a production secret in any `appsettings.*.json` file, even one that looks environment-specific.

23.7 Try it yourself

1. Add the CI workflow above to a real GitHub repository containing your TaskBoard project, push a small change, and watch it run in the Actions tab.
2. Deliberately introduce a failing test and confirm the `e2e-tests` job never even starts, because `needs: build-and-test` correctly gates it.
3. Add the `dotnet format --verify-no-changes` lint job, then deliberately misindent a file locally, push it, and confirm CI catches it.

Your pipeline can now build, test, and gate every change automatically. The final step — actually getting a built application in front of users — is the subject of the next chapter.

Deployment and Hosting

24.1 Matching hosting to render mode

Chapter 13 explained the technical differences between Server, WebAssembly, and Auto render modes. Deployment is where those differences translate into genuinely different infrastructure decisions.

- **Blazor Server** needs a live, long-running ASP.NET Core process with Web-Socket support — a traditional app service, container, or VM. It cannot be deployed as static files, because there's always server-side C# running for the app's entire interactive lifetime.
- **Blazor WebAssembly** (with no server-side API needs beyond serving static files) can be deployed as **pure static files** — no server-side process required at all, which opens up cheap, simple hosts like a CDN or static file host.
- **Blazor Auto** needs both: a running server process (for the Server-mode first visit and any prerendering) *and* the static WASM assets served alongside it.

24.2 Publishing

Every hosting target starts with the same command:

```
$ dotnet publish src/TaskBoard.Web -c Release -o ./publish-output
```

This compiles in Release configuration (optimizations on, debug symbols stripped or separated), trims WASM output if applicable, and produces a self-contained output folder ready to deploy.

24.3 Deploying to Azure App Service

Azure App Service is the most common target for Blazor Server or Auto apps needing a managed, always-on server process, and is what the CI/CD pipeline in Chapter 20 deployed to.

```
$ az webapp create --resource-group taskboard-rg \  
  --plan taskboard-plan --name taskboard-prod \  
  --runtime "DOTNETCORE:8.0"  
  
$ az webapp deploy --resource-group taskboard-rg \  
  --name taskboard-prod --src-path ./publish-output --type zip
```

Configuration that shouldn't live in source control (connection strings, API keys) is set as **Application Settings**, which App Service injects as environment variables at runtime — ASP.NET Core's configuration system picks these up automatically, overriding whatever's in `appsettings.json`, exactly as described at the end of Chapter 20:

```
$ az webapp config appsettings set --resource-group taskboard-rg \  
  --name taskboard-prod \  
  --settings ConnectionStrings__DefaultConnection="<connection-string>"
```

Note the double underscore (`__`) — this is ASP.NET Core's convention for representing a nested configuration section (`ConnectionStrings:DefaultConnection` in JSON) as a flat environment variable name, since most shells don't allow a colon in a variable name.

24.4 Deploying with Docker

Containerizing gives you a hosting-platform-agnostic artifact — the same image runs identically on Azure Container Apps, AWS, a bare VM, or a teammate's laptop, which is valuable both for consistency and for avoiding lock-in to one cloud provider's specific deployment tooling.

Dockerfile


```

# Stage 1: build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

COPY ["src/TaskBoard.Web/TaskBoard.Web.csproj", "src/TaskBoard.Web/"]
COPY ["src/TaskBoard.Domain/TaskBoard.Domain.csproj",
     ↪ "src/TaskBoard.Domain/"]
COPY ["src/TaskBoard.Infrastructure/TaskBoard.Infrastructure.csproj",
     ↪ "src/TaskBoard.Infrastructure/"]
RUN dotnet restore "src/TaskBoard.Web/TaskBoard.Web.csproj"

COPY . .
RUN dotnet publish "src/TaskBoard.Web/TaskBoard.Web.csproj" \
    -c Release -o /app/publish --no-restore

# Stage 2: run
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app/publish .

EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080

ENTRYPOINT ["dotnet", "TaskBoard.Web.dll"]

```

This is a **multi-stage build**: the first stage uses the full SDK image (large, includes the compiler) to build and publish; the second stage copies only the published output into a much smaller runtime-only image (aspnet:8.0, not sdk:8.0), which is standard practice for keeping production container images lean — there's no reason to ship an entire compiler toolchain to production just to run already-compiled code.

Copying .csproj files and restoring *before* copying the rest of the source (rather than one COPY . . up front) is a deliberate Docker layer-caching optimization: as long as your project references don't change, Docker reuses the cached restore layer on subsequent builds, even if your .razor and .cs files changed — meaningfully speeding up repeated builds in CI.

```
$ docker build -t taskboard:latest .  
$ docker run -p 8080:8080 -e ConnectionStrings__DefaultConnection="..."  
↪ taskboard:latest
```

24.5 Deploying Blazor WebAssembly as static files

If your app is pure WebAssembly with no server-side dependency beyond an already-separate API, `dotnet publish` produces a `wwwroot` folder that is a complete, self-contained static site — deployable to Azure Static Web Apps, GitHub Pages, Netlify, Cloudflare Pages, or literally any static file host with zero server-side runtime required.

```
$ dotnet publish src/TaskBoard.Web -c Release -o ./publish-output  
# The deployable artifact is: ./publish-output/wwwroot
```

Azure Static Web Apps is purpose-built for this and integrates directly with GitHub Actions:

```
- name: Deploy to Azure Static Web Apps  
  uses: Azure/static-web-apps-deploy@v1  
  with:  
    azure_static_web_apps_api_token: ${  
↪ secrets.AZURE_STATIC_WEB_APPS_TOKEN }  
    repo_token: ${ secrets.GITHUB_TOKEN }  
    action: upload  
    app_location: "publish-output/wwwroot"  
    skip_app_build: true
```

One WASM-specific configuration detail that trips people up: because WASM apps use client-side routing (Chapter 7’s Router component intercepts navigation entirely within the loaded page), a static host must be told to serve `index.html` for *any* path it doesn’t recognize as a real file — otherwise, a user directly visiting `/tasks/42` (rather than navigating there from `/`) gets a 404 from the host itself, before Blazor’s router ever gets a chance to run. Azure Static Web Apps handles this automatically; other static hosts typically need an explicit rewrite rule (commonly

a `staticwebapp.config.json` or a host-specific `_redirects` file) pointing all unmatched paths back to `index.html`.

24.6 Deploying to IIS (Windows Server environments)

Many enterprise environments, particularly ones with an existing Windows Server and IIS investment, deploy ASP.NET Core apps (Blazor Server included) to IIS rather than a cloud platform. This requires the **ASP.NET Core Hosting Bundle** installed on the server (which includes the .NET runtime and an IIS module, ANCM, that proxies requests from IIS into your app's Kestrel-based process), after which deployment is a matter of publishing to a folder IIS is configured to serve from:

```
$ dotnet publish src/TaskBoard.Web -c Release -o  
↪ C:\inetpub\wwwroot\taskboard
```

The published output includes a `web.config` file (generated automatically by `dotnet publish` for Windows-targeted publishes) that tells IIS how to launch and proxy to your app — you generally don't hand-edit this unless you need to tune specific IIS-level settings like request size limits.

24.7 A checklist before your first production deployment

- ☐ Secrets are in environment variables or a secret store, never in a committed `appsettings.*.json`.
- ☐ HTTPS is enforced (`app.UseHsts()`, `app.UseHttpsRedirection()` — present by default in the template from Chapter 2).
- ☐ Logging is configured to go somewhere durable (Application Insights, a log aggregation service) — not just the console, which disappears when a container restarts.
- ☐ For Blazor Server: `DisconnectedCircuitRetentionPeriod` and related options (Chapter 15) are tuned for your expected traffic, not left at defaults sized for a demo.

- ☐ For Blazor WebAssembly: the static host is configured to fall back to `index.html` for unmatched routes.
- ☐ The CI/CD pipeline from Chapter 20 runs the full test suite before any deploy step, and deploys are gated to main only.

24.8 Try it yourself

1. Publish the TaskBoard app and inspect the contents of the output folder — identify which files would need to go to a static host versus which require a running server process, based on your chosen render mode.
2. Build the Dockerfile above for your own project, run it locally with `docker run`, and confirm the app is reachable at `http://localhost:8080`.
3. If you have free-tier access to Azure, deploy TaskBoard to an Azure App Service instance end-to-end, from `az webapp create` through a working public URL.

You now have a complete path from an empty folder to a deployed, tested, continuously-integrated application. The final chapter pulls every concept in this book into one cohesive capstone project, and points toward where to go from here.

Capstone Project and Next Steps

25.1 Putting it all together: TaskBoard, end to end

Across this book, TaskBoard grew one capability at a time: a single component with a hardcoded list (Chapter 3), broken into reusable components with parameters (Chapter 5), driven by real forms (Chapter 8), backed by a real API through dependency injection (Chapters 9–10), sharing state across the nav bar and the list (Chapter 11), secured behind authentication (Chapter 14), covered by three tiers of automated tests (Chapter 16), pushing live updates over SignalR (Chapter 17), split into a properly layered solution (Chapter 18), and finally built, tested, and deployed automatically (Chapters 20–21).

This final chapter is a blueprint for assembling all of it into one coherent project — treat it as a checklist for a capstone you build yourself, not code to copy verbatim, since the exact shape should reflect the choices *you* made working through each chapter’s exercises.

25.1.1 The target feature set

A complete TaskBoard should let a user:

1. Register and log in (Chapter 14).
2. See a list of their own tasks, filtered by status, with pagination or virtualization for large lists (Chapters 7, 11, 15).
3. Create, edit, and delete tasks through validated forms (Chapter 8).
4. See live updates when a task is added or completed by a collaborator viewing the same board (Chapter 17).
5. Have the entire flow covered by unit tests (business logic), component tests (bUnit), and at least one true end-to-end test (Playwright) proving the critical “create a task” path works against a real running app (Chapter 16).

6. Build and deploy automatically on every merge to main, with tests gating the deploy (Chapter 20), to a real hosting target matching your chosen render mode (Chapter 21).

25.1.2 Suggested build order

If you're assembling this for the first time as a genuine capstone rather than following along chapter-by-chapter, this order minimizes rework:

1. **Solution structure first** (Chapter 18) — Domain, Infrastructure, Web, and test projects, even before writing a single component. Retrofitting architecture onto working code is always more painful than starting inside it.
2. **Domain model and a real database-backed repository** (Chapters 9, 10, 18) — get `ITaskService/ITaskRepository` and an EF Core implementation working and unit-tested before touching Blazor at all.
3. **Core components: list, card, form** (Chapters 3–8) — build the UI against your real repository from the start, rather than an in-memory placeholder, now that it exists.
4. **Routing and layout** (Chapter 7) — wire pages together into a navigable app.
5. **Authentication** (Chapter 14) — added once there's something worth protecting.
6. **Render mode decision** (Chapter 13) — pick deliberately, based on your actual expected usage pattern, not by default.
7. **Real-time and performance passes** (Chapters 15, 17) — added once the core flows are solid; premature virtualization or SignalR on a feature that doesn't need it yet is wasted complexity.
8. **Tests throughout, not bolted on at the end** — ideally each of steps 2–7 above includes its own tests as you go, per the testing pyramid from Chapter 16, rather than a separate “write tests” phase after the fact.
9. **CI/CD and deployment last** (Chapters 20–21) — once there's a stable, tested app worth automating the delivery of.

25.2 What this book didn't cover

No single book covers everything, and it's worth naming the gaps honestly rather than implying completeness:

- **Localization and internationalization** (`IStringLocalizer`, resource files) — relevant the moment your app needs more than one language.
- **Progressive Web App (PWA) features** for WebAssembly apps — offline support, install prompts, service workers.
- **Micro-frontend patterns** for very large organizations composing multiple independently-deployed Blazor apps into one experience.
- **Advanced Entity Framework Core topics** — migrations at scale, query performance tuning, multi-tenancy.
- **Accessibility (a11y)** in depth — this book’s examples use semantic HTML where natural, but a genuinely accessible application needs deliberate ARIA attribute usage, keyboard navigation testing, and screen-reader testing beyond this book’s scope.

25.3 Where to go next

- **Microsoft’s official Blazor documentation** (learn.microsoft.com/aspnet/core/blazor) is genuinely well-maintained and the correct first stop when an API surface changes in a .NET version newer than this book’s.
- **The ASP.NET Core GitHub repository** issues and discussions are where you’ll find the most current, detailed conversations about edge cases and upcoming changes, straight from the team building the framework.
- **Community component libraries** — MudBlazor, Radzen Blazor Components, and Telerik/Syncfusion’s commercial offerings — are worth evaluating before building common UI patterns (data grids, date pickers, charts) from scratch; there’s rarely a good reason to hand-roll a data grid with sorting, filtering, and paging when a mature, tested one already exists.
- **Real codebases.** Reading a mature, open-source Blazor application end to end teaches things no book can: how a real team actually organized their folders, named their services, and structured their tests, warts included.

25.4 A closing thought

Everything in this book follows from one idea introduced in Chapter 1: a Blazor component is fundamentally a C# class with a way of describing markup, participating in a tree, receiving parameters, and reacting to a lifecycle. Every feature explored

since then — forms, routing, state, real-time updates, testing, deployment — is that one idea, extended outward into a complete application and a complete team practice around it. When you hit something new that isn't in this book, that same starting question — *what class is this, what does it depend on, and where in its lifecycle am I* — will very often get you most of the way to understanding it yourself.

Good luck with what you build.

A

C# Refresher for Blazor

This appendix is a fast reference for the specific C# features Blazor code leans on most heavily — not a full language tutorial. If a code example elsewhere in this book used a construct you weren't fully sure of, it's very likely covered here.

A.1 `async/await`, in the shape Blazor uses it

Almost every service call and lifecycle override in this book returns `Task` and is marked `async`. The rule that matters most: **`async void` is only acceptable for top-level UI event handlers Blazor itself invokes** (`@onclick="async () => await SaveAsync()"` works because Blazor's event dispatch understands it); everywhere else, return `Task` so callers can `await` you and exceptions propagate correctly rather than being silently lost.

```
// Correct: awaitable, exceptions propagate to the caller
public async Task<TaskItem?> GetByIdAsync(int id)
{
    return await http.GetFromJsonAsync<TaskItem>($"api/tasks/{id}");
}

// Avoid outside of framework-invoked event handlers: async void
// swallows exceptions and can't be awaited by a caller
public async void LoadData() { ... }
```

A.2 Nullable reference types

Every project template used in this book has `<Nullable>enable</Nullable>` turned on (Chapter 2). This means `string` and `string?` are genuinely different types to the compiler: the former promises “never null,” the latter admits it might be. Blazor parameters are a common place this matters:

```
[Parameter]
public string Title { get; set; } = ""; // never null – safe default
↳ supplied

[Parameter]
public TaskItem? SelectedTask { get; set; } // legitimately absent
↳ sometimes
```

Use default! (the null-forgiving operator applied to default) only when you're certain a value will be populated before use in a way the compiler can't see — most commonly, a required [Parameter] with no sensible default value, as used throughout Chapter 5.

A.3 Records, for immutable data

A record gives you value-based equality and a concise declaration syntax, and is a natural fit for DTOs and state objects that shouldn't be mutated in place — exactly the pattern Chapter 11 used for Fluxor-style state:

```
public record TaskDto(int Id, string Title, bool IsDone);

public record TaskState(ImmutableList<TaskItem> Tasks, bool IsLoading)
{
    // 'with' produces a new record, copying every field except the ones
    ↳ named
    public TaskState MarkLoading() => this with { IsLoading = true };
}
```

A.4 Pattern matching

Used throughout this book for concise conditional logic, especially in switch expressions:

```
string StatusLabel(TaskItem task) => task switch
{
```

```
{ IsDone: true } => "Done",  
{ DueDate: var d } when d < DateTime.Today => "Overdue",  
_ => "Pending",  
};
```

A.5 LINQ, the querying vocabulary you'll use constantly

Nearly every list-filtering or list-transforming line in this book is LINQ:

```
var overdue = tasks.Where(t => t.DueDate < DateTime.Today && !t.IsDone);  
var titles = tasks.Select(t => t.Title).ToList();  
var remaining = tasks.Count(t => !t.IsDone);  
var sorted = tasks.OrderBy(t => t.DueDate).ThenByDescending(t =>  
    ↪ t.Priority);  
var grouped = tasks.GroupBy(t => t.Priority);
```

Where, Select, Count, OrderBy, and GroupBy cover the overwhelming majority of real-world data-shaping needs, and all of them are lazily evaluated — nothing actually runs until you enumerate the result (with `foreach`, `.ToList()`, or similar), which is exactly why EF Core (Appendix material aside, see Chapter 10b) can translate a LINQ query into SQL instead of running it in memory.

A.6 Generics, briefly

`List<TaskItem>`, `Task<T>`, `EventCallback<T>`, `RenderFragment<T>` — generics appear constantly in Blazor code, and the concept is simple even where the syntax looks dense: a generic type or method is parameterized by another type, decided at the call site.

```
public interface IRepository<T> where T : class  
{  
    Task<T?> GetByIdAsync(int id);  
    Task<List<T>> GetAllAsync();  
}
```

```
public class TaskRepository : IRepository<TaskItem> { /* ... */ }
```

A.7 Extension methods

An extension method lets you add a method to a type you don't own — including framework types — which is how many small utility helpers in real Blazor codebases are organized.

```
public static class TaskExtensions
{
    public static bool IsOverdue(this TaskItem task) =>
        !task.IsDone && task.DueDate < DateTime.Today;
}

// used exactly like an instance method, anywhere TaskItem is in scope:
if (task.IsOverdue()) { ... }
```

B

Razor Syntax Cheat Sheet

A single-page reference for the syntax introduced throughout this book, grouped by purpose. Chapter references point back to the full explanation.

B.1 Markup and expressions

Syntax	Meaning	Chapter
<code>@expression</code>	Output a C# expression's value	3
<code>@(expr)</code>	Output an expression, with explicit boundaries	3
<code>@{ statements }</code>	A code block; no output produced automatically	3
<code>@code { }</code>	Embed C# members (fields, methods, properties)	3
<code>@* comment *@</code>	Razor comment (not sent to the browser)	3

B.2 Directives

Directive	Purpose	Chapter
<code>@page "/route"</code>	Register this component as routable	3, 7

Directive	Purpose	Chapter
@using Namespace	Import a namespace for this file	3
@inject Type Name	Inject a DI service as a property	9
@inherits Base	Inherit from a custom base class	3
@implements IFoo	Declare an implemented interface	6
@attribute [Attr]	Apply a C# attribute to the generated class	3, 14
@layout Type	Override the default layout for this page	7
@rendermode Mode	Set the render mode for this component	13
@typeparam T	Declare a generic type parameter for a component	5

B.3 Binding and events

Syntax	Meaning	Chapter
@bind="field"	Two-way bind an input to a field/property	4
@bind:event="oninput"	Change which DOM event triggers the bind-back	4
@bind:format="..."	Format string for date/number binding	4

Syntax	Meaning	Chapter
@bind-Value="x"	Two-way bind a component's Value parameter	8
@onclick="Handler"	Bind a DOM event to a C# method or lambda	3, 4
@onclick:preventDefault	Suppress the browser's default action	4
@onclick:stopPropagation	Stop the event from bubbling to parents	4
@key="identity"	Help list diffing match by identity, not position	15
@ref="field"	Capture a reference to a DOM element or component	6, 12

B.4 Component parameters

```
[Parameter]
public string Title { get; set; } = "";

[Parameter, EditorRequired]
public TaskItem Task { get; set; } = default!;

[Parameter]
public EventCallback<string> OnRename { get; set; }

[Parameter]
public RenderFragment? ChildContent { get; set; }

[CascadingParameter]
public string Theme { get; set; } = "light";
```

B.5 Lifecycle methods, in call order

```

protected override async Task OnInitializedAsync() { }    // once
protected override async Task OnParametersSetAsync() { } // first
↳ render + every update
protected override async Task OnAfterRenderAsync(bool firstRender) { }
↳ // after DOM exists
protected override bool ShouldRender() => true;          // skip a
↳ render
public void Dispose() { }                                // teardown
↳ (sync)
public async ValueTask DisposeAsync() { }                 // teardown
↳ (async)

```

B.6 Built-in form input components

Component	HTML	Binds
InputText	<input type="text">	string
InputTextArea	<textarea>	string
InputNumber	<input type="number">	numeric types
InputDate	<input type="date">	DateTime, DateOnly
InputCheckbox	<input type="checkbox">	bool
InputSelect	<select>	any type
InputRadioGroup/InputRadio	radio buttons	any type

B.7 Common injectable framework services

```

@Inject NavigationManager Navigation
@Inject HttpClient Http
@Inject IJSRuntime JSRuntime
@Inject AuthenticationStateProvider AuthState

```



```
@inject ILogger<MyComponent> Logger  
@inject IStringLocalizer<MyComponent> Localizer
```

B.8 Route constraints

Constraint	Matches
{Id:int}	Integer
{Id:guid}	GUID
{Id:bool}	true/false
{Slug}	Any non-empty segment (no constraint)
{Id:int?}	Optional integer
{*rest}	Catch-all, remainder of the path

C

Troubleshooting Common Errors

A collection of errors and confusing behaviors beginners hit most often, with the actual cause and fix — organized so you can jump straight to a symptom.

C.1 “Cannot provide a value for property... There is no registered service of type...”

Cause: you used `@inject IFoo Foo` (or `[Inject]`) but never registered `IFoo` in `Program.cs`.

Fix: add the registration, matching the lifetime the service actually needs (Chapter 9):

```
builder.Services.AddScoped<IFoo, Foo>();
```

If you’re using Blazor Auto or WebAssembly (Chapter 13), remember services must be registered in **both** the server project and the client project — a service registered only server-side will throw this exact error the moment a component using it runs client-side.

C.2 A page shows stale data after clicking a link to “the same” page with a different ID

Cause: you put your data-loading logic in `OnInitializedAsync` instead of `OnParametersSetAsync`. Blazor reuses the component instance when navigating between two routes that resolve to the same component type, so `OnInitialized` — which runs only once, ever — never fires again.

Fix: move the loading logic to `OnParametersSetAsync`, with a guard against reloading when the relevant parameter hasn't actually changed (Chapter 6).

C.3 `InvalidOperationException: The current thread is not associated with the Dispatcher`

Cause: you called `StateHasChanged()` (or touched component state) from a callback running on a background thread — a `Timer` callback, a raw `Task.Run` continuation, a `SignalR` message handler — without marshalling back onto Blazor's synchronization context.

Fix: wrap the call: `await InvokeAsync(StateHasChanged);` (Chapter 6, Chapter 17).

C.4 Clicking a button does nothing, with no error at all

Usually one of:

- The method is `private` in a `.razor.cs` partial class but the file's namespace doesn't match what Razor generated for the `.razor` file — check both files declare the exact same namespace and class name (Chapter 3).
- You bound to the wrong event name — case matters (`@onclick`, not `@onClick`).
- The component was rendered with a stale `@key` and Blazor is diffing against an unrelated previous element — check for `@key` misuse on a list that was recently restructured (Chapter 15).

C.5 A `MissingMethodException` appears only in a published Release build, never in Debug

Cause: the WASM trimmer (Chapter 15) removed a type or method that's only referenced through reflection (common with some JSON serialization patterns, or dynamic DI resolution), so it looks unused from the trimmer's static-analysis perspective.

Fix: either restructure the code to avoid reflection-based access to that type, or explicitly tell the trimmer to keep it:

```
<ItemGroup>
  <TrimmerRootAssembly Include="MyAssemblyThatNeedsToSurviveTrimming" />
</ItemGroup>
```

C.6 Forms: `EditForm` submits, but validation messages never show up

Cause: you forgot `<DataAnnotationsValidator />` inside `<EditForm>` — without it, no attribute-based validation actually runs, even if the model has `[Required]` and similar attributes (Chapter 8).

Fix:

```
<EditForm Model="model" OnValidSubmit="HandleValidSubmit">
  <DataAnnotationsValidator />
  <ValidationSummary />
  ...
</EditForm>
```

C.7 `NavigationException` or a blank page after calling `NavigationManager.NavigateTo`

Cause (Blazor Server specifically): calling `NavigateTo` with `forceLoad: true` mid-render, or navigating from a context where a full page reload conflicts with an in-progress render, can surface confusing exceptions. Also check that the target route actually exists and matches a real `@page` directive — a typo'd route silently falls through to your `NotFound` content rather than throwing, which can look like “nothing happened” if `NotFound` renders blank.

C.8 Blazor Server: the app “freezes” after being idle in a background browser tab

Cause: most browsers throttle or suspend timers and network activity for backgrounded tabs, which can cause the SignalR connection Blazor Server depends on to appear to hang; when the tab regains focus, Blazor’s automatic reconnection (enabled by default in the template) should recover it within a few seconds.

Fix: if reconnection doesn’t happen automatically, check that the reconnection UI (Components/App.razor’s default reconnect modal, or your customized version) isn’t being hidden by custom CSS, and confirm `DisconnectedCircuitRetentionPeriod` (Chapter 15) is long enough for your users’ realistic idle patterns.

C.9 `dotnet ef migrations add` fails with “Unable to create an object of type ‘ApplicationDbContext’ ”

Cause: the EF Core CLI tools instantiate your `DbContext` outside of the normal `Program.cs` startup pipeline, so they can’t resolve constructor dependencies through your app’s DI container.

Fix: add a design-time factory:

```
public class ApplicationDbContextFactory :
    ↪ IDesignTimeDbContextFactory<ApplicationDbContext>
{
    public ApplicationDbContext CreateDbContext(string[] args)
    {
        var options = new DbContextOptionsBuilder<ApplicationDbContext>()
            .UseSqlServer("Server=(localdb)\\_
            ↪ mssqllocaldb;Database=TaskBoard;Trusted_Connection=True;"_
            ↪ )
            .Options;
        return new ApplicationDbContext(options);
    }
}
```

C.10 A bUnit test throws JSInteropException immediately on render

Cause: the component being tested calls into IJSRuntime (directly, or indirectly through a child component) during a lifecycle method bUnit’s fake renderer doesn’t provide real JS interop for.

Fix: register a stub or a “loose” JS interop fake in your test’s service collection:

```
Services.AddSingleton<IJSRuntime>(Mock.Of<IJSRuntime>());  
// or, for interop that returns specific values:  
JSInterop.Mode = JSRuntimeMode.Loose;
```

(the second line uses bUnit’s own `TestContext.JSInterop` helper, which is generally the more maintainable option — Chapter 16.)

C.11 CI passes locally with dotnet test but fails only in GitHub Actions

Almost always one of: a hardcoded localhost port assumption in an E2E test that conflicts with the CI runner’s environment (Chapter 20); a missing `--with-deps` when installing Playwright browsers on a fresh Linux runner (Chapter 16); or a test that depends on file-system state left over from a previous local run, which a clean CI checkout won’t have. Reproduce it by running the exact same commands your workflow file runs, from a clean checkout, before assuming it’s an environment quirk rather than a real gap in the test’s setup.

D

Glossary

AOT (Ahead-of-Time compilation) — Compiling C# directly to WebAssembly's native instruction format at build time, rather than shipping IL to be interpreted in the browser. Trades a larger download for faster execution. (Chapter 15)

bUnit — A testing library for rendering and interacting with Blazor components in an isolated, in-memory test host, without a real browser. (Chapter 16)

Circuit — In Blazor Server, the live, server-side representation of one connected browser tab's component tree and state, kept alive for the duration of the SignalR connection. (Chapter 13)

Component — A `.razor` file compiled into a C# class inheriting `ComponentBase`, combining markup and logic into one reusable unit. (Chapter 3)

CSS isolation — Blazor's build-time mechanism for scoping a component's `.razor.css` styles so they apply only to that component's own rendered markup. (Chapter 12)

EditContext — The object an `EditForm` uses internally to track a model's validation state and which fields have been modified. (Chapter 8)

EventCallback — A Blazor-specific delegate type used for child-to-parent communication, which correctly triggers a re-render and supports async handlers. (Chapter 5)

Fluxor — A Redux-style, unidirectional state management library for Blazor, built around actions, reducers, and a single immutable state tree. (Chapter 11)

Hub (SignalR) — A server-side class exposing methods clients can call and providing a mechanism to push messages back to connected clients. (Chapter 17)

IL (Intermediate Language) — The CPU-independent instruction format .NET compiles C# into; further compiled to native code either just-in-time (JIT) or ahead-of-time (AOT).

Interactive Auto — A .NET 8 render mode that uses Server rendering on a user's first visit and switches to WebAssembly on subsequent visits once the runtime is cached. (Chapter 13)

JS interop — The mechanism by which C# code calls JavaScript functions and JavaScript calls back into C# methods. (Chapter 12)

Prerendering — Rendering a component to static HTML on the server before its interactive runtime (Server or WASM) attaches, so the user sees content immediately. (Chapter 13)

Razor — The templating syntax mixing HTML-like markup with C#, using @ to switch between the two, that .razor and .cshtml files are written in. (Chapter 3)

Render mode — The setting controlling where a component's C# executes and how UI updates reach the browser: static, Interactive Server, Interactive WebAssembly, or Interactive Auto. (Chapter 13)

Render tree — Blazor's in-memory representation of a component's current markup, used to compute the minimal diff needed to update the real DOM. (Chapter 6, 13)

Scoped (DI lifetime) — A service lifetime that creates one instance per “scope” — in Blazor Server, effectively one instance per connected user's circuit. (Chapter 9)

SignalR — A real-time communication library over WebSockets (with automatic fallback to other transports), used both internally by Blazor Server and directly for custom real-time features. (Chapter 13, 17)

Singleton (DI lifetime) — A service lifetime creating exactly one instance for the entire application's lifetime, shared across every user. (Chapter 9)

Trimming — The build-time process of removing unreferenced code from a WebAssembly app's published output to reduce download size. (Chapter 15)

Transient (DI lifetime) — A service lifetime creating a new instance every time the service is requested. (Chapter 9)

Virtualization — Rendering only the currently-visible items of a long list (plus a small buffer), via Blazor's built-in <Virtualize> component, to avoid the cost of rendering every item's DOM node at once. (Chapter 15)

WebAssembly (WASM) — A binary instruction format supported natively by modern browsers, allowing compiled code (including a .NET runtime and your Blazor app) to run client-side at near-native speed. (Chapter 1, 13)

E

Further Practice Projects

Reading and typing along with a book gets you comfortable with syntax; building something of your own choosing, with nobody's code to lean on, is what actually cements it. Each project below is deliberately specified at a “what,” not a “how” — filling in the how, including making your own design decisions and occasionally hitting a wall you have to debug your way out of, is the point.

E.1 Project 1: Habit Tracker (reinforces Parts I–II)

A single-user application for tracking daily habits (e.g., “read 20 minutes,” “no sugar”).

Required features: - A list of habits, each with a name and a target frequency (daily, weekly, N times per week). - A way to mark a habit “done” for today, with the mark persisted (not lost on refresh). - A simple visual streak indicator — how many consecutive days a habit has been completed. - Basic validation on habit creation (name required, frequency must be positive).

Stretch goals: - A calendar-style view showing the last 30 days per habit, colored by completion. - Editing and archiving (not deleting) old habits, so history is preserved.

What this exercises: component composition and parameters (Chapter 5), forms and validation (Chapter 8), a real persistence layer (Chapter 10b), and — for the streak calculation — a good test of your LINQ comfort (Appendix A).

E.2 Project 2: Shared Grocery List (reinforces Parts II–III)

A multi-user application where a household shares one live-updating grocery list.

Required features: - Multiple named lists (e.g., “Weekly groceries,” “Hardware store”). - Adding, checking off, and removing items, with changes visible to every

other person viewing the same list within a second or two, with no manual refresh. - Basic authentication — each person has an account; a list can be shared by invite.

Stretch goals: - Show a small “Alex is typing...” indicator while someone is adding an item. - Group items by store aisle/category automatically based on a simple keyword mapping.

What this exercises: authentication and authorization (Chapter 14), SignalR groups (Chapter 17) for scoping updates to one shared list rather than broadcasting globally, and state management across components watching the same live data (Chapter 11).

E.3 Project 3: Personal Finance Dashboard (reinforces Parts III–IV, full stack)

A single-user application for logging expenses and visualizing spending by category.

Required features: - Manual expense entry: amount, category, date, optional note. - A monthly summary view, totaled by category. - CSV import of a bank statement export (a real, if slightly messy, data-parsing challenge). - The full pipeline: EF Core-backed persistence (Chapter 10b), authentication (Chapter 14), a test suite covering at minimum the CSV parsing logic and one full bUnit component test (Chapter 16), and a working CI pipeline (Chapter 20) that runs those tests on every push.

Stretch goals: - A budget-vs-actual view per category, with a visual warning when a category is over budget. - Deploy it for real, using Chapter 21’s guidance, to any target that fits your render-mode choice — this is the project worth actually putting somewhere reachable by a URL, even briefly.

What this exercises: genuinely everything in this book, end to end, applied to a domain the earlier chapters never touched — which is exactly the point. If you can specify, build, test, and ship this third project largely without returning to earlier chapters for reference, you’ve internalized the material.

E.4 A note on scope

All three projects are deliberately small enough to finish in days, not months — resist the temptation to over-scope a personal project into something that never ships. A

finished, small, imperfect application teaches far more than an ambitious one abandoned at 60% complete.

F

Further Reading and Resources

F.1 Official documentation

- **Microsoft Learn — ASP.NET Core Blazor** (learn.microsoft.com/aspnet/core/blazor) is the canonical, actively-maintained reference for every API surface this book covers, and the correct first stop when something changes in a .NET version newer than this book.
- **The .NET API Browser** (learn.microsoft.com/dotnet/api) for exact signatures of any type mentioned in this book — `EventCallback<T>`, `RenderFragment`, `EditContext`, and everything else.
- **ASP.NET Core GitHub repository** (github.com/dotnet/aspnetcore) — issues and discussions here are where the most current, detailed conversations about edge cases and upcoming changes happen, straight from the team building the framework.

F.2 Component libraries worth evaluating before building your own

- **MudBlazor** — a large, actively maintained, free component library following Material Design, with data grids, date pickers, dialogs, and forms out of the box.
- **Radzen Blazor Components** — free, broad component coverage, paired with a visual Blazor Studio tool if you want a designer-assisted workflow.
- **Telerik UI for Blazor** and **Syncfusion Blazor Components** — mature, commercially-licensed libraries with enterprise support, particularly strong data-grid and charting components.

Evaluate one of these before hand-rolling a data grid, rich text editor, or chart component from scratch — there’s rarely a good reason to reinvent well-tested, accessibility-considered versions of these specific, hard-to-get-right components.

F.3 Testing tools referenced in this book

- **bUnit** (bunit.dev) — documentation, including advanced scenarios (testing components using cascading parameters, JS interop, and authorization) beyond Chapter 16’s introduction.
- **Playwright for .NET** (playwright.dev/dotnet) — the full API reference for browser automation, including mobile viewport emulation and network mocking.
- **axe-core** (github.com/dequelabs/axe-core) — the accessibility scanning engine referenced in the accessibility chapter, with a catalog of every rule it checks.

F.4 On architecture and broader software practice

The architectural ideas in Chapter 18 (Clean/Onion/Hexagonal Architecture) are not Blazor- or even .NET-specific — reading about them in their original, framework-agnostic form deepens the “why,” not just the “how”:

- Robert C. Martin’s writing on Clean Architecture — the dependency-rule framing used throughout Chapter 18 traces back to this body of work.
- Any solid resource on Domain-Driven Design (DDD) for further depth on structuring the Domain layer itself, once your application’s business rules grow more complex than TaskBoard’s.

F.5 Staying current

Blazor is still evolving meaningfully between major .NET versions — render modes (Chapter 13) themselves were a significant .NET 8 addition. Two low-effort habits keep you current without constant active research:

- Skim the **.NET release notes** for each annual major release (nearly always shipped in November) — they're written for exactly this purpose and rarely take more than twenty minutes to read in full.
- Follow the **ASP.NET Core team's official blog** for early, detailed previews of what's changing before it ships, which is far more useful than discovering a breaking change only after upgrading.